

Тестирование — это процесс анализа и проверки программного обеспечения, цель которого — найти несоответствия между ожидаемым и фактическим результатом.

Простыми словами, мы берем требования (ТЗ) — то есть *ожидаемый* результат, и сравниваем с тем, как программа работает в реальности — это *фактический* результат. Если они не совпадают, мы фиксируем баг. Глобальная цель моей работы как тестировщика — обнаружить эти ошибки как можно раньше, чтобы сэкономить деньги компании и выпустить качественный продукт, которым будет удобно пользоваться

При этом я понимаю, что современное тестирование — это не только поиск багов в готовом приложении, но и их предотвращение. Поэтому идеальный процесс тестирования для меня начинается ещё на этапе чтения требований и документации, до того как программисты написали первый код

«**Требования** — это спецификация продукта, которая четко описывает, как система должна функционировать и выглядеть. Для меня как для тестировщика требования — это главный ориентир, из которого я беру **Ожидаемый результат** для своих тестов. Без требований тестирование превращается в гадание. Требования помогают нам понять, работает ли программа так, как задумал бизнес, или в ней есть дефекты».

Как мы уже знаем, требования (как и само тестирование) делятся на два больших лагеря:

Функциональные — описывают возможности программы (что она умеет делать).

Функциональное ТЕСТИРОВАНИЕ (Практика вживую)

Определение: Это процесс, когда тестировщик руками или кодом проверяет, **работают ли эти самые функции** из ТЗ.

- **Пример работы QA:** Тестировщик открывает готовый сайт, нажимает на кнопку «Добавить в корзину» и смотрит: попал ли туда товар? Он проверяет выполнение *функционального требования*.

- 1.
2. **Нефункциональные** — описывают её свойства (скорость, безопасность, дизайн, удобство).

Нефункциональное ТЕСТИРОВАНИЕ (Практика вживую)

Определение: Это процесс, когда тестировщик проверяет **качество работы** системы под нагрузкой, на безопасность или удобство.

- **Пример работы QA:** Тестировщик запускает специальную программу (например, JMeter) и искусственно нагоняет на сайт 10 000 виртуальных пользователей. Он проверяет, выдержит ли сайт, то есть тестирует *нефункциональное требование*.

1. Testing (Тестирование)

Определение: Это непосредственный технический процесс проверки готового функционала сайта, цель которого — **найти баги**, сравнивая ожидаемый результат (из ТЗ) с фактическим (на экране).

Полный расклад:

Это работа «в полях», когда код уже написан программистами и передан тебе на проверку. Ты открываешь сайт в браузере, берешь в руки чек-лист и начинаешь кликать по кнопкам, вводить данные и смотреть, что происходит.

Пример для собеседования:

Разработчики сделали новую форму оплаты заказа картой.

- **Что делает тестировщик:** Он заходит на сайт, добавляет товар в корзину, переходит на форму оплаты, вводит номер тестовой карты, нажимает кнопку «Оплатить» и проверяет, списались ли деньги и создан ли заказ. Если вместо подтверждения оплаты сайт выдает белый экран — тестировщик фиксирует **баг**. Это чистый Testing.

2. QC (Quality Control / Контроль качества)

Определение: Это этап проверки, который включает в себя тестирование, но фокусируется на **оценке качества всего готового продукта** перед его выпуском. Ответ на вопрос: «Готова ли эта версия сайта к выходу в свет?».

Полный расклад:

QC — это уровень контроля качества на контрольно-пропускном пункте. Специалист уровня QC смотрит на общую картину: все ли важные тесты пройдены, критичны ли оставшиеся мелкие баги, соответствует ли продукт стандартам компании.

Пример для собеседования:

Завтра у интернет-магазина запланирован большой релиз — обновление личного кабинета.

- **Что происходит на уровне QC:** Руководитель команды тестирования (или старший QA) собирает все результаты тестов. Он видит: *«Мы провели 300 тестов. Нашли 5 багов, но они незначительные (например, съехал текст в подвале сайта). Все главные функции — оплата, выбор адреса, корзина — работают идеально. Продукт соответствует критериям качества компании. Мы подписываем документ и даем добро на выпуск обновления»*. Это Quality Control.

3. QA (Quality Assurance / Обеспечение качества)

Определение: Это глобальный процесс настройки и улучшения всей разработки, цель которого — **предотвратить появление багов** на самых ранних этапах, еще до того, как программисты приступят к коду.

Полный расклад:

QA настраивает процессы так, чтобы у программистов и аналитиков изначально было меньше шансов ошибиться. Это превентивная медицина для проекта.

Пример для собеседования:

В интернет-магазине постоянно случались баги: программисты делали скидки по промокодам, которые считались неправильно, потому что аналитики путано писали формулы в ТЗ.

- **Что делает QA-инженер:** Он меняет сам *процесс* работы. Он приходит к аналитикам и говорит: *«Давайте перед передачей задач в разработку мы (QA) будем проводить аудит ваших*

требований. Мы будем проверять формулы на бумаге». Также он договаривается с программистами: «Прежде чем отдавать нам код на тестирование, вы обязаны прогнать его через автоматический синтаксический чекер и показать код коллеге (сделать код-ревью)». Ошибки исправляются еще на этапе чертежей $\rightarrow$ качество растет. Это Quality Assurance.

Как выдать этот пример на собеседовании (Готовый спич)

Если тебя спросят: «В чем разница между QA, QC и Testing? Расскажи на примере сайта», отвечай так:

«Давайте представим процесс создания интернет-магазина.

- **Тестирование (Testing)** — это когда программист уже создал кнопку "Купить", а я, как тестировщик, нажимаю на неё, проверяю, добавляется ли товар в корзину и не списывается ли лишняя сумма. Моя цель здесь — найти конкретные баги в готовом коде.
- **Контроль качества (QC)** — это когда перед Черной пятницей мы оцениваем готовность всего интернет-магазина целиком. Мы смотрим на отчеты: прошла ли корзина тесты, выдержит ли сервер нагрузку, и принимаем решение — можно ли выпускать эту версию сайта для реальных покупателей.
- **Обеспечение качества (QA)** — это когда мы понимаем, что программисты часто допускают ошибки в расчете скидок в корзине, и меняем сам процесс работы: заставляем аналитиков писать ТЗ более прозрачно, внедряем проверку требований и настраиваем автоматические проверки кода у разработчиков, чтобы **предотвратить** появление багов в будущем.

Какие есть 7 принципов тестирования?

1. Тестирование показывает наличие дефектов, а не их отсутствие (Testing shows presence of defects, not their absence)

Определение: Тестирование может доказать, что в программе **есть баги**, но оно никогда не докажет, что багов там **вообще не осталось**.

Полный расклад:

Даже если ты проверил сайт вдоль и поперек и прогнал 10 000 тестов, и все они прошли успешно — это не значит, что сайт идеален. Это значит лишь то, что твои 10 000 тестов не смогли найти ошибку. Но на 10 001-м шаге или при специфическом сочетании условий баг всё равно может вылезти.

Пример:

Ты протестировал форму оплаты на сайте во всех популярных браузерах (Chrome, Safari, Edge). Всё работает. Но завтра придет пользователь со старого планшета через редкий браузер Opera Mini, и у него форма оплаты сломается.

2. Исчерпывающее тестирование недостижимо (Exhaustive testing is impossible)

Определение: Невозможно проверить **абсолютно все** комбинации данных, путей и сценариев в приложении.

Полный расклад:

Если пытаться проверить все возможные варианты ввода, то тестирование одного простого поля займет годы. Задача тестировщика — не проверить всё на свете, а с помощью специальных техник (мы разберем их позже) выбрать самые важные и опасные комбинации.

Пример:

В поле «Количество товара» в корзине можно ввести любое число. Ты не можешь проверить цифру 1, потом 2, потом 3... и так до миллиарда, а потом еще отрицательные числа и буквы. Ты проверяешь только несколько ключевых точек: 1, 0, 999 (максимум), -5 (негативный тест) и текст "пять".

3. Раннее тестирование (Early testing)

Определение: Процесс тестирования должен начинаться **на самых ранних этапах** разработки софта, лучше всего — с анализа требований.

Полный расклад:

Как мы уже знаем, найти и исправить ошибку в тексте требований (ТЗ) стоит копейки. А вот если эту ошибку аналитика запрограммируют, оформят дизайном и выпустят на сайт — её исправление обойдется компании в сотни раз дороже.

Пример:

Аналитик написал в ТЗ для интернет-магазина: *«При покупке двух товаров третий идет в подарок»*. Тестировщик прочитал это еще до разработки и спросил: *«А если в корзине три разных по цене товара, какой из них бесплатный — самый дешевый или самый дорогой?»*. Аналитик исправил ТЗ. Ошибка предотвращена на бумаге.

4. Скопление дефектов (Defects clustering)

Определение: Большая часть багов обычно скрывается **в нескольких ключевых модулях** программы. Это так называемый принцип Парето в IT: 80% ошибок сидят в 20% кода.

Полный расклад:

Если в каком-то модуле сайта постоянно находятся баги, это звоночек: скорее всего, этот кусок кода написан очень сложно, криво или программист там запутался. Если ты нашел в корзине 5 багов подряд, продолжай копать именно там — там точно сидят еще несколько штук.

Пример:

В интернет-магазине модуль «Каталог товаров» работает идеально, а вот в модуле «Расчет международной доставки и скидок» баг сидит на баге. Тестировщик направляет туда максимум своего внимания.

5. Парадокс пестицида (Pesticide paradox)

Определение: Если постоянно прогонять **одни и те же тесты**, со временем они перестают находить новые баги.

Полный расклад:

Аналогия из биологии: если травить тараканов одним и тем же ядом (пестицидом), у них выработается иммунитет. В IT то же самое: программисты со временем запоминают твои стандартные проверки и пишут код так, чтобы он их проходил. Тест-кейсы нужно постоянно обновлять, изменять и придумывать новые нестандартные сценарии.

Пример:

Если ты каждый день проверяешь форму регистрации, вводя только имя Иван и фамилию Иванов, ты никогда не узнаешь, что сайт ломается, если регистрируется человек с двойной фамилией (например, Мамин-Сибиряк) или с апострофом (д'Артаньян).

6. Тестирование зависит от контекста (Testing is context dependent)

Определение: Разные программы тестируются **совершенно по-разному**. Универсального подхода не существует.

Полный расклад:

Нельзя применять одни и те же методы к простому сайту-визитке и к сложному банковскому софту или медицинской программе. В одном месте важна скорость и красота, в другом — железобетонная надежность и точность до копейки.

Пример:

Если на сайте интернет-магазина одежды на полсекунды зависнет картинка с платьем — это неприятно, но не смертельно. Но если на полсекунды зависнет софт кардиостимулятора или система управления тормозами автомобиля — это катастрофа. Поэтому софт для машин тестируют в разы строже, по другим стандартам и инструментам.

7. Заблуждение об отсутствии ошибок (Absence-of-errors fallacy)

Определение: Найти и исправить все баги — **бессмысленно**, если сама программа в итоге получилась неудобной и ненужной пользователю.

Полный расклад:

Бывает так: тестировщики вылизали сайт до блеска, в коде нет ни одной ошибки, всё работает идеально по ТЗ. Но сам продукт сделан так неудобно, что пользователи не понимают, как им пользоваться, и уходят. Качество — это не просто отсутствие багов, это когда продукт решает задачу клиента.

Пример:

Вы создали интернет-магазин без единого бага. Но чтобы купить там обычную футболку, пользователю нужно заполнить 40 полей анкеты, подтвердить паспортные данные и ввести индекс вручную. Сайт работает без ошибок, но покупать там никто ничего не будет.

Что сказать на собеседовании

Если тебя попросят назвать 7 принципов тестирования, не обязательно тараторить их как робот. Назови 3-4 главных и объясни их суть:

«Я знаю 7 ключевых принципов тестирования. Самые важные из них, на мой взгляд:

1. **Исчерпывающее тестирование невозможно** — мы не можем проверить терабайты всех комбинаций данных, поэтому используем техники тест-дизайна.
2. **Тестирование зависит от контекста** — интернет-магазин и приложение для космического корабля проверяются по-разному.
3. **Парадокс пестицида** — тесты нужно постоянно обновлять, иначе они перестают находить новые баги.
4. И помню про **Заблуждение об отсутствии ошибок** — продукт должен быть в первую очередь полезным и удобным для человека, а не просто "безбаговым"».

Какие бывают уровни тестирования: Unit, Integration, System, Acceptance?

1. Unit-тестирование (Модульное / Компонентное)

Определение: Это проверка **микроскопических кусочков кода** (отдельных функций или методов) в полной изоляции от остальной программы.

Полный расклад:

Этим уровнем занимаются **сами программисты**, а не тестировщики. Написал разработчик математическую функцию — он тут же пишет микро-тест, который проверяет, правильно ли эта функция считает. Здесь нет интерфейса, картинок или баз данных. Только чистый код.

Пример:

Программист написал функцию калькулятора стоимости поездки: она должна брать расстояние в километрах и умножать на тариф (например, $\$1 \text{ км} = 20 \text{ рублей}$).

- **Unit-тест:** Автоматически подставляет в эту функцию цифру 5 и проверяет, что на выходе получилось ровно 100. Всё, функция изолированно работает верно.

2. Integration-тестирование (Интеграционное)

Определение: Это проверка того, как отдельные, уже проверенные модули программы **общаются и работают вместе**.

Полный расклад:

По отдельности функции работают шикарно, но когда мы пытаемся их соединить, они могут друг друга «не понять». Здесь тестировщик проверяет стыки, швы и передачу данных между разными частями системы или между приложением и базой данных.

Пример:

Мы соединяем модуль калькулятора стоимости с модулем платежного шлюза банка.

- **Интеграционный тест:** Мы проверяем: когда калькулятор насчитал 100 рублей, передается ли эта точная сумма в банк для списания? Не потерялись ли данные по дороге? Смог ли банк вернуть ответ: «*Деньги списаны*»?

3. System-тестирование (Системное)

Определение: Это проверка **всего приложения целиком** как единой монолитной системы.

Полный расклад:

Вот тут начинается главная работа классического тестировщика (QA). Программа полностью собрана, у неё есть дизайн, бэкенд и база данных. Мы больше не копаемся в коде. Мы открываем приложение на телефоне и проверяем сквозные сценарии (End-to-End) от начала и до конца, ведя себя как реальный пользователь.

Пример:

Тестировщик берет телефон, открывает приложение такси:

1. Вводит адрес «Дом» и «Работа».
2. Видит цену на экране, выбирает способ оплаты «Карта».
3. Нажимает кнопку «Заказать».
4. Проверяет, что заказ ушел, водителю пришло уведомление, а с карты списались деньги.

Мы проверили всю систему в сборе.

4. Acceptance-тестирование (Приемочное)

Определение: Это финальный этап тестирования, на котором проверяется, готов ли продукт к выпуску и решает ли он бизнес-задачи заказчика.

Полный расклад:

Здесь баги уже практически не ищут — их должны были вычистить на прошлых уровнях. Цель приемочного тестирования — вынести вердикт: «Бизнес доволен? Мы сделали именно то, что просил клиент?». Часто в этом тестировании участвуют не только QA, но и сами заказчики, аналитики или фокус-группа реальных пользователей (Бета-тестирование).

Пример:

Заказчик (владелец службы такси) садится вместе с командой разработки перед релизом. Он открывает приложение и проверяет: *«Так, функции работают. Но главное — удобно ли клиенту вызывать машину? Поймет ли обычный человек, как сменить тариф с Эконома на Комфорт? Да, всё интуитивно. Принимаю работу, выпускаем в App Store».*

Что сказать на собеседовании (Шпаргалка)

Если тебя попросят перечислить и объяснить уровни тестирования, отвечай так:

1. **Unit-тестирование** — проверка изолированных строчек и функций кода. Делается разработчиками.
2. **Интеграционное** — проверка того, как эти кусочки кода и разные модули взаимодействуют друг с другом.
3. **Системное** — это основной уровень работы QA, когда мы тестируем полностью собранное приложение от начала и до конца через интерфейс.
4. **Приемочное (Acceptance)** — финальная проверка продукта на соответствие бизнес-требованиям перед сдачей заказчику или релизом».

Что такое Smoke testing?

Smoke testing (Дымовое тестирование) — это самая первая, быстрая и поверхностная проверка приложения после сборки, задача которой — убедиться, что программа вообще запускается, не «дымит» и её главные функции работают.

В IT «дымовой тест» проводится каждый раз, когда программисты присылают тестировщикам новую сборку (версию) приложения.

- **Цель Smoke-теста:** Не найти все баги на свете, а понять — **можно ли вообще тратить время на глубокое тестирование этой сборки.**
- **Длительность:** Обычно это очень короткий набор тестов (от 5 до 15 минут).
- **Результат:**
 - Если Smoke-тест **пройден успешно**, сборка принимается в работу, и тестировщики начинают копать вглубь.
 - Если Smoke-тест **провален** (нашли критический баг в главной функции), тестирование останавливается. Сборка заворачивается обратно со словами: *«Ребята, у вас приложение падает при запуске, чините, мы эту версию тестировать не будем».*
- **«Smoke-тестирование** — это минимальный набор тестов на проверку работоспособности критически важных функций приложения. Мы проводим его каждый раз, когда получаем новую сборку от разработчиков.
- **Цель Smoke-теста** — быстро убедиться, что стабильна сама система и основные бизнес-пути работают. Если Smoke-тест падает, мы сразу возвращаем сборку разработчикам на доработку, не тратя время на детальное тестирование. Например, для интернет-магазина дымовым тестом будет проверка: запускается ли приложение, работает ли авторизация и можно ли дойти до оформления заказа».

Что такое Sanity testing?

Sanity-тестирование (Санитарное тестирование) — это очень быстрый и узконаправленный тест, который проводится после **исправления конкретного бага** или добавления мелкой фичи. Его цель — убедиться, что конкретная ошибка действительно исправлена, а логика программы в этом месте не «сошла с ума».

- **Smoke-тест** применяется на самых ранних, еще нестабильных сборках (Build 1, 2, 3), чтобы проверить систему в общем.
- **Sanity-тест** применяется на уже относительно стабильных сборках (Build 30, 31), когда проект готов, но в него внесли локальное исправление (Bug fix) и нужно точно проверить именно это место.

«Sanity-тестирование — это быстрый, сфокусированный тест, который мы проводим после исправления бага или выпуска небольшого обновления. Его цель — убедиться, что конкретная функция теперь работает корректно и изменения не сломали логику внутри этого модуля.

Главное отличие от **Smoke-тестирования** в том, что Smoke проверяет критически важные функции всей системы в ширину на новой сборке, чтобы понять, жива ли она вообще. А Sanity идет в глубину конкретной фичи на уже стабильном проекте. Например, если в интернет-магазине починили промокоды, то проверка работы именно этих промокодов и корзины — это Sanity-тест».

Что такое Regression testing?

«Регрессионное тестирование — это тип тестирования, который проводится после **любого изменения кода (исправление бага, добавление фичи, рефакторинг)**, чтобы убедиться, что существующий функционал приложения продолжает работать исправно и новые изменения не вызвали побочных дефектов в нетронутых участках системы.

Пример 1. Обновление дизайна (Новая фича)

В приложении онлайн-кинотеатра решили внедрить темную тему. Разработчики переписали стили интерфейса и выкатили обновление. Тестировщик включает темную тему — всё выглядит отлично. Но во

время **регрессионного теста** он заходит в плеер и понимает, что навигационная кнопка «Пауза» стала невидимой (белые иконки наложились на белый фон). Новая фишка сломала старый критический функционал.

Пример 2. Исправление бага

В приложении заказа такси был баг: если пользователь вводил промокод со спецсимволами (например, SUPER%), приложение вылетало. Разработчик исправил эту валидацию. Тестировщик проверил — промокод SUPER% теперь применяется отлично (это был ретест). Но во время **регресса** выяснилось, что из-за этой правки теперь вообще перестала работать привязка новых банковских карт — код валидации задел общий платежный шлюз.

«На проекте я использую все четыре вида. Когда разработчики присылают новую сборку, я сначала провожу Smoke-тест, чтобы понять, жива ли система вообще. Если она жива, и в ней чинили конкретный баг, я делаю Retesting (проверяю само исправление) и сразу за ним Sanity-тест (проверяю адекватность всего этого модуля). А перед тем, как катить всё обновление на пользователей, мы запускаем Регрессионное тестирование, чтобы убедиться, что эти правки не сломали другие, старые части приложения».

Что такое нагрузочное тестирование?

Скрипт ответа, который сокрушит собеседование

Вопрос тимлида: «Расскажи, какие виды тестирования производительности ты знаешь?»

Твой идеальный ответ:

«Я не просто перечисляю виды производительности списком, а разделяю все 6 типов на **три архитектурных вектора**, в зависимости от того, на какую часть системы мы давим.

Вектор первый: Пользовательская нагрузка (User Load)

Здесь мы проверяем, как веб-сервер и бэкенд справляются с потоком людей. Сюда входят три вида:

- **Load Testing (Нагрузочное):** Проверяем систему под плановым пиковым трафиком. Смотрим на базовые метрики: время отклика (Response Time) и количество запросов в секунду (RPS).
- **Stress Testing (Стресс):** Перешагиваем через лимиты ТЗ и намеренно "ломаем" систему, чтобы найти точку деградации производительности и посмотреть, как приложение восстанавливается после падения.
- **Spike Testing (Скачки):** Имитируем мгновенный, лавинообразный наплыв пользователей (например, в момент пуш-рассылки) и проверяем, не заклинит ли сервер от резкого удара.

Вектор второй: Данные и Время (Data & Time)

Здесь мы проверяем, как система дружит со временем и базами данных, даже если пользователей на сайте мало:

- **Endurance / Soak Testing (Выносливость):** Держим среднюю нагрузку долго — 24 или 48 часов. Тут мы ищем скрытые дефекты архитектуры: утечки памяти (Memory Leaks) или переполнение дискового пространства логами.
- **Volume Testing (Объемное):** Пользователей может быть всего трое, но мы забиваем базу данных миллионами тяжелых строк. Цель — проверить скорость работы SQL-запросов и правильность настройки индексов в БД.

Вектор третий: Инфраструктура (Infrastructure)

- **Scalability Testing (Масштабируемость):** Мы проверяем "эластичность" системы. Если идет наплыв людей и облако автоматически подключает новые серверные мощности — соразмерно ли растет скорость работы приложения, или кривой код нивелирует любое мощное железо?

Итог: На реальном проекте я бы не ограничивался одним "нагрузочным" тестом, а комбинировал эти подходы, собирая метрики через Prometheus и Grafana, чтобы выдать бизнесу полную картину стабильности продукта».

Что такое usability testing?

«**Usability testing** — это проверка интерфейса на удобство и простоту использования. Мы оцениваем приложение глазами конечного пользователя.

На собеседованиях я обычно разделяю три главные метрики, которые мы проверяем:

1. **Эффективность (Efficiency):** Сколько времени и кликов тратит пользователь, чтобы, например, оформить заказ в корзине.
2. **Ошибочность (Errors):** Как часто пользователи совершают случайные ошибки из-за запутанного дизайна (например, нажимают не ту кнопку, потому что они расположены слишком близко).
3. **Удовлетворенность (Satisfaction):** Насколько интерфейс визуально приятен и комфортен.

Я люблю приводить простой пример: сайт может быть написан без единого бага, но если на нем используется бледно-серый мелкий шрифт на белом фоне, то пользователи со слабым зрением просто не смогут им пользоваться. Юзабилити-тестирование помогает бизнесу не терять клиентов из-за плохого дизайна».

Когда мы тестируем удобство мобильного приложения, мы проверяем интерфейс по нескольким критериям:

1. **Заметность элементов:** Достаточно ли крупная кнопка «Вызвать такси»? Поймет ли бабушка, что на неё нужно нажать?
2. **Понятные тексты:** Если на экране возникнет ошибка, что там будет написано? Понятное «*У вас не привязана карта, выберите наличные*» или пугающий технический код «*Error 0x0045f: null pointer exception*»?
3. **Количество шагов:** Сколько кликов нужно сделать пользователю, чтобы заказать машину? Если для банальной поездки нужно пройти 7 экранов, ввести индекс, отчество и подтвердить почту — это ужасное юзабилити. В идеале всё должно решаться в 2–3 клика.

Что такое security testing?

«**Security testing** — это оценка защищенности системы от несанкционированного доступа, утечек данных

и вредоносных атак. Его главная цель — сохранить три базовых принципа безопасности: **Конфиденциальность** (чтобы данные не украли), **Целостность** (чтобы данные не подменили) и **Доступность** (чтобы сайт не упал от DDoS-атаки).

Как QA, на своем уровне я проверяю базовые вещи:

- Безопасность авторизации (сложность паролей, блокировка при переборе).
- Разграничение прав пользователей (чтобы обычный юзер не получил права админа через подмену ID в ссылке).
- Передаются ли личные данные в зашифрованном виде (по протоколу HTTPS, а не HTTP).
- • **Обычное тестирование:** Ты заходишь на сайт под своим логином `ivan_2026`, покупаешь пиццу «Пепперони», она падает в корзину, всё супер.
- • **Тестирование безопасности:** Ты, ради шутки или злого умысла, пробуешь в адресной строке браузера вместо своего ID (`site.com/user/ivan_2026`) вручную написать чужой (`site.com/user/petr_2026`). Если сайт взял и открыл тебе личный кабинет Петра с его домашним адресом и привязанной картой — это **огромная дыра в безопасности**.

Что такое localization testing?

Localization testing (L10n) — это проверка адаптации программного продукта под особенности конкретного целевого региона, включая языковые, культурные, юридические и технические стандарты.

На практике я разделяю проверки локализации на четыре критических блока:

1. **Лингвистический:** Проверка корректности перевода, контекста терминов (чтобы не было машинного перевода) и отсутствия непереуведенных строк (так называемого хардкода).
2. **Графический (UI):** Контроль за тем, чтобы более длинные или короткие слова на других языках не ломали верстку, не перекрывали кнопки и не вызывали искажения интерфейса.
3. **Региональные стандарты:** Проверка форматов даты, времени, разделителей тысяч и дробей в числах, правильного отображения местной валюты и адресов.
4. **Юридический:** Проверка соответствия местным законам (например, наличие политики GDPR для стран Евросоюза).

Также я отличаю локализацию от **интернационализации (I18n)**: интернационализация — это подготовка архитектуры кода к поддержке многих языков, а локализация — это непосредственная кастомизация продукта под конкретную страну».

Чем валидация отличается от верификации?

Верификация — это проверка продукта на соответствие спецификациям и требованиям. Мы отвечаем на вопрос: *"Правильно ли мы пишем код и соответствует ли он ТЗ?"*. Это процесс сверки с документами, который включает в себя статический анализ требований и ревью кода.

Валидация — это оценка готового продукта на соответствие нуждам конечного пользователя и бизнеса. Мы отвечаем на вопрос: *"То ли мы вообще сделали? Решает ли программа проблему юзера?"*. Это всегда динамический процесс проверки работающей системы.

На практике продукт может успешно пройти верификацию (быть сделан строго по ТЗ), но полностью провалить валидацию, если само ТЗ изначально было составлено неверно и не учло реальный пользовательский опыт».

Верификация (Verification)

Техническое задание (ТЗ), макеты, регламенты.

Как проверяется?

Чаще всего **статически**: вычиткой документов, ревью кода, проверкой архитектуры.

Когда начинается?

На самых ранних этапах проекта, когда кода еще даже нет.

Валидация (Validation)

Ожидания пользователя, здравый смысл, бизнес-логика.

Как проверяется? Всегда **динамически**: на реальном запущенном приложении.

Когда начинается? На поздних этапах, когда есть готовый работающий билд.

Что такое SDLC?

«**SDLC (Software Development Life Cycle)** — это период времени от момента принятия решения о создании программного продукта до момента его полного вывода из эксплуатации. Он состоит из шести классических этапов: сбор требований, проектирование, разработка, тестирование, релиз и поддержка.

Как QA-инженер, я понимаю, что **тестирование не должно начинаться только на четвертом этапе**, когда код уже написан. По канонам современного IT, тестирование должно внедряться в SDLC как можно раньше — еще на этапе анализа требований (мы тестируем ТЗ на полноту и непротиворечивость). Это помогает находить логические ошибки до того, как разработчики напишут кривой код, что экономит компании огромные деньги».

Что такое STLC?

«**STLC (Software Testing Life Cycle)** — это структурированный процесс тестирования, который состоит из шести последовательных этапов: анализ требований, планирование, разработка тест-кейсов, подготовка окружения, выполнение тестов и закрытие цикла с подведением итогов.

На реальных проектах эти этапы часто идут параллельно с разработкой. Для меня, как для QA, критически важен первый этап — **анализ требований**. Чем раньше мы найдем ошибку в ТЗ, тем дешевле она обойдется компании.

Вся суть STLC сводится к тому, что тестирование — это не просто хаотичный поиск багов на готовом продукте, а контролируемый процесс, который начинается одновременно со стартом проекта и заканчивается финальным отчетом перед релизом»

1. Анализ требований (Requirement Analysis)

Что ищем: Логические дыры, противоречия и неточности. Наша задача на этом этапе — задать аналитикам каверзные вопросы *до* того, как программисты напишут код с ошибками.

2. Планирование тестирования (Test Planning)

Что пишется: На этом этапе рождается **Тест-план (Test Plan)**. Это документ, который отвечает на вопросы: «*Что именно мы тестируем? Какими силами? Сколько времени это займет? И на каких устройствах мы будем проверять софт?*».

3. Разработка тест-дизайна (Test Case Development)

Что создается: Мы пишем **тест-кейсы (Test Cases)** и составляем **чек-листы**. Например: «*Шаг 1: Нажать кнопку X. Ожидаемый результат: открылось окно Y*». Когда код придет к нам, мы не будем тратить время на придумывание тестов, у нас всё будет готово.

4. Настройка тестовой среды (Test Environment Setup)

Что делаем: Мы проверяем, что у нас есть доступы к этому серверу, базы данных заполнены нужной тестовой информацией, а на телефонах установлены правильные тестовые сборки приложения.

5. Выполнение тестов (Test Execution)

Что делаем: Мы проходим по своим тест-кейсам. Если фактический результат не совпадает с ожидаемым — мы заводим **Баг-репорты (Bug Reports)** в Jira и отправляем их разработчикам. После починки мы делаем ретест и регресс.

6. Завершение цикла тестирования (Test Cycle Closure)

Что делаем: Мы собираем метрики и пишем **Отчет о результатах тестирования (Test Summary Report)** для бизнеса. В нем мы пишем: «*Было пройдено 150 тестов, найдено 30 багов, все критические исправлены. Продукт готов к релизу*».

Главное отличие в том, что SDLC отвечает на вопрос: «*Как нам в целом создать и запустить этот бизнес-продукт?*», а STLC отвечает на вопрос: «*Как нам максимально эффективно и качественно проверить то, что создается внутри SDLC?*». Они работают параллельно: пока разработчики пишут код в рамках SDLC, QA-команда пишет тест-кейсы в рамках STLC».

Что такое Agile?

«**Agile** — это гибкий подход к разработке программного обеспечения, противопоставленный классической каскадной модели (Waterfall). В его основе лежит итеративная разработка: продукт создается не целиком за один огромный цикл, а небольшими жизнеспособными частями (инкрементами), которые регулярно тестируются и демонстрируются заказчику.

Как тестировщик, я очень ценю Agile, потому что в таких командах QA подключается к работе с самых первых дней спринта. Мы тестируем требования, готовим тесты параллельно с разработкой и выдаем быструю обратную связь. Это позволяет не копить баги к концу проекта, а исправлять их сразу, пока код еще «свежий».

- **Agile** — это общая *религия / философия* (быть гибкими, общаться, выпускать продукт частыми релизами).

- **Scrum** и **Kanban** — это конкретные *инструменты и фреймворки*, с помощью которых эту религию внедряют на практике.

- В *Scrum* работа делится на жесткие отрезки времени (спринты по 2 недели), есть ежедневные созвоны и четкие роли (Скрам-мастер, Владелец продукта).
- В *Kanban* нет спринтов, там есть непрерывный поток задач на доске (To Do $\rightarrow$ In Progress $\rightarrow$ Testing $\rightarrow$ Done) и фокус на том, чтобы задачи не копились в одном столбце.

Что такое Scrum?

«**Scrum** — это итеративный фреймворк для управления разработкой в рамках философии Agile. Работа в Scrum строится короткими фиксированными циклами — спринтами. В Скраме есть три роли: Владелец продукта, Скрам-мастер и Команда разработки, а также четыре ключевых события: Планирование, Дейли, Демо и Ретроспектива.

Роль QA в Scrum-команде уникальна: тестировщик является полноценной частью команды разработки (Development Team) и разделяет общую ответственность за результат спринта.

Я участвую во всех активностях:

- **На Планирование спринта** я оцениваю задачи с точки зрения тестирования и помогаю декомпозировать их.
- **На Ежедневный созвон (Daily Scrum / «Дейли»)** я подсвечиваю проблемы со стендами или задержками фич.
- **На Обзор спринта (Sprint Review / «Демо»)** я часто помогаю демонстрировать готовый функционал.
- **На Ретроспектива (Sprint Retrospective / «Ретро»)** я предлагаю улучшения в процессах, чтобы повысить качество кода и оптимизировать STLC внутри спринта».

3 главные роли в Scrum

Product Owner (Владелец продукта):

Scrum Master (Скрам-мастер): **Он следит, чтобы все соблюдали правила Скрама, проводит созвоны и убирает любые препятствия с пути команды (например, если у тестировщика сломался компьютер, Скрам-мастер бежит решать эту проблему с сисадминами).**

Команда разработки (Developers & QA): **Программисты и тестировщики.**

3 главных артефакта Scrum

1. Product Backlog (Бэклог продукта)

Что это: Это единый, постоянно обновляемый список **вообще всех задач, фич, требований и багов**, которые нужно реализовать в продукте за всё время его существования.

- **Кто за него отвечает:** Исключительно Владелец продукта (Product Owner). Он расставляет задачи по приоритету — самые важные и ценные для бизнеса всегда лежат наверху.
- **Обязательство (Цель): Product Goal (Цель продукта).** Весь бэклог должен работать на одну глобальную цель (например, *«Запустить мобильное приложение такси и выйти на 10 000 поездок в сутки к концу года»*).

2. Sprint Backlog (Бэклог спринта)

Что это: Это список задач, который команда разработки выбрала из Бэклога продукта на **текущий двухнедельный спринт**, плюс план по их реализации (как именно мы будем их кодить и тестировать).

- **Кто за него отвечает:** Вся команда разработки. На планировании команда решает, сколько верхних задач из Бэклога продукта она способна «переварить» за 2 недели. Изменять Бэклог спринта во время самого спринта без ведома команды нельзя.

- **Обязательство (Цель): Sprint Goal (Цель спринта).** Это краткое описание того, ради чего мы вообще затеяли эти две недели работы. Например: *«Добавить в приложение оплату картой МИР»*. Если какая-то задача не помогает достичь этой цели, её можно отложить.

3. Increment (Инкремент продукта)

Что это: Это **реальный, работающий кусок программы**, который команда создала за текущий спринт, объединенный со всеми кусками кода, которые были написаны в предыдущих спринтах.

- **Главное правило:** Инкремент должен быть полностью готов к использованию. Нельзя сказать: *«Мы написали код, но не протестировали его — вот наш инкремент»*. Не протестированный код — это просто мусор, а не инкремент. К концу спринта эта фича должна без проблем запускаться на боевом сервере.
- **Обязательство (Цель): Definition of Done (DoD / Критерии готовности).** Это официальный чек-лист внутри команды, который определяет, что задача действительно завершена. Например: *код написан, прошел ревью, протестирован на тестовом стенде, багов Critical и High нет*. Если задача не соответствует DoD, она не попадает в инкремент.

5 официальных событий (церемоний) Scrum

1. Спринт (The Sprint)

Что это: Главное событие-контейнер. Это фиксированный отрезок времени, за который команда должна превратить идеи из бэклога в работающий и протестированный софт (инкремент).

- **Длительность (Timebox):** От 1 до 4 недель. Самый популярный стандарт в индустрии — **2 недели**. Как только один спринт заканчивается, следующий начинается мгновенно, без пауз и выходных дней между ними.

2. Планирование спринта (Sprint Planning)

Что это: Встреча в самый первый день спринта, где команда решает, что именно она успеет сделать за предстоящие две недели, и формулирует **Цель спринта (Sprint Goal)**.

- **Длительность:** Максимум **4 часа** для двухнедельного спринта (из расчета до 2 часов на каждую неделю итерации).
- **Роль QA:** Мы оцениваем задачи с точки зрения тестирования. Если разработчик говорит: *«Я напишу этот код за 2 часа»*, тестировщик должен вовремя вклиниться: *«Да, но проверять его мы будем два дня, потому что там задеты старые модули. Эту задачу нельзя брать на флажке спринта»*. Также на планировании мы помогаем сразу прописать базовые сценарии для проверки задач.

3. Ежедневный Скрам (Daily Scrum / Дейли)

Что это: Быстрая утренняя летучка команды разработки, цель которой — сверить часы, понять прогресс движения к Цели спринта и скорректировать план на ближайшие 24 часа.

- **Длительность:** Строго **15 минут** каждый день в одно и то же время.
- **Роль QA:** Тестировщик рассказывает, что он успел проверить за вчера, что берет в работу сегодня и — самое главное — подсвечивает проблемы. Например: *«Разработчик отдал задачу на тестирование, но на тестовом стенде она даже не запускается (упал билд). Задача заблокирована, мне нужна помощь DevOps или автора кода»*.

4. Обзор спринта (Sprint Review / Демо)

Что это: Встреча в конце спринта, где команда показывает реальный, работающий продукт Владелец Продукта (Product Owner), заказчикам и стейкхолдерам. Никаких презентаций и рассказов о «тяжелой работе» — только демонстрация живого софта.

- **Длительность:** Максимум **2 часа** для двухнедельного спринта.
- **Роль QA:** Иногда именно тестировщик «крутит» приложение на экране во время демонстрации, так как он знает все сценарии и закоулки новой фичи лучше всех. Мы также внимательно слушаем обратную связь от бизнеса: если заказчик на Демо говорит *«Ой, а я думал, эта кнопка будет работать иначе»*, мы сразу понимаем, что в следующем спринте ТЗ изменится, и готовимся к новым тестам.

5. Ретроспектива спринта (Sprint Retrospective / Ретро)

Что это: Финальное событие спринта. Внутренний разбор полетов команды. Здесь не обсуждают продукт, здесь **обсуждают процессы, инструменты и взаимоотношения людей**. Цель — придумать 1–3 конкретных улучшения, которые команда внедрит уже в следующем понедельничном спринте.

- **Длительность:** Максимум **1.5 часа** для двухнедельного спринта.
- **Роль QA:** Это важнейший инструмент для тестировщика, чтобы защитить качество. Если разработчики всю неделю закидывали вас задачами в последние 4 часа до релиза, именно на Ретро нужно вежливо, но твердо сказать: *«Ребята, у нас бутылочное горлышко на этапе QA. Давайте договоримся отдавать задачи на тест равномерно, иначе мы будем регулярно пропускать критические баги»*.

На всех этих встречах тестировщик — это активный участник, а не пассивный слушатель. На планировании я подсвечиваю риски тестирования, на дейли сообщаю о блокерах на стендах, на демо помогаю показать готовый функционал, а на ретроспективе защищаю процессы обеспечения качества (QA), чтобы команда работала эффективнее.

Kanban (Канбан)

«**Kanban** — это фреймворк в рамках методологии Agile, который сфокусирован на визуализации процессов и управлении потоком задач. В отличие от Scrum, здесь нет фиксированных итераций (спринтов) и специфических ролей вроде Скрам-мастера. Работа идет непрерывным потоком.

Ключевой инструмент здесь — **WIP-лимиты (Work in Progress)**, то есть ограничение на количество одновременно выполняемых задач в каждой колонке. Для меня, как для QA, Kanban удобен тем, что WIP-лимиты защищают этап тестирования от перегрузки и "бутылочных горлышек" (bottlenecks).

Мы поставляем ценность непрерывно (Continuous Delivery): как только задача прошла тестирование, она сразу может катиться в релиз. Основными метриками эффективности в Канбане для нас являются **Lead Time** и **Cycle Time** — мы стремимся уменьшать время прохождения задачи от бэклога до продакшена»

Что такое техники тест-дизайна?

Техники тест-дизайна — это методы создания тест-кейсов, которые позволяют минимизировать количество проверок, сохраняя при этом высокое тестовое покрытие.

На практике я чаще всего комбинирую две фундаментальные техники

1. Эквивалентное разделение (Equivalence Partitioning)

Суть: Мы берем бесконечный массив входных данных и разбиваем его на группы (классы). Мы исходим из предположения, что внутри одной группы программа ведет себя абсолютно одинаково. Если тест сработал для одного значения из группы, то для остальных он тоже сработает.

Реальный IT-пример: Поле ввода пароля при регистрации

Требование безопасности: Пароль должен содержать от 8 до 20 символов.

Пошаговый алгоритм:

1. Выделяем **валидный** (разрешенный) класс: строки длиной от 8 до 20 символов.
2. Выделяем **невалидные** (запрещенные) классы:
 - Слишком короткие строки: от 1 до 7 символов.
 - Слишком длинные строки: 21 и более символов.
 - Отдельный спец-класс: пустая строка (0 символов).
3. Выбираем по **одной** случайной точке из каждой группы для теста.

Набор тестов (Тест-кейсы):

- Тест 1: Пароль из 12 символов (Ожидаемый результат: Пароль принят).
- Тест 2: Пароль из 4 символов (Ожидаемый результат: Ошибка «Слишком короткий пароль»).
- Тест 3: Пароль из 30 символов (Ожидаемый результат: Ошибка «Превышена длина пароля»).
- Тест 4: Не вводить ничего (Ожидаемый результат: Ошибка «Поле обязательно для заполнения»).

2. Анализ граничных значений (Boundary Value Analysis)

Суть: Эта техника — верный спутник предыдущей. Баги обожают прятаться на краях диапазонов, потому что разработчики в коде часто путают знаки «меньше» ($\$<\$$) и «меньше или равно» ($\$\leq\$\$$). Мы тестируем сами границы и их ближайших соседей.

Реальный IT-пример: Форма заказа билетов на Авиасейлс

Требование: Один пассажир может забронировать за раз от 1 до 9 билетов.

Пошаговый алгоритм:

У нас есть две критические границы: **1** (нижняя) и **9** (верхняя). По классическому стандарту мы берем саму границу, шаг назад и шаг вперед (3 точки на границу).

Набор тестов (Тест-кейсы):

- **Исследуем нижнюю границу (1):**
 - Проверяем **0** билетов (Система должна выдать ошибку или заблокировать кнопку).
 - Проверяем **1** билет (Валидная граница — успешное бронирование).
 - Проверяем **2** билета (Внутри диапазона — успешное бронирование).
- **Исследуем верхнюю границу (9):**
 - Проверяем **8** билетов (Внутри диапазона — успешное бронирование).
 - Проверяем **9** билетов (Валидная граница — успешное бронирование).
 - Проверяем **10** билетов (Система должна выдать ошибку «Максимум 9 билетов»).

3. Таблицы принятия решений (Decision Tables)

Суть: Используется, когда поведение системы зависит от комбинации нескольких условий. Вместо того чтобы хаотично проверять разные варианты, мы составляем матрицу, где прописываем все логические исходы.

📖 Реальный IT-пример: Выдача онлайн-кредита в Финтех-приложении

Требования банка для одобрения заявки:

1. Возраст заявителя должен быть 21 год или старше.
2. Ежемесячный доход должен быть больше 50 000 рублей.
3. Кредитная история должна быть чистой.

✂ Пошаговый алгоритм:

Мы выписываем все условия и строим комбинации «Да / Нет» (Истина / Ложь). Если условий 3, то максимальное число комбинаций — $2^3 = 8$. Но мы можем их оптимизировать, если один критический фактор сразу отменяет всё.

Условия (Conditions)	Тест 1	Тест 2	Тест 3	Тест 4
1. Возраст ≥ 21 ?	Да	Нет	Да	Да
2. Доход > 50 к?	Да	Да	Нет	Да
3. Кредитная история ок?	Да	Да	Да	Нет
Действие (Actions)				
Одобрить кредит?	👉 Да	✗ Нет	✗ Нет	✗ Нет

4. Диаграмма переходов состояний (State Transition Testing)

Суть: Эта техника применяется для систем, которые ведут себя как «конечные автоматы». У объекта есть несколько фиксированных состояний (статусов), и он переходит из одного в другое только под воздействием определенных триггеров (событий).

📖 Реальный IT-пример: Заказ в интернет-магазине (Ozon / Wildberries)

Статусы заказа: Черновик $\rightarrow$ Оплачен $\rightarrow$ Собирается $\rightarrow$ В пути $\rightarrow$ Выдан.

✂ Пошаговый алгоритм:

1. Рисуем карту (граф) правильных переходов: из Черновика после оплаты мы переходим в статус Оплачен.
2. Составляем тесты на **позитивные переходы** (проверяем, что правильный бизнес-процесс работает).
3. Составляем тесты на **негативные переходы** (пытаемся перевести систему в статус в обход логики).

📋 Набор тестов (Тест-кейсы):

- Позитивный: Создать заказ $\rightarrow$ Оплатить $\rightarrow$ Убедиться, что статус сменился на Оплачен.
- Негативный: Создать заказ $\rightarrow$ НЕ оплачивать $\rightarrow$ Попробовать вызвать API-команду «Передать курьеру». (Ожидаемый результат: Система блокирует действие, статус остается Черновик).

5. Парное тестирование (Pairwise Testing)

Суть: Магия оптимизации комбинаторики. Большинство багов вызываются сочетанием максимум **двух факторов**. Вместо того чтобы перебирать сотни комбинаций всех настроек со всеми, мы тестируем только такие комбинации, где каждая пара значений пересекается друг с другом хотя бы один раз.

📁 Реальный IT-пример: Форма фильтрации билетов на сайте РЖД

У нас есть фильтры для поиска поезда:

1. **Тип вагона:** Купе, Плацкарт, СВ (3 варианта)
2. **Время отправления:** Утро, День, Вечер (3 варианта)
3. **Опции:** Можно с животными, Только с кондиционером (2 варианта)

Если проверять всё: $3 \times 3 \times 2 = 18$ тестов. А если добавить туда еще 5 фильтров, тестов станет больше 500.

✂ Пошаговый алгоритм:

Мы не пишем тесты руками. Мы берем специальный инструмент (например, бесплатную программу *PiCT* от Microsoft или онлайн-генератор Pairwise) и вбиваем туда наши параметры. Утилита выдает оптимизированную таблицу, где тестов будет в два раза меньше (в данном случае — всего 9 вместо 18), но абсолютно все пары (например, «Купе + С животными», «Плацкарт + Утро») будут проверены.

📋 Пример оптимизированной матрицы тестов:

- Тест 1: Купе + Утро + С животными
- Тест 2: Купе + День + С кондиционером
- Тест 3: Купе + Вечер + С кондиционером
- Тест 4: Плацкарт + Утро + С кондиционером
- ... и так далее до 9 тестов.

Что такое предугадывание ошибок?

Предугадывание ошибок (Error Guessing) — это особая техника тест-дизайна, которая основана не на математических формулах или строгих алгоритмах, а на опыте, интуиции и знаниях тестировщика.

Топ-5 классических приемов «Предугадывания ошибок» в IT

Вот что делает опытный тестировщик, когда включает эту технику:

1. Бешеное мультикликание (Double Spend / Multi-click)

- **Гипотеза:** Разработчик мог забыть заблокировать кнопку отправки формы после первого нажатия.
- **Что делаем:** Нажимаем на кнопку «Оплатить» или «Заказать» быстро-быстро пять раз подряд.
- **Ожидаемый баг:** С карточки пользователя списываются деньги пять раз или создается пять одинаковых заказов.

2. Ввод «запрещенки» и спецсимволов

- **Гипотеза:** Программисты часто проверяют поля на буквы и цифры, но забывают про кодировки и спецсимволы.
- **Что делаем:** В поле «Имя» вставляем эмодзи (👊🔥), иероглифы, кавычки (" или ') или кусок кода (`<script>alert(1)</script>`).
- **Ожидаемый баг:** Сайт ломается, верстка «плывет» или база данных выдает внутреннюю ошибку (Error 500).

3. Стресс-тест буфера обмена (Copy-Paste)

- **Гипотеза:** На поле ввода стоит ограничение «не более 10 символов», но оно работает только тогда, когда пользователь печатает руками с клавиатуры.
- **Что делаем:** Копируем огромный текст из Википедии на 10 000 слов и вставляем его в поле через Ctrl+V.
- **Ожидаемый баг:** Система не справляется с объемом, не обрезает текст и падает.

4. Нарушение стабильности среды (Условия прерывания)

- **Гипотеза:** Код отлично работает в идеальных условиях, но ломается при внешних помехах.
- **Что делаем:** Запускаем тяжелое скачивание файла в приложении и прямо посреди процесса выключаем Wi-Fi или переводим телефон в авиарежим.
- **Ожидаемый баг:** Приложение намертво зависает или «вылетает» (крашится) вместо того, чтобы вежливо сказать: «Упс, пропал интернет».

5. Пробелы-невидимки

- **Гипотеза:** Разработчик забыл настроить очистку строки от лишних пробелов (Trimming).
- **Что делаем:** При логине в поле email вводим test@mail.ru (с кучей пробелов до и после адреса).
- **Ожидаемый баг:** Система скажет «Пользователь не найден», потому что будет искать email вместе с пробелами.

Чек-лист

Чек-лист — это высокоуровневый документ тестирования, который содержит краткий перечень проверок без пошагового описания и ожидаемых результатов.

Я выберу чек-лист вместо тест-кейсов в нескольких ситуациях:

1. Если на проекте **жесткие ограничения по времени** (например, мы работаем по быстрым спринтам в Scrum) и тратить время на написание детальных тест-кейсов нерентабельно.
2. Если **требования часто меняются** и поддерживать базу тест-кейсов в актуальном состоянии слишком дорого для бизнеса.
3. Если команда тестирования хорошо знает продукт и им не нужны пошаговые инструкции для выполнения базовых проверок.

Реальный IT-пример: Чек-лист для формы входа (Login Form)

Вот как выглядит реальный рабочий чек-лист тестировщика для проверки окна авторизации на сайте:

- ☐ Авторизация с валидными (правильными) данными.
- ☐ Авторизация с невалидным паролем.
- ☐ Авторизация с незарегистрированным email.
- ☐ Проверка обязательности полей (попытка войти с пустыми полями).
- ☐ Маскирование пароля (символы при вводе заменяются на точки ***).
- ☐ Работа ссылки «Забыли пароль?».
- ☐ Валидация формата email (попытка ввести адрес без знака @).

Когда тестировщик проходит эти проверки, он просто ставит статус: **Passed** (Пройдено) или **Failed** (Упало — если найден баг).

❓ Плюсы и минусы чек-листов

👉 Плюсы:

1. **Скорость:** В условиях жестких дедлайнов (например, в Scrum) писать подробные тест-кейсы часто просто некогда. Чек-лист пишется за 10–15 минут.
2. **Легкость поддержки:** Если на сайте кнопка «Войти» переехала из левого угла в правый, тест-кейс придется переписывать (менять шаги), а в чек-листе пункт «Авторизация с валидными данными» останется неизменным.
3. **Свобода мысли:** Чек-лист не загоняет тебя в жесткие рамки шагов, позволяя по ходу дела применять технику *предугадывания ошибок*.

👉 Минусы:

1. **Субъективность:** Если чек-лист напишет Senior-тестировщик («Проверить API-токен»), пришедший на проект Джун может просто не понять, как именно это сделать, क्योंकि в документе нет шагов.
2. **Сложность автоматизации:** Автоматизаторам (QA Automation) гораздо удобнее писать автотесты по детальным тест-кейсам, а не по кратким тезисам чек-листа.

Тест-кейс

Тест-кейс (Test Case) — это детальный сценарий проверки программы, который содержит пошаговую инструкцию: что нужно сделать и какой конкретно результат мы ожидаем получить.

Вот тот же самый тест-кейс, но в **компактном (Agile) формате**. В реальных проектах, где нет времени

расписывать каждый чих, QA часто используют именно такой сжатый вид: шаги объединяются по смыслу, а ожидаемый результат пишется один раз в конце.

ТС-AUTH-042: Восстановление пароля зарегистрированного пользователя

- **Приоритет:** High
- **Предусловия:** Создан юзер qa_test_2026@example.com (пароль OldPassword123!). Открыта страница [shop2026.com/login] (https://shop2026.com/login). Есть доступ к тестовой почте.

#	Шаги (Actions)	Ожидаемый результат (Expected Result)
1	Кликнуть «Забыли пароль?», ввести email qa_test_2026@example.com и нажать «Отправить ссылку».	Появилось зеленое уведомление об отправке. На почту пришло письмо со ссылкой для сброса.
2	Перейти по ссылке из письма, ввести во оба поля новый пароль NewPassword2026! и нажать «Сохранить».	Пароль изменен. Произошел автоматический редирект на страницу /login.
3	Авторизоваться с новым паролем NewPassword2026!.	Успешный вход. Открылся Личный кабинет пользователя (/profile).

- **Постусловия:** В БД вернуть юзеру исходный пароль OldPassword123!.

Баг-репорт

«Баг-репорт — это официальный артефакт тестирования, цель которого — локализовать дефект, зафиксировать его в таск-трекере (например, в Jira) и предоставить разработчику всю необходимую информацию для его исправления.

Чтобы баг-репорт был качественным, я всегда следую правилу "**Что? Где? Когда?**" при написании заголовка. Обязательно указываю параметры окружения (Environment) и пишу атомарные, понятные шаги воспроизведения.

Из чего состоит правильный баг-репорт

1. Тема / Заголовок (Summary)

- *Плохо:* «Не работает кнопка оплаты».
- *Хорошо:* «Ошибка 500 при нажатии кнопки "Оплатить картой" в корзине пустых товаров».

2. Окружение (Environment)

- *ОС:* iOS 17.4
- *Браузер / Стенд:* Safari v17 / Тестовый стенд (Staging-2)
- *Версия приложения:* v1.51

3. Шаги воспроизведения (Steps to Reproduce / STR)

- Открыть сайт shop.com.
- Добавить любой товар в корзину.

- Перейти в корзину и нажать кнопку «Удалить товар».

4. Ожидаемый результат (Expected Result)

Как система должна работать по ТЗ.

- *Пример:* Товар удаляется из корзины, появляется надпись «Ваша корзина пуста».

5. Фактический результат (Actual Result)

Что произошло на самом деле. То место, где всё сломалось.

- *Пример:* Товар не удаляется, страница зависает, в консоли браузера появляется ошибка 500 Internal Server Error.

6. Вложения (Attachments)

Разработчики не верят на слово — им нужны доказательства. Мы обязательно прикрепляем:

- **Скриншот** (если баг визуальный — например, кнопка съехала).
- **Видеозапись экрана** (если баг в динамике — например, форма зависает).
- **Логи** (текстовые файлы с сервера или консоли, где записана техническая ошибка).

7. Серьезность (Severity) и Приоритет (Priority)

- **Severity:** Насколько сильно баг ломает логику программы (например, *Blocker* — если приложение вообще не запускается, или *Trivial* — если на экране просто опечатка в слове).
- **Priority:** Как быстро этот баг нужно починить бизнесу (High, Medium, Low).

✂ Severity (Серьезность) — Технический взгляд

Severity определяет, **насколько сильно баг ломает техническую логику приложения** и мешает ли он дальнейшему тестированию. Эту оценку выставляет **тестировщик**, потому что именно он знает архитектуру и требования.

По международным стандартам (ISTQB) Severity делится на 5 уровней:

- **S1. Blocker (Блокирующий):** Система полностью парализована. Дальнейшее тестирование невозможно.
 - *Пример:* Приложение «падает» с белым экраном сразу при запуске. Или кнопка «Войти» не нажимается, и никто не может попасть внутрь.
- **S2. Critical (Критический):** Сломана ключевая бизнес-логика, и **нет обходного пути (workaround)**, но остальная часть приложения работает.
 - *Пример:* В интернет-магазине можно ходить по каталогу, но при попытке оплатить картой корзина очищается, а деньги со счета списываются в пустоту.
- **S3. Major (Значительный):** Фича работает неправильно, но её можно использовать через «костыль» (обходной путь). Либо сломана важная, но не основная функция.
 - *Пример:* Кнопка «Поделиться в Telegram» выдает ошибку. Пользоваться сайтом можно, покупать можно, но фича твиттера/телеграма сломана.
- **S4. Minor (Минорный):** Мелкая проблема с интерфейсом (UI), которая никак не влияет на функционал.
 - *Пример:* Картинка товара съехала на три пикселя влево и немного перекрывает текст описания.
- **S5. Trivial (Тривиальный):** Почти незаметный баг, опечатка или эстетический дефект.
 - *Пример:* В самом низу сайта (в футере) в слове «Контакты» пропущена буква «о» («Кнтакты»).

Priority определяет, **как быстро этот баг нужно починить команде разработки**. Этот параметр выставляет **Владелец продукта (Product Owner)** или **Тимлид**, ориентируясь на планы бизнеса, репутацию компании и дедлайны релиза.

Здесь градация обычно проще:

- **P1. High / Urgent (Высокий):** Бросаем все текущие задачи и чиним прямо сейчас. Выкатываем срочный фикс (hotfix) на боевой сервер.
- **P2. Medium (Средний):** Баг неприятный, но не горит. Берем в работу планово, починим в рамках текущего двухнедельного спринта.
- **P3. Low (Низкий):** Пустяковый баг. Складываем в бэклог. Починим, когда у разработчиков будет свободное время (то есть, скорее всего, никогда).

Жизненный цикл дефекта

«**Жизненный цикл дефекта** — это транзиты статусов баг-репорта в Jira от момента его создания до полного закрытия.

Основной "позитивный" путь включает в себя статусы **New** (когда я только завел баг), **Assigned** (передан конкретному разработчику), **Fixed** (код исправлен разработчиком), **Retest** (я заново перепроверяю фичу на тестовом стенде) и, если проверка успешна, **Verified** и **Closed**.

Однако в работе важны и другие статусы. Если разработчик не до конца исправил дефект, я перевожу его в **Reopened**. Если я ошибся в требованиях или завел задачу-повторку, баг получит статус **Rejected** или **Duplicate**. Для некритичных дефектов, исправление которых переносится из-за жестких дедлайнов бизнеса, используется статус **Deferred**».

Тест-план (Test Plan)

Из чего состоит реальный IT тест-план?

Современные тест-планы в Agile-командах не пишутся в виде 100-страничных талмудов. Обычно это лаконичный документ в Confluence (базе знаний проекта), состоящий из следующих ключевых блоков:

1. Область тестирования (Scope)

Здесь жестко очерчиваются границы, чтобы менеджеры не требовали проверить то, чего нет в ТЗ.

- **Что тестируем (In Scope):** Новый личный кабинет пользователя, регистрацию через Госуслуги, интеграцию с платежным шлюзом.
- **Что НЕ тестируем (Out of Scope):** Мобильное приложение (оно будет в следующем релизе), старый дизайн главной страницы.

2. Стратегия и подходы (Approach / Testing Types)

Какие именно виды тестирования будут применяться.

- *Пример:* На этом проекте мы проводим ручное функциональное тестирование, кроссбраузерное тестирование (в Chrome, Safari, Edge), а также автоматизированное API-тестирование (на Python + PyTest) для бэкенда. Нагрузочное тестирование проводить не будем (нет бюджета).

3. Критерии (Entry & Exit Criteria)

Это правила игры, которые защищают тестировщиков от хаоса.

- **Критерии начала (Entry Criteria):** Когда мы имеем право начать тесты? *Пример: Разработчики закончили код фичи, код прошел Code Review, и билд успешно развернут на Staging-сервере.*
- **Критерии окончания (Exit Criteria):** Когда мы можем выдохнуть и отдать продукт на релиз? *Пример: Выполнены все 100% критических тест-кейсов, в системе не осталось ни одного открытого бага с Severity Blocker и Critical, а количество минорных багов не превышает 5 штук.*

4. Ресурсы и Окружение (Resources & Environments)

- **Человеческие ресурсы:** На проекте работают 2 ручных тестировщика (Аня и Макс) и 1 автоматизатор (Денис).
- **Технические ресурсы (Железо и Стенды):** Нам нужны тестовый сервер QA-stand-3, база данных с тестовыми пользователями, физические смартфоны iPhone 14 и Samsung Galaxy S23.

5. График и Расписание (Schedule)

Конкретные дедлайны, завязанные на календарь.

- *Пример:* Анализ требований — до 15 числа, написание тест-кейсов — до 20 числа, проведение первого смоук-теста билда — 21 числа, регрессионное тестирование — с 22 по 28 число.

6. Риски (Risks & Contingencies)

Профессиональный QA-лид всегда думает о худшем и прописывает план Б.

- *Риск:* Сторонняя платежная система (шлюз), с которой мы интегрируемся, может упасть во время тестов.
- *Решение (План Б):* Разработчики должны написать «заглушку» (Mock-сервер), которая будет имитировать ответы платежки, чтобы мы не простаивали.

Матрица трассировки (RTM)

«Матрица трассировки (RTM) — это двумерная таблица, предназначенная для отслеживания связей между требованиями к продукту, тест-кейсами и дефектами.

Мы используем её на проекте для решения трех ключевых задач:

1. **Проверка тестового покрытия (Test Coverage Check):** Она гарантирует, что ни одно требование аналитиков не осталось без внимания и для каждого написан как минимум один тест.
2. **Анализ влияния изменений (Impact Analysis):** Если бизнес меняет одно из требований, с помощью матрицы мы мгновенно видим, какие именно тест-кейсы нам нужно актуализировать.
3. **Прозрачность для бизнеса:** Матрица позволяет наглядно показать заказчику, какие бизнес-требования уже успешно протестированы, а какие заблокированы открытыми багами».

Реальный IT-пример матрицы трассировки

Представь, что аналитики принесли три требования для интернет-магазина. Тестировщики написали тест-кейсы, провели тесты и завели найденные баги.

Вот как связываются эти данные внутри матрицы трассировки:

ID Требования	Описание требования (ТЗ)	ID Тест-кейса	Статус теста	ID Найденного Бага
REQ-01	Пользователь должен иметь возможность войти по номеру телефона.	TC-AUTH-01	Passed	Нет багов
		TC-AUTH-02	Passed	
REQ-02	После ввода номера система отправляет SMS с 4-значным кодом.	TC-AUTH-03	Passed	BUG-88: SMS приходит с задержкой в 5 минут
		TC-AUTH-04	Failed	
REQ-03	Срок действия SMS-кода должен составлять ровно 3 минуты.	—	—	КРИТИЧЕСКИЙ ПРОПУСК

Какие инструменты используют тестировщики: Jira, Azure DevOps, YouTrack, Redmine, TestRail, Qase, TestIT, Figma, Miro, Lucidchart?

Все перечисленные тобой программы — это ключевой стек инструментов, с которым ручной или автоматизированный тестировщик (QA) сталкивается каждый день. Они делятся на 4 четкие категории в зависимости от задач.

1. Баг-трекеры и Системы управления задачами (Bug & Task Trackers)

Эти инструменты нужны для организации рабочего процесса всей IT-команды. Тестировщики используют их, чтобы регистрировать найденные дефекты, следить за ходом спринтов и проверять задачи, которые разработчики передали в тестирование.

- **Jira** — мировой индустриальный стандарт. Огромный комбайн, где настраивается любой рабочий процесс (Workflow).

- *Как использует QA:* Оформляет баг-репорты, привязывает их к задачам разработчиков, двигает карточки ошибок по колонкам доски (New $\rightarrow$ In Progress $\rightarrow$ Ready for Test $\rightarrow$ Closed).
- **Azure DevOps** — мощная экосистема от Microsoft, объединяющая в себе таск-трекер, репозитории кода и пайплайны сборки (CI/CD).
 - *Как использует QA:* Следит за релизами, проверяет, в какую именно сборку (билд) попал исправленный код, и запускает автотесты прямо из интерфейса системы.
- **YouTrack** — баг-трекер от JetBrains. Популярен благодаря высокой скорости работы, умному текстовому поиску и возможности полностью управлять процессом с клавиатуры (хоткеями).
 - *Как использует QA:* Быстро заводит дефекты и мгновенно фильтрует их сложными запросами (например, найти все критические баги бэкенда, созданные за неделю).
- **Redmine** — бесплатный open-source инструмент «старой школы». Простой, аскетичный, но гибкий за счет плагинов. Часто встречается в стартапах, веб-студиях или госсекторе.
 - *Как использует QA:* Заводит баги в базовых текстовых формах и ведет учет времени, потраченного на проверку конкретной фичи.

2. Системы управления тестированием (Test Management Systems — TMS)

Профессиональный софт, созданный специально для QA. Здесь хранится вся тестовая документация (библиотеки тест-сетов, тест-кейсы, чек-листы), и здесь же фиксируются результаты их прохождения.

- **TestRail** — классическая, одна из самых популярных коммерческих TMS в мире. Идеальна для крупных проектов со строгой иерархией тестов. Внедряет AI-помощников для генерации тестовых шагов.
 - *Как использует QA:* Группирует тесты по модулям, запускает прогоны (Test Runs), проставляя каждому тест-кейсу статус Passed (прошел) или Failed (упал), и выгружает графики покрытия для менеджеров.
- **Qase** — современная, быстрая облачная TMS с чистым интерфейсом. Популярна в Agile-командах за счет бесшовной интеграции с Jira и Slack.
 - *Как использует QA:* Пишет лаконичные чек-листы. Если тест падает, Qase позволяет в один клик отправить все шаги воспроизведения прямо в Jira, создавая готовый баг-репорт.
- **TestIT** — популярная коммерческая TMS, активно применяемая на русскоязычном рынке. Заточена под плотное объединение ручных проверок и автотестов.
 - *Как использует QA:* Формирует глобальные тест-планы, связывает требования аналитиков с тестами (строит матрицу трассировки) и собирает результаты прогонов автотестов из систем автоматизации.

3. Инструменты для работы с дизайном (UI/UX Design)

Тестировщик обязан проверять не только логику работы кода, но и визуальное соответствие интерфейса задумке дизайнера.

- **Figma** — главный инструмент UI/UX проектирования веб- и мобильных приложений.
 - *Как использует QA:* Открывает макет в режиме разработчика (Dev Mode), смотрит точные размеры элементов в пикселях (px), отступы (padding/margin), hex-коды цветов, шрифты и сверяет их с тем, что получилось на реальном сайте через DevTools браузера. Также проверяет тексты на опечатки до того, как они уйдут в код.

4. Визуализация и проектирование (Mind Mapping & Diagrams)

Эти инструменты нужны QA на этапе **тест-дизайна** — когда самого кода еще нет, но нужно разложить сложную логику новой фичи по полочкам, чтобы понять, какие тесты пригодятся.

- **Miro** — интерактивная онлайн-доска для совместной работы команд.

- *Как использует QA:* Рисует интеллект-карты (Mind Maps). Например, в центре пишет «Корзина», а от неё ведет ветки: «Валидация промокодов», «Способы оплаты», «Расчет доставки», расписывая позитивные и негативные сценарии. Это помогает визуально охватить всю фичу и не забыть логические проверки.
- **Lucidchart** — инструмент для создания строгих блок-схем, диаграмм процессов и UML-моделей.
 - *Как использует QA:* Строит схемы переходов состояний (State Transition Diagrams). Например, для статуса заказа: Создан $\rightarrow$ Оплачен $\rightarrow$ Доставляется $\rightarrow$ Выдан. На схеме тестировщик сразу видит, какие переходы легальны, а какие являются багом безопасности (например, если система позволит перевести заказ из статуса Создан сразу в Выдан, минуя оплату).

В реальном рабочем процессе все эти инструменты завязаны в единую цепочку: тестировщик изучает логику по схеме в **Lucidchart**, сверяет интерфейс с **Figma**, проходит тесты по чек-листу в **Qase**, а если что-то ломается — заводит карточку бага в **Jira**.

Клиент-серверная архитектура

«Клиент-серверная архитектура — это концепция разделения системы на клиентскую часть (интерфейс, принимающий ввод от пользователя) и серверную часть (бизнес-логику и работу с данными). Они общаются между собой по сети с помощью протоколов (например, HTTP/HTTPS) посредством циклов "запрос-ответ".

Как тестировщику, мне это понимание необходимо для локализации багов. Если на экране не отображается аватарка пользователя, я открываю вкладку **Network в DevTools** и смотрю:

- Если клиент отправил правильный запрос, а сервер вернул ошибку 500 Internal Server Error или пустой ответ — баг на стороне бэкенда (сервера).
- Если сервер вернул корректные данные (200 OK), но на экране ничего не поменялось или интерфейс поплыл — это баг на стороне фронтенда (клиента)».

Что такое HTTP и HTTPS?

«**HTTP** — это протокол прикладного уровня для передачи данных, который отправляет информацию в открытом виде через 80-й порт. **HTTPS** — это его защищенная модификация, работающая через порт 443, где данные шифруются на транспортном уровне с помощью протоколов SSL/TLS.

Как тестировщик, при проверке веб-приложений я всегда обращаю внимание на протокол. Например, если на форме регистрации или оплаты используется HTTP вместо HTTPS — это **критический баг безопасности (Security Bug)**, так как персональные данные и реквизиты карт пользователей могут быть перехвачены третьими лицами.

Также знание портов (80 и 443) необходимо мне при настройке инструментов перехвата трафика, таких как **Charles Proxy** или **Fiddler**, чтобы правильно настроить проксирование и установить SSL-сертификат для расшифровки HTTPS-пакетов».

Что такое DNS?

«**DNS** — это система, которая переводит буквенные адреса сайтов в цифровые IP-адреса, потому что компьютеры умеют общаться только по цифрам.

В работе тестировщика мне это нужно в двух случаях. Во-первых, если тестовый стенд по ссылке выдает ошибку "Сайт не найден", я проверяю, привязали ли вообще этот домен к IP-адресу сервера. Во-вторых, я использую файл `hosts` на своем ПК как локальный DNS. Прописывая туда IP-адрес конкретного

тестового сервера или компьютера разработчика, я могу перенаправлять свой браузер на нужную мне сборку сайта, не дожидаясь, пока изменения выкатят на весь интернет».

Пример 2: Секретный трюк с файлом `hosts` (Его обожают спрашивать!)

В твоём компьютере (внутри Windows или MacOS) есть один волшебный текстовый файл, который называется `hosts`.

Этот файл — это твоя **личная карманная записная книжка**, которая имеет приоритет над всей мировой справочной службой DNS.

Представь, что в официальной справочной написано, что ресторан «Ромашка» находится на улице Ленина. Но ты в своей личной записной книжке зачеркнул этот адрес и написал: *«Для меня ресторан Ромашка теперь находится у меня на даче»*. Теперь, когда ты будешь ехать в «Ромашку», твой навигатор повезет тебя на дачу, потому что твоя личная книжка для него важнее городской справочной.

Как это используют тестировщики в ИТ: Разработчик Вася сделал новую версию сайта у себя на компьютере (его локальный IP-адрес, например, 192.168.1.50). Настоящий, живой сайт компании (`myshop.ru`) работает на боевом сервере, и туда ходят реальные покупатели. Тебе нужно протестировать Васиные изменения, но так, чтобы обычные покупатели ничего не заметили.

Что ты делаешь?

1. Ты открываешь на своём компьютере файл `hosts`.
2. Дописываешь туда строчку: 192.168.1.50 `myshop.ru`.
3. Сохраняешь.

Что произошло? Теперь, когда лично ТЫ вводишь в браузере адрес `myshop.ru`, твой компьютер не идет в интернет-справочную. Он смотрит в файл `hosts` и открывает сайт с компьютера разработчика Васи. Ты спокойно тестируешь, а обычные люди со всего мира, вводя тот же адрес, попадают на старый рабочий сайт. Ты создал для себя безопасную песочницу для тестов.

Какие бывают HTTP-статус-коды?

HTTP-статус-коды (или коды ответов сервера) — это **короткие трехзначные числа, которые сервер отправляет твоему браузеру в ответ на любой запрос. Это цифровой «язык жестов», с помощью которого сервер сообщает, как прошли дела на его стороне.**

✂ Главные коды, которые Junior QA обязан знать наизусть

На собеседовании тимлид не будет спрашивать все 60+ существующих кодов. Но есть «золотая классика», которую используют на проектах каждый день.

Группа 2xx: Успех

- **200 OK** — Главный код интернета. Запрос прошел идеально, страница успешно загрузилась, товар добавился в корзину.
- **201 Created** — Успешно создано. Часто приходит в ответ на регистрацию. Сервер говорит: «Отлично, новый пользователь сохранен в базу данных».

Группа 3xx: Перенаправление

- **301 Moved Permanently** — Страница переехала навсегда. Например, компания сменила старый сайт `old-shop.ru` на `new-shop.ru`. Браузер автоматически перекинет тебя на новый адрес.

Группа 4xx: Виноват Клиент (Тестировщик или Юзер)

- **400 Bad Request** — Плохой, кривой запрос. Браузер отправил серверу какую-то кашу вместо четких данных.
 - *Пример на работе:* Ты тестируешь отправку анкеты и забыл заполнить обязательное поле «Имя», а твой код фронтенда пропустил это. Сервер ругается кодом 400.
- **401 Unauthorized** — Не авторизован.
 - *Пример на работе:* Ты пытаешься зайти в Личный кабинет, но не ввел логин и пароль. Сервер говорит: «Покажи паспорт, анонимов не пускаю».
- **403 Forbidden** — Запрещено (Доступ ограничен).
 - *Пример на работе:* Ты авторизовался как обычный покупатель, но пытаешься по прямой ссылке зайти в панель администратора сайта. Сервер тебя узнал, но говорит: «У тебя нет прав админа, пошел вон».
- **404 Not Found** — Не найдено. Самый известный код.
 - *Пример на работе:* Ты опечатался в адресе ссылки (`myshop.ru/katolog` вместо `/catalog`). Компьютер пришел на сервер, а сервера такой страницы никогда не было.

Группа 5xx: Виноват Бэкенд (Программист или Админ)

- **500 Internal Server Error** — Внутренняя ошибка сервера. Главный кошмар разработчиков и любимый улов тестировщика.
 - *Пример на работе:* Ты нажал кнопку «Оплатить», код сервера попытался сложить копейки с рублями, запутался и намертво завис. Сервер падает. **Это 100% баг бэкенда**, пишем баг-репорт на разработчика.
- **502 Bad Gateway** — Плохой шлюз.
 - *Пример на работе:* В компании есть главный сервер, который принимает запросы, и куча мелких серверов, которые считают логику. Главный сервер постучался к мелкому, а тот «лежит» и не отвечает.
- **504 Gateway Timeout** — Время ожидания ответа истекло.
 - *Пример на работе:* Ты нажал кнопку «Скачать отчет за 5 лет». Сервер начал собирать терабайты данных, думал 30 секунд, браузер устал ждать ответа и выдал эту ошибку.

Что такое cookies?

«**Cookies** — это небольшие текстовые файлы, которые сервер отправляет в браузер пользователя для сохранения состояния (State) между HTTP-запросами, так как сам протокол HTTP является stateless (не хранит состояние). В них хранятся ID сессий, настройки интерфейса или данные корзины.

В тестировании я проверяю куки через вкладку **Application в DevTools**. Я смотрю, как система ведет себя при изменении или удалении кук: корректно ли уничтожается сессия при нажатии на Logout, проставляются ли для кук флаги безопасности (например, `HttpOnly`, чтобы их нельзя было украсть через скрипты), и очищается ли корзина, если удалить куки вручную».

Что такое localStorage и sessionStorage?

«**localStorage и sessionStorage** — это механизмы веб-хранилища (Web Storage), которые позволяют сохранять данные в формате "ключ-значение" прямо на стороне клиента. Их ключевое отличие от кук — увеличенный объем памяти (до 5 МБ) и то, что эти данные не отправляются автоматически на сервер с каждым HTTP-запросом.

Разница между ними в том, что данные в **localStorage** персистентны, то есть хранятся бессрочно, пока их явно не удалят. А данные в **sessionStorage** существуют только в рамках текущей сессии вкладки браузера и уничтожаются в момент её закрытия.

В тестировании я проверяю эти хранилища через вкладку **Application** в **DevTools**. Я отслеживаю, корректно ли туда записываются флаги настроек, ID товаров, состояния интерфейса, и как приложение реагирует на искусственное изменение или очистку этих данных через панель разработчика».

Что такое WebSocket?

«**WebSocket** — это протокол двусторонней связи (Full-Duplex), который устанавливает одно постоянное TCP-соединение между клиентом и сервером. В отличие от классического HTTP, где общение идет строго по схеме "запрос-ответ", WebSocket позволяет серверу самостоятельно отправлять (push) данные клиенту без предварительного запроса, что критично для приложений реального времени (чаты, биржи, игры).

Как тестировщик, я проверяю работу сокетов во вкладке **Network** -> **WS** в **DevTools**. Там я анализирую вкладку **Messages**, отслеживая структуру и корректность отправляемых и принимаемых фреймов (данных). Также в сценарии тестирования веб-сокетов я обязательно включаю проверки на обработку разрыва соединения (как фронтенд обрабатывает потерю сети и восстанавливает ли сессию сокета автоматически)».

Где это применяется? (Примеры на проектах)

Веб-сокеты нужны везде, где данные на экране должны обновляться мгновенно, без перезагрузки страницы пользователем:

- **Мессенджеры и чаты (Telegram Web, WhatsApp Web):** Когда твой друг нажимает кнопку «Отправить», сообщение мгновенно прилетает на сервер, а сервер тут же «вытаскивает» его в сокет твоего браузера. Ты видишь текст сразу.
- **Онлайн-игры в браузере:** Каждое микродвижение твоего персонажа отправляется на сервер, а сервер транслирует координаты других игроков тебе в режиме реального времени.
- **Биржи и котировки акций / криптовалют:** Графики цены Биткоина или акций прыгают каждую миллисекунду. Бегать за ними обычными запросами компьютер бы устал, поэтому данные текут сплошным потоком через сокет.
- **Совместная работа (Miro, Google Документы):** Когда твой коллега двигает карточку на доске Miro или пишет букву в документе, ты видишь курсор его мыши прямо сейчас.

Как тестировщик проверяет WebSocket?

Тимлид на собеседовании спросит: «Отлично, теорию ты знаешь. А где в браузере ты будешь смотреть эти сокеты, если чат сломался?».

Твои действия на проекте:

1. Открываешь панель разработчика (F12), переходишь во вкладку **Network** (Сеть).
2. В фильтрах сверху выбираешь круглую кнопку **WS** (это сокращение от WebSockets).
3. Перезагружаешь страницу чата. В списке появится строчка соединения.
4. Кликаешь на неё и переходишь на вкладку **Messages** (Сообщения).

Перед тобой откроется живой лог. Ты увидишь стрелочки:

- **Зеленые стрелочки вверх (\$\uparrow\$):** Это сообщения, которые твой браузер отправил на сервер (например, текст `{"message": "Привет!"}`).

- **Красные (или синие) стрелочки вниз (\$\downarrow\$):** Это то, что сервер прислал тебе обратно в реальном времени.

Что проверять QA-инженеру:

- **Баг «Чат завис»:** Пользователь пишет сообщение, оно уходит на сервер (есть стрелочка вверх), но в интерфейсе чата ничего не меняется. Ты смотришь в лог: если от сервера пришел ответ с ошибкой или не пришел вообще — баг на бэкенде.
- **Обрыв связи (Reconnection):** Что будет, если у тестировщика на секунду пропадет интернет (или он включит «Режим полета»)? Хорошо написанное приложение должно автоматически попытаться переоткрыть WebSocket-соединение (сделать Reconnect), как только сеть вернется, а не умирать навсегда.

Как проверять адаптивность без кода?

Что именно искать тестировщику во время этой проверки? (Чек-лист багов)

Когда ты сжал сайт до размеров телефона, внимательно посмотри на интерфейс и проверь:

1. **Нет ли горизонтального скrolла?** Покупатель на телефоне должен листать сайт только пальцем вверх и вниз. Если сайт начинает мотаться влево-вправо, а справа появилась пустая белая полоса — это баг (верстка «поплыла»).
2. **Кликабельны ли элементы пальцем?** На компьютере мы наводим на маленькую кнопку острым курсором мыши. На телефоне мы тыкаем в неё толстым большим пальцем. Если две кнопки или ссылки стоят вплотную друг к другу, пользователь промахнется. Расстояние между ними должно быть комфортным.
3. **Спряталось ли меню?** Огромная «шапка» сайта с кучей ссылок (О нас, Каталог, Доставка, Контакты) на мобилке должна красиво спрятаться в маленькую иконку с тремя полосками (её в IT называют «Гамбургер-меню»). Проверь, что она открывается и закрывается.
4. **Не налезли ли тексты друг на друга?** Часто бывает, что на телефоне огромный заголовок ломается на три строчки и закрывает собой картинку или кнопку «Купить».

Это чистой воды визуальное тестирование (UI-тестирование), и для него тебе нужен только твой зоркий глаз!

Какие вкладки DevTools нужны тестировщику?

«В ежедневной работе тестировщика я чаще всего использую четыре вкладки DevTools:

1. **Elements** — для инспектирования HTML-кода и CSS-стилей, верстки, а также проверки адаптивности.
2. **Console** — для отслеживания JS-ошибок. Если на странице что-то блокируется, я ищу там логи ошибок.
3. **Network** — для анализа клиент-серверного взаимодействия: проверяю статус-коды ответов (200, 404, 500), исследую Payload и Response, а также тестирую работу WebSocket-соединений.
4. **Application** — для проверки хранения данных на стороне клиента: смотрю содержимое Cookies, LocalStorage и SessionStorage.

Этого набора мне полностью хватает, чтобы локализовать практически любой баг на фронтенде или бэкенде».

Что такое GUI-проверки?

«**GUI-проверки** — это тестирование графического интерфейса пользователя. Их цель — убедиться, что визуальные элементы приложения (кнопки, формы, шрифты, отступы, картинки) отображаются корректно, находятся на своих местах и соответствуют требованиям или макетам из Figma.

В рамках GUI-тестирования я проверяю не только статическую картинку, но и динамику интерфейса: как элементы реагируют на наведение мыши (эффект *hover*), блокируются ли кнопки при невалидных данных, правильно ли работают всплывающие окна и не ломается ли расположение элементов при разных разрешениях экрана.

Для точных GUI-проверок я использую инструмент **Elements в DevTools**, чтобы сверять CSS-стили (цвета, размеры, отступы) напрямую с дизайн-системой».

Что именно проверяет QA-инженер во время GUI-тестов?

1. Визуальный дизайн (Внешний вид)

Мы открываем макет в Figma и сверяем его с реальным сайтом.

- Тот ли цвет у кнопок? (например, в Figma задан цвет #FF0000 — строго красный, а разработчик сделал малиновый).
- Правильные ли шрифты и их размеры?

2. Верстка и расположение (Layout)

- Находятся ли элементы на своих местах? (например, кнопка «Войти» должна быть в правом верхнем углу, а не по центру экрана).
- Ровные ли отступы между блоками карточек товаров?

3. Поведение и состояния элементов (Интерактивность)

Элементы на сайте не статичны, они должны реагировать на действия человека.

- **Ховер-эффект (Hover):** Меняет ли кнопка цвет, когда ты просто наводишь на неё курсор мыши? (Это подсказывает пользователю, что на кнопку можно нажать).
- **Активное состояние:** Нажимается ли кнопка визуально (имитация клика)?
- **Заблокированное состояние (Disabled):** Если пользователь не заполнил поле с телефоном, кнопка «Отправить» должна стать серой и блеклой, показывая, что сейчас она неактивна.

4. Навигация

- Открывается ли меню при клике на «гамбургер»?
- Закрывается ли всплывающее (модальное) окно, если нажать на крестик в углу или кликнуть по темному фону вокруг него?

Важное отличие: GUI против Usability (Юзабилити)

Начинающие тестировщики часто путают эти два понятия, а тимлиды любят ловить на этом на собеседованиях.

- **GUI-тестирование** отвечает на вопрос: «**Правильно ли это сделано по макету?**». Если дизайнер нарисовал ядовито-зеленую кнопку, на которой белым шрифтом написано «КУПИТЬ», и разработчик сделал точь-в-точь — GUI-тест пройден. Всё по инструкции.
- **Usability-тестирование** отвечает на вопрос: «**Удобно ли это человеку?**». Если ты открываешь эту ядовито-зеленую кнопку и понимаешь, что от белого текста на зеленом фоне через 3 секунды начинают болеть глаза и ничего не понятно — это проблема юзабилити (удобства использования), хотя по GUI-метрикам всё совпало.

Что такое PixelPerfect?

Pixel Perfect — это методология верстки и тестирования интерфейсов, цель которой — добиться максимального (в идеале попиксельного) соответствия готового сайта исходному дизайн-макету.

На практике для этого я использую специальные браузерные расширения, например, **PerfectPixel**. Я сохраняю макет из Figma в виде PNG-изображения, накладываю его полупрозрачным слоем поверх реального сайта в браузере и визуально оцениваю отклонения.

Этот метод позволяет мне быстро находить баги в микро-отступах, размерах шрифтов, смещениях иконок и передавать разработчикам точные указания, где именно верстка поплыла относительно макета».

Что такое API-тестирование?

«**API-тестирование** — это проверка программного интерфейса приложения, то есть бизнес-логики и интеграций на уровне обмена запросами и ответами, без участия графического интерфейса.

При тестировании API я проверяю, как сервер обрабатывает входящие HTTP-запросы: корректно ли приходят **статус-коды** (например, 200, 400, 401, 500), соответствует ли структура ответа **JSON-схеме** из документации, и правильно ли обрабатывает валидация на негативные сценарии.

В качестве основного инструмента я использую **Postman** для составления коллекций запросов, а требования к эндпоинтам изучаю в **Swagger**. Такое тестирование позволяет локализовать баги на бэкенде еще до того, как будет готова визуальная часть приложения».

Главные инструменты API-тестировщика

- **Postman / Insomnia:** Основные программы, где ручной тестировщик создает и отправляет запросы.
- **Swagger / OpenApi:** Интерактивная документация, которую пишут разработчики. Из неё мы узнаем, какие запросы сервер вообще умеет принимать.

Что именно проверяет QA в API?

1. **Функциональность:** Делает ли бэкенд то, что должен? (Считает ли корзина скидку правильно? Переводятся ли деньги между счетами?).
2. **Обработка ошибок:** Как сервер реагирует на плохие данные? Если отправить пустой пароль, вернет ли сервер понятную ошибку (например, статус 400 Bad Request и текст «Пароль слишком короткий»), или выдаст внутренний сбой?
3. **Безопасность (Security):** Можно ли получить данные чужого профиля, если просто подменить `userId` в запросе? (Это один из самых частых и опасных багов).
4. **Производительность:** Как быстро сервер отвечает на запрос? (Норма — до 200–500 миллисекунд. Если запрос идет 5 секунд — это баг).

Что такое REST API?

«**REST API** — это архитектурный стиль взаимодействия между клиентом и сервером, построенный на базе протокола HTTP. Его главная особенность в том, что система оперирует **ресурсами**, каждый из которых имеет свой уникальный URL-адрес (эндпоинт).

Для управления ресурсами REST использует стандартные HTTP-методы: **GET** для чтения, **POST** для создания, **PUT** или **PATCH** для обновления и **DELETE** для удаления данных. Также REST является **stateless-архитектурой**, то есть сервер не хранит состояние сессии клиента, и каждый запрос должен содержать всю необходимую информацию для его выполнения, включая токены авторизации.

В тестировании REST API я проверяю правильность интеграции методов, корректность возвращаемых HTTP-статус-кодов и соответствие JSON-ответов сервера заявленным требованиям в документации

(например, в Swagger)».

✂ 4 главных глагола REST API (То, что тестируют каждый день)

• GET (Прочитать / Получить данные)

- *Запрос:* GET /api/v1/products/5
- *Что делает:* Ты протискаешь этот запрос, и сервер просто присылает тебе в ответ JSON-описание товара под номером 5 (название, цену, описание). Сервер при этом ничего внутри себя не меняет.

• POST (Создать новое)

- *Запрос:* POST /api/v1/products + в тело запроса прикладываешь JSON с данными нового товара.
- *Что делает:* Сервер берет эти данные и сохраняет в базу новый товар. В ответ прилетает код 201 Created.

• PUT (Обновить / Отредактировать)

- *Запрос:* PUT /api/v1/products/5 + в тело прикладываешь новую цену.
- *Что делает:* Сервер находит старый товар №5 и заменяет в нем данные на новые (например, ты поменял цену с 500 на 600 рублей).

• DELETE (Удалить)

- *Запрос:* DELETE /api/v1/products/5
- *Что делает:* Сервер стирает этот товар из базы данных. Если ты после этого попробуешь сделать GET на этот же адрес, сервер честно ответит тебе ошибкой 404 Not Found.

Важное правило REST: Независимость (Stateless)

Еще одно золотое правило REST, про которое всегда спрашивают на собеседованиях — это **Stateless** (отсутствие состояния).

Это значит, что сервер «забывчивый». Он не помнит, что ты делал секунду назад. Каждый твой запрос для него — как первый раз в жизни.

- *Пример:* Ты не можешь сначала отправить запрос «Я Вася, пароль 123», а следующим запросом прислать просто «Покажи мои заказы». Сервер на втором запросе уже забудет, кто ты.
- *Как это решается:* В каждом REST-запросе клиент (браузер или Postman) обязан передавать всю информацию о себе «с нуля» — обычно в заголовках запроса передается специальный секретный ключ-паспорт (**Токен авторизации**). Сервер видит токен, понимает, что это Вася, и отдает данные.

Что такое SOAP?

«**SOAP** — это строгий протокол обмена данными, основанный на формате XML. В отличие от REST, который является гибким архитектурным стилем, SOAP требует жесткого соблюдения стандартов.

Каждый запрос в SOAP упаковывается в так называемый **конверт (Envelope)**, состоящий из заголовка (Header) и тела (Body), а вся структура сервиса жестко зафиксирована в файле описания **WSDL**.

SOAP чаще всего применяется в системах с повышенными требованиями к безопасности и надежности транзакций, например, в банковском секторе или платежных шлюзах. Для его тестирования я использую

специализированный инструмент **SoapUI**, куда импортирую WSDL-контракт, формирую XML-тела запросов и проверяю корректность ответов бэкенда и обработку ошибок в блоке Fault».

- **Идентификация** — «Как тебя зовут?» (Ты называешь свое имя / логин).
- **Аутентификация** — «Докажи, что это ты!» (Ты предъявляешь пароль / паспорт).
- **Авторизация** — «Что тебе здесь разрешено делать?» (Система проверяет твои права доступа).

Эти три процесса представляют собой последовательные этапы контроля доступа в любой безопасной системе:

1. **Идентификация** — это начальный этап, на котором пользователь заявляет о себе системе, сообщая свой уникальный идентификатор (например, логин или Email). Система проверяет, существует ли такой объект в базе.
2. **Аутентификация** — это следующий шаг, проверка подлинности заявленного пользователя. Система требует доказательств (пароль, токен, биометрия), чтобы убедиться, что логин вводит именно его владелец. В случае ошибки на этом этапе API возвращает статус **401 Unauthorized**.
3. **Авторизация** — это финальный этап, который происходит после успешного входа. Система проверяет права доступа и роль пользователя (например, admin или user) при его попытке обратиться к конкретному ресурсу. Если прав недостаточно, сервер выдает ошибку **403 Forbidden**.

В тестировании я проверяю всю цепочку: от валидации уникальности логинов при идентификации, до устойчивости к брутфорсу при аутентификации и строгого соблюдения матрицы прав ролей при авторизации»

Что такое Basic Auth?

Basic Auth (Базовая аутентификация) — это стандартный метод проверки подлинности, встроенный в протокол HTTP. Он работает через передачу учетных данных в заголовке запроса **Authorization**. Браузер соединяет логин и пароль через двоеточие, кодирует эту строку в формат **Base64** и отправляет на сервер.

Главный минус этого метода — отсутствие встроенного шифрования, так как Base64 легко декодируется обратно в исходный текст. Поэтому Basic Auth применим исключительно в связке с протоколом **HTTPS**.

В тестировании я проверяю Basic Auth через вкладку **Network** в **DevTools**, контролируя наличие заголовка `Authorization: Basic` в Request Headers, а также проверяю корректность обработки сервером статус-кода **401 Unauthorized** при отправке невалидных данных или отмене ввода».

Что такое Bearer Token?

Bearer Token — это тип доступа (Access Token), используемый в HTTP-аутентификации. Его название означает, что доступ к ресурсу предоставляется любому «предъявителю», который отправляет этот токен в заголовке запроса. Сам токен генерируется сервером после успешного логина пользователя (часто в формате JWT) и отправляется клиенту.

Клиент прикрепляет его к каждому последующему запросу в заголовке `Authorization: Bearer <token>`. Поскольку сам по себе токен не шифрует личность предъявителя при краже, его использование безопасно только поверх протокола **HTTPS** и с ограничением по времени жизни (TTL).

В тестировании API я проверяю Bearer Token с помощью **Postman** (через вкладку Authorization) или в **DevTools**, контролируя корректность статус-кодов. Основные проверки включают негативные сценарии: отправка запросов без токена, с измененным токеном, с истекшим сроком действия (проверка возврата **401 кодом**), а также попытка получить доступ к чужим ресурсам с валидным токеном другого юзера (ожидаем **403 код**).»

Как тестировщик проверяет Bearer Token?

1. В Postman (При тестировании API)

Тебе не нужно вручную прописывать заголовки. В Postman есть специальная вкладка:

1. Выбираешь нужный запрос к API.
2. Переходишь во вкладку **Authorization** (под адресной строкой).
3. В выпадающем списке **Type** выбираешь **Bearer Token**.
4. В поле **Token** справа просто вставляешь скопированный ключ. Postman сам упакует его в правильный HTTP-заголовок.

2. В браузере (Через DevTools)

1. Открой **DevTools** $\rightarrow$ вкладку **Network**.
2. Сделай любое действие на сайте (например, обнови профиль).
3. Кликни по этому запросу и найди раздел **Headers** $\rightarrow$ **Request Headers**.
4. Убедись, что там присутствует строка `Authorization: Bearer <твой_токен>`.

☐ Чек-лист негативных тестов для QA:

- **Тест на протухший токен:** Что будет, если отправить запрос с истекшим токеном? (Сервер должен вернуть 401 Unauthorized).
- **Тест на поломанный токен:** Что будет, если изменить в токене хотя бы одну букву или стереть её? (Сервер должен отказать в доступе, код 401).
- **Тест на отсутствие токена:** Что будет, если вообще убрать заголовок Authorization? (Код 401).
- **Тест на чужой токен (Авторизация):** Что будет, если Вася возьмет свой валидный токен, но попытается с ним удалить профиль Петры? (Сервер должен узнать Васю, но вернуть код **403 Forbidden** — доступ запрещен).
- **Только HTTPS:** Передавать Bearer-токены через незащищенный HTTP категорически запрещено. Хакер перехватит этот токен за секунду и получит полный контроль над аккаунтом жертвы.
- **Срок жизни (TTL — Time To Live):** У токена обязательно должен быть короткий срок действия (например, 15 минут или 1 час). Называется он Access Token. Когда его срок истекает, он превращается в «тыкву».
- **Обновление токена (Refresh Token):** Чтобы пользователю не приходилось вводить логин и пароль каждые 15 минут, сервер вместе с обычным токеном выдает второй — Refresh Token (он живет долго, например, месяц). Когда короткий токен умирает, приложение втихаря от пользователя отправляет длинный токен на сервер и получает взамен новую пару ключей.

Что такое OAuth 2.0?

«**OAuth 2.0** — это общепринятый протокол делегированной авторизации. Он позволяет стороннему приложению получить ограниченный доступ к защищенным ресурсам пользователя на другом сервере (например, Google или VK) без необходимости передавать приложению прямой логин и пароль.

Архитектура OAuth 2.0 включает четыре основные роли: **Resource Owner** (пользователь), **Client** (наше приложение), **Authorization Server** (сервер, проверяющий права) и **Resource Server** (сервер с данными). Результатом успешного прохождения сценария OAuth является выдача клиенту **Access Token** (чаще всего в формате Bearer/JWT), с которым приложение в дальнейшем ходит в API.

При тестировании OAuth 2.0 я в первую очередь проверяю обработку негативных сценариев: отмену авторизации пользователем, отзыв токена, поведение системы при истечении времени жизни токена (TTL), а также строгое соответствие запрашиваемых прав доступа (Scopes) требованиям безопасности».

Что такое GraphQL?

«**GraphQL** — это язык запросов и манипулирования данными для API, разработанный как альтернатива REST. Его ключевое отличие от REST заключается в том, что клиент сам строго определяет в запросе структуру и объем необходимых ему данных, избегая проблем over-fetching и under-fetching.

Вместо множества эндпоинтов GraphQL использует **один единый эндпоинт**, работающий через POST-запросы. Все операции делятся на три типа: **Queries** для чтения, **Mutations** для изменения данных и **Subscriptions** для получения обновлений в реальном времени.

Как тестировщик, при работе с GraphQL я помню о его главной особенности: практически любые ответы, включая ошибочные, возвращаются со статус-кодом **200 OK**. Поэтому в тест-кейсах я проверяю не HTTP-статусы, а тело JSON-ответа на наличие массива `errors`. Для тестирования и изучения схемы API я использую **Postman** или специализированные инструменты вроде **GraphQL Playground**».

Инструменты для тестирования GraphQL

- **Apollo Sandbox / GraphQL Playground / Altair:** Специальные интерактивные песочницы. Ты вставляешь туда URL, и они благодаря функции *интроспекции* автоматически подтягивают всю документацию (схему) сайта. Ты видишь все доступные поля и можешь кликами собирать запросы.
- **Postman:** В современных версиях Postman при создании запроса можно выбрать тип GraphQL вместо Raw/JSON, и он будет подсвечивать синтаксис и помогать автодополнением полей.

🔗 Главная ловушка GraphQL для тестировщика (Ловят на собеседованиях!)

Внимание, это самый популярный вопрос на интервью!

В REST API всё просто: сломался бэкенд — упала ошибка 500 Internal Server Error, ввели кривой пароль — прилетела 400 Bad Request. Мы смотрим на статус-коды.

В GraphQL почти все запросы возвращают HTTP-статус 200 ок, даже если внутри всё полностью сломалось или такого пользователя не существует!

Почему так: С точки зрения протокола HTTP, запрос дошел до сервера GraphQL успешно, сервер его обработал и ответил. А то, что внутри запроса была ошибка — это уже внутреннее дело GraphQL.

Как это тестировать: Нельзя верить статус-коду 200 ок. Тестировщик обязан развернуть тело ответа (Response JSON) и проверить, нет ли там специального блока `errors`.

Что такое gRPC?

«**gRPC** — это высокопроизводительный фреймворк от Google для удаленного вызова процедур (RPC). В отличие от классического REST, он работает поверх протокола **HTTP/2** и использует бинарный формат

передачи данных **Protocol Buffers (Protobuf)** вместо текстового JSON. Это обеспечивает огромную скорость и поддержку двунаправленного стриминга данных.

gRPC чаще всего применяется для внутреннего взаимодействия между микросервисами. Вся логика базируется на строгом контракте — файле с расширением `.proto`.

В тестировании gRPC я опираюсь на этот `.proto`-файл, который загружаю в специализированные инструменты, такие как **Postman** или **Kreya**. Я проверяю корректность вызова методов, валидацию типов данных, обработку стримов и соответствие специфических gRPC-статусов ошибок (например, `Invalid Argument` или `Unauthenticated`) требованиям системы».

Учитывая, что gRPC работает с бинарными данными и контрактами, хочешь узнать, как ручному тестировщику быстро настроить Postman для отправки первого gRPC-запроса, имея на руках `.proto`-файл?

Как тестировщик проверяет gRPC?

Поскольку глазами бинарный код не прочесть, для тестирования gRPC нужны специальные инструменты, умеющие расшифровывать Protobuf:

1. **Контрактное тестирование:** Первым делом QA берет у разработчиков `.proto`-файл. Без него протестировать систему невозможно, так как инструмент не поймет, какие функции вообще есть на сервере.
2. **Инструменты (GUI-клиенты):**
 - **Postman:** В современных версиях есть полноценная поддержка gRPC. Ты загружаешь туда `.proto`-файл, Postman сам генерирует список доступных методов, а бинарный ответ от сервера автоматически переводит в красивый и читаемый JSON.
 - **Kreya / BloomRPC:** Специализированные программы исключительно под gRPC (аналоги Postman, но заточенные только под контракты и стриминг).
3. **Что мы проверяем:**
 - Соответствие типам данных (если в контракте указано число `int32`, сервер не должен принимать текст).
 - Скорость ответа и стабильность стриминг-каналов при долгой нагрузке.
 - Вместо HTTP-статусов (200, 404) в gRPC используются свои **внутренние статус-коды** (например: 0 OK, 1 CANCELLED, 3 INVALID_ARGUMENT, 16 UNAUTHENTICATED). Их тоже нужно сверять.

Что такое Kibana/Elasticsearch?

«Elasticsearch и Kibana — это ключевые компоненты Elastic Stack, используемые для централизованного сбора, анализа и мониторинга логов.

Elasticsearch — это нереляционная база данных и аналитический движок, который хранит логи в формате JSON-документов и обеспечивает практически мгновенный полнотекстовый поиск по ним. **Kibana** — это фронтенд-интерфейс для Elasticsearch, предоставляющий UI для поиска данных, фильтрации и построения интерактивных дашбордов.

В своей повседневной работе как QA я использую Kibana для анализа поведения бэкенда и локализации багов. При возникновении серверных ошибок я ищу логи во вкладке Discover с помощью языка запросов **KQL**, фильтрую по таймстемпам, уровням логирования или уникальным Correlation ID. Найденный Stack Trace я прикрепляю к баг-репорту, что значительно ускоряет исправление дефекта разработчиками».

👤 Зачем Kibana и Elasticsearch нужны тестировщику (QA)?

Для QA-инженера Kibana — это **главный инструмент для чтения серверных логов**.

Когда ты тестируешь сайт или мобильное приложение, и вдруг на экране выскакивает ошибка (или в DevTools прилетает статус 500 Internal Server Error), бэкэнд-разработчик на твой баг-репорт обязательно скажет: «*Пришли логи с сервера, я не знаю, что упало*».

Твои действия в Kibana:

- **Выставляешь таймпикер** (промежуток времени) — например, Last 15 minutes, чтобы не копаться в гигабайтах старых данных.
- **Пишешь поисковый запрос** на специальном легком языке **KQL (Kibana Query Language)**.
 - *Пример:* level : "error" (показать только логи с уровнем «ошибка»).
 - *Пример:* status : 500 AND message : "payment" (найти ошибки 500, связанные с оплатой).
 - Также можно искать по уникальному userId или correlationId (сквозному идентификатору запроса).
- **Находишь упавший лог**, разворачиваешь его, копируешь детальный **Stack Trace** (полный технический отчет об ошибке из кода бэкенда) и прикрепляешь его к баг-репорту. Разработчик увидит конкретную строчку кода, где всё сломалось, и быстро починит баг.

🔍 Из чего состоит стек (Что такое ELK)?

Коротко упомянем классическую аббревиатуру **ELK**, которую часто пишут в требованиях к вакансиям:

- **Elasticsearch** — хранит данные и ищет по ним.
- **Logstash** (или современные агенты **Beats**) — «пылесосы», которые собирают текстовые логи из разных серверов, чистят их, приводят к единому виду и отправляют в Elasticsearch.
- **Kibana** — визуализирует и показывает всё это людям.

Что такое Kafka?

«**Apache Kafka** — это распределенный брокер сообщений и платформа потоковой передачи данных, спроектированная для асинхронного взаимодействия микросервисов в системах с высокой нагрузкой. Она работает по модели **Publish-Subscribe** (издатель-подписчик).

Основными компонентами системы являются **продюсеры**, которые отправляют сообщения, **топики**, где эти сообщения временно или постоянно хранятся в виде лога изменений, и **консьюмеры**, которые вычитывают эти данные по мере своей готовности.

В тестировании систем с Kafka я использую инструменты визуализации вроде **Offset Explorer** или **AKHQ**. Мои проверки делятся на валидацию работы продюсеров (проверка факта и корректности структуры отправленного сообщения в топик) и консьюмеров (отправка тестового события в топик и проверка реакции системы).

4 главных термина Kafka, которые нужно знать QA

Вся архитектура Кафки держится на четырех понятиях:

1. **message / Event (Сообщение / Событие):** Сами данные, которые передаются по сети. Обычно это обычный JSON-текст (например: {"orderId": 45, "status": "paid"}).
2. **topic (Топик):** Тематический канал или «папка», куда складываются сообщения. У каждого топика есть имя (например, payment-events или user-logs).
3. **Producer (Продюсер / Отправитель):** Микросервис, который генерирует данные и «публикует» их в нужный топик Кафки.

4. **Consumer (Консьюмер / Получатель):** Микросервис, который «подписан» на топик, слушает его и забирает сообщения для обработки.

♂ Зачем Кафка тестировщику и как её проверять?

В микросервисной архитектуре баг может случиться «между сервисами». Например, пользователь нажал кнопку оплаты, деньги списались, а статус заказа в личном кабинете остался «Не оплачено».

Тестировщику нужно понять: сервис оплаты вообще отправил сообщение? Или сервис заказов его получил, но не смог прочитать?

✂ Инструменты для QA:

Так как Кафка работает внутри бэкенда, глазами в браузере её не увидеть. QA-инженеры используют специальные программы-клиенты с графическим интерфейсом:

- **Offset Explorer** (старое название *Kafka Tool*) — самая популярная классическая утилита.
- **AKHQ / Kafdrop / UI for Apache Kafka** — современные красивые веб-панели, которые разворачиваются на проекте.
- **Postman** — в свежих версиях тоже появилась вкладка для тестирования событийных систем (Kafka/Event Streams).

□ Базовый чек-лист тестирования Kafka:

- **Тест отправки (Producer):** Ты совершаешь действие на сайте → заходишь в программу (например, *Offset Explorer*) → открываешь нужный топик и проверяешь: появилось ли там новое сообщение? Правильная ли у него структура JSON и все ли поля на месте?
- **Тест обработки (Consumer):** Ты можешь сам вручную с помощью утилиты отправить «фейковое» сообщение в топик Кафки и посмотреть, как твой бэкенд-сервис его прочитает и отобразит на сайте.
- **Негативный тест на «битые» данные:** Что будет, если отправить в топик некорректное сообщение (например, вместо числа передать текст)? Сервер должен корректно отправить такое сообщение в специальный технический топик-помойку (**DLQ — Dead Letter Queue**), а не упасть и заблокировать всю очередь.
- **Идемпотентность (Дубликаты):** Что произойдет, если из-за сбоя сети Кафка пришлет консьюмеру одно и то же сообщение о списании денег два раза? Система должна понять, что этот `orderId` уже обрабатывался, и не списывать деньги повторно.

▤ Краткое сравнение: REST против Kafka

Критерий	REST API	Apache Kafka
Тип связи	Синхронный (запрос-ответ здесь и сейчас).	Асинхронный (отправил и забыл, получатель заберет позже).
Связанность	Сервисы жестко зависят друг от друга.	Сервисы изолированы, они знают только про Кафку.

Критерий	REST API	Apache Kafka
Сохранение данных	Данные пролетают через сеть и не хранятся в API.	Сообщения могут храниться в топиках днями или неделями.
Когда применять	Когда ответ нужен мгновенно (Авторизация, баланс карты).	Когда важна скорость обработки больших потоков (Логи, аналитика, заказы).

Какие бывают типы мобильных приложений?

Мобильные приложения делятся на три основных типа: **нативные, мобильные веб-приложения и кроссплатформенные (или гибридные)**.

- **Нативные** пишутся под конкретную платформу на Swift или Kotlin, имеют прямой доступ к API операционной системы и аппаратной части устройства, обеспечивая максимальную производительность.
- **Веб-приложения** — это адаптированные под мобильные экраны сайты (включая формат PWA), которые работают внутри браузера. Они дешевле в разработке, но ограничены в доступе к железу устройства.
- **Кроссплатформенные** создаются на фреймворках вроде Flutter или React Native. У них общая кодовая база для iOS и Android, они компилируются в нативный код и распространяются через сторы.

Как QA, при тестировании нативных приложений я уделяю больше внимания прерываниям, работе с железом и разрешениями. В веб-приложениях фокус смещается на кроссбраузерность и адаптивность верстки, а в кроссплатформенных приложениях — на поиск специфичных для конкретной ОС дефектов отображения интерфейса

Особенности тестирования: Что ищет QA?

В зависимости от типа приложения, фокус тестировщика смещается:

1. **В Нативных:** Мы жестко тестируем **фрагментацию устройств** (как приложение ведет себя на старой версии Android 10 против Android 14, на экранах с «челкой» и без). Проверяем работу в бэграунде (что будет, если во время заполнения формы юзеру позвонят).
2. **В Веб/PWA:** Основной упор на **кроссбраузерность** (как сайт отображается в мобильном Safari на iPhone и в Chrome на дешёвом Xiaomi) и **адаптивную верстку** (не поехали ли кнопки).
3. **В Гибридных:** Самая частая проблема — **UI-баги специфичные для платформы**. Например, на Android кнопка отображается идеально, а на iOS из-за особенностей рендеринга она уехала под системную шторку или обрезалась. Также проверяется скорость отклика интерфейса.

1. Нативные приложения (Native Apps)

Это приложения, созданные специально под конкретную операционную систему (iOS или Android) на её «родном» языке программирования.

- **Технологии:** Для iOS это **Swift** или **Objective-C**. Для Android — **Kotlin** или **Java**.

- **Как устанавливаются:** Строго через официальные магазины приложений (App Store и Google Play).
- **Аналогия:** Костюм, сшитый на заказ в дорогом ателье по твоим индивидуальным меркам. Сидит идеально, двигаться удобно.

👉 **Плюсы:** Максимальная скорость работы, плавная анимация. Имеют полный и прямой доступ к «железу» телефона (камера, контакты, GPS, Bluetooth, датчик шагов, FaceID). Могут полноценно работать без интернета.

👎 **Минусы:** Самая дорогая разработка. Нужно писать и поддерживать два абсолютно разных кода — отдельно для Apple, отдельно для Android.

🌐 2. Мобильные веб-приложения (Web Apps)

Это не приложения в привычном смысле, а обычные сайты, которые разработчики адаптировали под размеры экранов смартфонов.

- **Технологии:** Стандартный веб-стек — **HTML, CSS, JavaScript**.
- **Как устанавливаются:** Их не нужно скачивать. Ты просто открываешь мобильный браузер (Safari или Chrome), вводишь адрес сайта и пользуешься им.
- **Аналогия:** Универсальный дождевик-пончо. Подходит абсолютно любому человеку любого роста, но сидит мешковато и выглядит простовато.

👉 **Плюсы:** Самая дешевая разработка. Один код работает на любом устройстве, где есть браузер. Не нужно проходить жесткую модерацию в App Store и ждать одобрения обновлений сутками.

👎 **Минусы:** Не работают без интернета. Работают медленнее нативных. Доступ к функциям телефона сильно ограничен (например, веб-сайт не может просто так использовать датчики смартфона или присылать полноценные пуш-уведомления на iOS).

✂ 3. Кроссплатформенные и гибридные приложения (Cross-platform / Hybrid)

Компромисс, созданный ради экономии бюджетов. Разработчики пишут один общий код, который затем специальными фреймворками компилируется (превращается) в приложения и для iOS, и для Android.

- **Технологии:** Самые популярные фреймворки сегодня — **Flutter** (язык Dart) и **React Native** (язык JavaScript).
- **Как устанавливаются:** Так же, как и нативные — скачиваются из App Store и Google Play. Пользователь визуально даже не поймет, что это гибрид.
- **Аналогия:** Стандартная одежда из магазина масс-маркета. Стоит адекватных денег, подходит большинству, выглядит отлично, но в экстремальных условиях (например, в тяжелых 3D-играх) может «разойтись по швам».

👉 **Плюсы:** Экономия времени и денег до 40–50%. Один код для обеих платформ, одна команда разработки вместо двух. Хороший доступ к функциям устройства через плагины.

👎 **Минусы:** Из-за промежуточного слоя (фреймворка) они могут слегка подтормаживать на старых телефонах. Если Apple или Google выпускают крутую фичу в новой версии ОС, разработчикам гибридов приходится ждать, пока фреймворк обновится, чтобы её внедрить.

⚡ 4. PWA (Progressive Web Apps)

Продвинутая эволюция веб-приложений. Это сайт, который притворяется нативным приложением.

- **Как это работает:** Ты заходишь на сайт через браузер телефона, и сайт предлагает тебе: «Добавить на главный экран». На телефоне появляется обычная иконка. Когда ты на неё нажимаешь, сайт открывается на весь экран — без адресной строки браузера и лишних кнопок.
- **Пример в жизни:** Из-за санкций и удаления приложений банков из App Store, многие банки (например, Т-Банк или Альфа-Банк) перешли именно на формат PWA для пользователей iPhone.

Чем нативные приложения отличаются от гибридных и PWA?

Главное отличие заключается в технологиях разработки и способе взаимодействия с операционной системой устройства.

- **Нативные приложения** пишутся под конкретную платформу на её родном языке (Swift/Kotlin). Они компилируются в бинарный код, обеспечивают максимальную производительность и имеют полный доступ к аппаратному обеспечению телефона через native API.
- **Гибридные приложения** имеют единую кодовую базу (например, на Flutter или React Native) и запускаются на обеих платформах. Они дешевле в разработке, но используют промежуточный слой для обращения к функциям ОС, что может незначительно снижать производительность.
- **PWA** — это продвинутые веб-сайты, которые визуальнo имитируют интерфейс приложения. Они устанавливаются в обход сторов прямо из браузера, мгновенно обновляются, но их возможности жестко ограничены политиками безопасности браузеров (особенно на iOS).

В тестировании нативных приложений я больше фокусируюсь на фрагментации ОС и аппаратных прерываниях. В гибридных — на кроссплатформенных дефектах UI, а в PWA — на кроссбраузерности и корректной работе сервис-воркеров при кэшировании данных

Что это значит для тестировщика (QA)?

Твоя стратегия тестирования будет полностью меняться в зависимости от типа приложения:

Если ты тестируешь Нативное приложение: Ты будешь страдать от **фрагментации устройств**. Приложение может идеально работать на чистом Android 14 от Google, но намертво падать на Android 12 со специфической оболочкой от Xiaomi. Твой фокус — тестирование на разных версиях ОС и экранах с разным разрешением.

Если ты тестируешь Гибридное приложение: Твой главный враг — **UI-баги отображения**. Из-за того, что код один, какая-нибудь кнопка может отлично выглядеть на Android, но на iOS намертво уехать под «челку» экрана или системную шторку. Ты будешь искать места, где интерфейс «поехал».

Если ты тестируешь PWA: Ты, по сути, тестируешь сайт. Твой фокус — **кроссбраузерность** и очистка кэша. Нужно проверять, как PWA ведет себя в движке Safari (на iOS) и в движке Chrome/Blink (на Android). Самый частый баг здесь — когда разработчики выкатили обновление, а у пользователя осталась старая верстка из-за «залипшего» кэша.

Чем нативные приложения отличаются от гибридных и PWA?

Главное отличие заключается в технологиях разработки и способе взаимодействия с операционной системой устройства.

- **Нативные приложения** пишутся под конкретную платформу на её родном языке (Swift/Kotlin). Они компилируются в бинарный код, обеспечивают максимальную производительность и имеют полный доступ к аппаратному обеспечению телефона через native API.
- **Гибридные приложения** имеют единую кодовую базу (например, на Flutter или React Native) и запускаются на обеих платформах. Они дешевле в разработке, но используют промежуточный слой для обращения к функциям ОС, что может незначительно снижать производительность.

- **PWA** — это продвинутые веб-сайты, которые визуально имитируют интерфейс приложения. Они устанавливаются в обход сторов прямо из браузера, мгновенно обновляются, но их возможности жестко ограничены политиками безопасности браузеров (особенно на iOS).

В тестировании нативных приложений я больше фокусируюсь на фрагментации ОС и аппаратных прерываниях. В гибридных — на кроссплатформенных дефектах UI, а в PWA — на кроссбраузерности и корректной работе сервис-воркеров при кэшировании данных».

Что это значит для тестировщика (QA)?

Твоя стратегия тестирования будет полностью меняться в зависимости от типа приложения:

Если ты тестируешь Нативное приложение: Ты будешь страдать от **фрагментации устройств**. Приложение может идеально работать на чистом Android 14 от Google, но намертво падать на Android 12 со специфической оболочкой от Xiaomi. Твой фокус — тестирование на разных версиях ОС и экранах с разным разрешением.

Если ты тестируешь Гибридное приложение: Твой главный враг — **UI-баги отображения**. Из-за того, что код один, какая-нибудь кнопка может отлично выглядеть на Android, но на iOS намертво уехать под «челку» экрана или системную шторку. Ты будешь искать места, где интерфейс «поехал».

Если ты тестируешь PWA: Ты, по сути, тестируешь сайт. Твой фокус — **кроссбраузерность** и очистка кэша. Нужно проверять, как PWA ведет себя в движке Safari (на iOS) и в движке Chrome/Blink (на Android). Самый частый баг здесь — когда разработчики выкатили обновление, а у пользователя осталась старая верстка из-за «залипшего» кэша.

Какие особенности есть у мобильного тестирования?

«Главное отличие мобильного тестирования от веб-тестирования — это необходимость проверять продукт в условиях динамичной и нестабильной среды реального устройства.

Ключевые особенности включают в себя: во-первых, **высокую фрагментацию** девайсов, экранов и операционных систем (особенно на Android). Во-вторых, обязательное тестирование **мобильных прерываний** — звонков, пушей, разряда батареи. В-третьих, проверку работы в **нестабильных сетевых условиях** при переключении между Wi-Fi и мобильной сетью. Также критически важно тестировать специфику **железа** (GPS, биометрия, повороты экрана) и сценарии **обновления приложения** поверх старых версий без потери пользовательских данных».

1. Кошмар девайс-фрагментации

В мире существуют тысячи моделей телефонов, и все они разные. Тестировщик не может проверить приложение на одном смартфоне и сказать: «Готово!».

- **Размеры и формы экранов:** Кнопки и текст могут идеально смотреться на классическом iPhone, но полностью уехать под «челку», «каплю» для камеры или обрезаться на новомодных складных экранах (Fold).
- **Фрагментация ОС и оболочек:** На iOS всё относительно стабильно, а вот Android — это дикий Запад. Один и тот же Android может вести себя совершенно по-разному на «чистом» Google Pixel и на смартфонах Xiaomi или Samsung со своими глубоко измененными системными оболочками.

2. Мобильные прерывания (Interruptions)

Мобильное приложение никогда не живет в вакууме. Смартфон — это в первую очередь средство связи и гаджет. QA-инженер обязан проверить, как ведет себя приложение, если посреди важного процесса (например, ввода данных карты) происходит **прерывание**:

- Входящий телефонный звонок или СМС.
- Push-уведомление от мессенджера.

3. Нестабильные сетевые условия

Пользователи ПК обычно сидят на стабильном домашнем или офисном интернете. Мобильные пользователи постоянно перемещаются. Тестировщик проверяет:

- **Смена сетей:** Что происходит, когда телефон переключается с домашнего Wi-Fi на мобильный 4G/5G (например, когда ты выходишь из подъезда на улицу)?
- **Мертвые зоны:** Как приложение реагирует на полную потерю сети (в лифте, метро или туннеле)? Оно должно показать красивую заглушку «Нет интернета», а не намертво зависнуть с бесконечным крутящимся колесиком загрузки.
- **Медленный интернет:** Имитация старого 3G-соединения.

4. Железо и ресурсы смартфона

Смартфон жестко ограничен в ресурсах по сравнению с компьютером.

- **Потребление батареи:** Если приложение за 15 минут тестов превращает телефон в горячий кирпич и сжигает 20% заряда — это серьезный баг оптимизации.
- **Утечки памяти и хранилище:** Что будет, если на телефоне осталось всего 50 МБ свободного места? Не упадет ли приложение?
- **Работа с датчиками:** Тестирование интеграции с камерой, GPS-геолокацией, биометрией (FaceID/отпечаток), NFC-чипом и акселерометром (переворот экрана из портретного режима в ландшафтный).

5. Специфика обновлений и установки

В вебе разработчики выкатили код на сервер — и у всех пользователей мгновенно открылся новый сайт. В мобилках всё иначе:

- **Установка и удаление:** Проверяется чистая установка приложения.
- **Обновление поверх старой версии:** Самый частый источник багов.

Чем эмулятор отличается от реального устройства?

«Основное отличие заключается в том, что реальное устройство — это физический аппарат со своей аппаратной архитектурой (ARM), реальными датчиками и ограничениями железа, в то время как эмулятор — это программная модель, симулирующая работу смартфона на базе ресурсов ПК.

Эмуляторы незаменимы на ранних этапах тестирования и при автоматизации: они бесплатны, позволяют быстро переключаться между десятками версий ОС и легко дебажатся. Однако они работают в идеальной среде.

На реальных устройствах мы обязаны проводить финальное тестирование, так как только там можно проверить реальный пользовательский опыт (User Experience и жесты), аппаратные прерывания (звонки, СМС, низкий заряд батареи), работу с физическими датчиками (камера, NFC, биометрия), а также реальную производительность приложения на слабом железе в условиях нестабильной мобильной сети».

💡 Идеальная стратегия тестирования на проекте

Ни один нормальный проект не тестируется *только* на телефонах или *только* на эмуляторах. Их комбинируют:

1. **80% времени (в начале спринта):** Разработчики написали новую фичу. Тестировщик открывает эмулятор, быстро проверяет общую логику, верстку, гоняет негативные тесты на валидацию полей. Это быстро и удобно.
2. **20% времени (перед релизом):** Сборка готова к выпуску. Тестировщик берет в руки **реальные устройства** (хотя бы топ-3 популярных девайса у текущих пользователей приложения) и проводит финальный регресс. Проверяет критические пути, платежи, камеру и пуши.

Что такое краудтестинг?

«Краудтестинг (Crowdsourced Testing) — это метод тестирования, при котором к проверке программного обеспечения привлекается большое географически распределенное сообщество независимых тестировщиков. Взаимодействие происходит через специализированные платформы вроде uTest или Test IO, а оплата чаще всего строится по модели **pay-per-bug** (за каждый подтвержденный уникальный дефект).

Главная ценность краудтестинга — это возможность протестировать продукт на огромном количестве **реальных устройств** в условиях реального мира (разные локации, провайдеры связи, языковые особенности), что физически невозможно повторить силами локальной штатной QA-команды.

Однако краудтестинг не заменяет штатный QA, а дополняет его, обычно на этапе финального UI/UX или регрессионного тестирования B2C-продуктов. К его минусам можно отнести риски конфиденциальности и необходимость тратить ресурсы на валидацию большого количества дубликатов и нерелевантных баг-репортов, присылаемых крауд-тестировщиками».

Что такое TestFlight, Firebase App Distribution, Test IO, Beta Family?

Категория 1. Инструменты бета-дистрибуции (Службы доставки билдов)

Эти инструменты нужны разработчикам, чтобы просто загрузить готовый файл приложения (билд) на сервер и отправить ссылку своей команде тестировщиков (In-house QA). Здесь нет «наемных» людей, это просто технический транспорт.

🍏 TestFlight

Это **официальный и единственный «родной» инструмент компании Apple** для тестирования iOS-приложений.

- **Как это работает:** Разработчик собирает приложение в Xcode и отправляет его в TestFlight. Тестировщик скачивает на свой iPhone одноименное приложение TestFlight, принимает инвайт (приглашение по почте) и внутри него скачивает тестируемую программу.
- **Особенность:** Делится на два лагеря. *Внутреннее тестирование* (твоя команда до 100 человек) — билд доступен мгновенно. *Внешнее тестирование* (публичная ссылка для обычных людей до 10 000 человек) — этот билд должна сначала проверить и одобрить модерация самой Apple, что может занять время.

📦 Firebase App Distribution

Это мощный инструмент от **Google (часть экосистемы Firebase)**, который является главным спасением для кроссплатформенного тестирования.

- **Как это работает:** В отличие от TestFlight, Firebase умеет доставлять приложения **и на Android, и на iOS**.
- **Особенность:** Разработчики обожают его за скорость и отсутствие бюрократии. Никакой модерации со стороны Google или Apple здесь нет. Как только программист нажал кнопку «Собрать», тестировщикам на почту или в специальное приложение *App Tester* прилетает уведомление: «*Доступна новая версия, скачивай!*». Это идеальный инструмент для ежедневного внутреннего тестирования (Daily Builds).

Категория 2. Платформы краудтестинга (Где берут «толпу»)

Эти платформы мы только что обсуждали в теме краудтестинга. Сюда компании приходят не просто за технической доставкой, а **за живыми людьми и их девайсами**.

✂ Test IO

Это одна из крупнейших мировых краудтестинговых платформ (сейчас принадлежит корпорации EPAM).

- **Для кого это:** Это настоящий полигон для коммерческого тестирования. Сюда приходят крупные бренды (интернет-магазины, банки, игры) и нанимают профессиональное QA-сообщество.
- **Как это работает:** Платформа работает по жесткой модели **Pay-per-bug** (оплата за найденный дефект). На Test IO очень строгая модерация баг-репортов — если ты оформил баг не по правилам или прислал дубликат, твою работу отклонят.

Многие начинающие тестировщики регистрируются на Test IO как фрилансеры, чтобы набить руку, получить первый реальный опыт в резюме и заработать первые деньги на поиске багов.

□ Beta Family

Это тоже платформа для тестирования силами сообщества, но с сильным уклоном в **UX (User Experience) и реальных пользователей**, а не в суровый QA-регресс.

- **Для кого это:** Идеально подходит для стартапов и небольших мобильных приложений. Здесь ищут не столько технические поломки бэкенда, сколько обратную связь от целевой аудитории.
- **Как это работает:** Заказчик может отфильтровать людей по жестким критериям: например, «*Мне нужны только женщины 25–35 лет из Франции, играющие в казуальные игры*». Тестировщики скачивают приложение, проходят по заданному сценарию (квесту) и в конце заполняют подробную анкету-отзыв: понятен ли интерфейс, красивая ли анимация, купят ли они этот продукт.

Как перехватывать мобильный трафик через Charles/Fiddler?

Для перехвата мобильного трафика я настраиваю прокси-сервер по принципу Man-in-the-Middle с помощью инструментов Charles Proxy или Fiddler.

Процесс состоит из трех шагов. Сначала я настраиваю ПК: разрешаю удаленные подключения в настройках Charles или Fiddler и активирую SSL Proxying для дешифровки HTTPS. Затем на мобильном устройстве, находящемся в той же Wi-Fi сети, я захожу в свойства подключения и вручную прописываю IP-адрес компьютера и порт программы в качестве HTTP-прокси. Третий шаг — установка и обязательное включение полного доверия для корневого SSL-сертификата прокси-канала на самом смартфоне.

Как анализировать мобильные логи?

«Анализ мобильных логов я делю на работу с клиентскими системными логами и работу с системами мониторинга крашей.

Если баг воспроизводится локально на тестовом устройстве Android, я подключаю его по USB и анализирую логи через **Logcat в Android Studio** (или через `adb logcat`), фильтруя по имени пакета приложения и тегам уровня Error/Fatal. Для поиска причин падений или зависаний я использую ключевые слова вроде *Exception*, *Crash*, *ANR*. На iOS я использую утилиту **Console** на macOS или выгружаю готовые файлы краш-логов через Xcode.

Если дефект произошел удаленно на этапе бета-тестирования, я анализирую дебаг-панели систем сборки логов, таких как **Firebase Crashlytics** или **Sentry**. При локализации бага по логам я вычленяю тип ошибки (например, `NullPointerException`) и нахожу в стек-трейсе конкретный класс и строчку кода приложения, где произошел сбой, после чего прикрепляю этот лог к баг-репорту».

🔍 Чек-лист: Как читать Stack Trace (След Ошибки)

Когда ты нашел лог падения, перед тобой будет простыня текста. Читать её нужно **сверху вниз**:

Plaintext

```
FATAL EXCEPTION: main
Process: com.qa.example, PID: 12345
java.lang.NullPointerException: Attempt to invoke virtual method 'java.lang.String
.toString()' on a null object reference
    at com.qa.example.MainActivity.onPaymentClick(MainActivity.kt:42)
    at com.qa.example.MainActivity$onCreate$1.onClick(MainActivity.kt:20)
```

1. **Первая строчка** указывает на тип катастрофы: `FATAL EXCEPTION` и суть проблемы: `NullPointerException` (код попытался обратиться к объекту, которого не существует в памяти).
2. **Строчки ниже (начинающиеся с `at...`)** — это путь падения внутри кода.
3. Ищи строчку, где упоминается имя твоего приложения (в примере: `com.qa.example...`). Текст в скобках `MainActivity.kt:42` говорит разработчику: *«Бро, у тебя всё взорвалось в файле MainActivity на 42-й строчке кода, когда пользователь нажал кнопку оплаты (onPaymentClick)»*.

Что такое реляционные базы данных?

«Реляционные базы данных — это системы управления базами данных, в которых информация организована в виде двумерных таблиц со строгой структурой. Взаимодействие между таблицами строится на основе связей между первичными (**Primary Key**) и внешними (**Foreign Key**) ключами. Для управления данными и написания запросов используется стандартный язык **SQL**.

Ключевым преимуществом реляционных БД является соответствие требованиям **ACID**, что гарантирует надежность, транзакционность и целостность данных даже при системных сбоях. Самыми популярными решениями на проектах являются PostgreSQL и MySQL.

В работе QA-инженера знание реляционных баз данных необходимо для проверки корректности сохранения данных с фронтенда, подготовки тестовых состояний системы напрямую через таблицы, а также для валидации бизнес-логики бэкенда при проведении интеграционного тестирования».

👤♂ Зачем базы данных тестировщику (QA)?

Тестировщик постоянно заглядывает «под капот» системы. Браузер или мобилка — это просто красивая обертка. Все действия пользователя в итоге изменяют строки в базе данных.

Что проверяет QA в базе данных:

1. **Корректность записи (Data Integrity):** Пользователь зарегистрировался на сайте. Тестировщик открывает базу данных и проверяет: создалась ли строка в таблице `Users`? Хэшируется ли пароль в целях безопасности или лежит открытым текстом (что является багом)?
2. **Валидация типов данных:** Если в поле «Возраст» разработчик базы данных указал числовой тип, а ты через API умудрился отправить туда текст «Тридцать лет», база должна выдать ошибку и не упасть.
3. **Подготовка тестовых данных:** Иногда, чтобы протестировать редкий баг, тебе нужен пользователь, у которого на балансе ровно 0 рублей, заблокирована карта и есть 5 активных заказов. Проще и быстрее зайти в базу данных и за секунду поменять эти значения SQL-запросом, чем создавать такого пользователя вручную через интерфейс сайта.

Что такое нереляционные базы данных?

«Нереляционные базы данных (NoSQL) — это класс систем управления базами данных, которые не используют традиционную табличную реляционную модель и не требуют строгой предопределенной схемы данных. Они оптимизированы под высокую скорость работы, горизонтальное масштабирование и обработку огромных объемов неструктурированной информации.

Основные типы NoSQL баз — это **документные** (например, MongoDB, хранящая данные в JSON-подобном формате), **ключ-значение** (Redis, используемый для кэширования), **колоночные** для Big Data и аналитики (Cassandra, ClickHouse), а также **графовые** для сложных взаимосвязей.

В работе QA специфика NoSQL заключается в том, что валидация данных переносится с уровня БД на уровень приложения. Я проверяю, как бэкенд обрабатывает документы с изменяющейся структурой, пишу специфические запросы к коллекциям (например, в MongoDB) и тестирую сценарии инвалидации данных, если на проекте используются key-value хранилища вроде Redis для кэширования сессий».

👤♂ Что тестировщику (QA) проверять в NoSQL базах?

Если на твоём проекте используется, например, MongoDB, твоя рутина будет отличаться от работы с классическим SQL:

1. **Запросы без SQL:** В MongoDB ты не пишешь `SELECT * FROM users`. Там используется свой JavaScript-подобный синтаксис. Например: `db.users.find({age: {$gt: 18}})` (найти пользователей старше 18 лет). Эти команды нужно будет освоить.
2. **Проверка "свободы" бэкенда:** Так как база принимает всё подряд, разработчики могут забыть написать валидацию в коде самого бэкенда. Тестировщик проверяет: что будет, если через API отправить в документ поле с неожиданным типом данных? Не сломается ли фронтенд, когда попытается прочитать JSON с кривой структурой?
3. **Тестирование кэша (Redis):** QA проверяет инвалидацию (сброс) кэша. Например: пользователь сменил пароль. Изменился ли пароль в кэше Redis мгновенно, или пользователь со старым токеном авторизации сможет заходить в систему еще полчаса?
4. • **В реляционной базе (SQL):** Данные хранятся как в анкете строгого образца. Есть поля «Имя», «Фамилия», «Возраст». Если у тебя нет фамилии — поле будет пустым (`NULL`), но удалить саму колонку для одного человека нельзя.
5. • **В нереляционной базе (NoSQL):** Данные хранятся как заметки на стикерах. На одном стикере ты написал: «Купить молоко, марка Домик в деревне». На втором: «Позвонить маме, срочно,

телефон такой-то». На третьем вообще нарисовал схему. Они все висят на одном холодильнике (в одной базе), но структуры у них абсолютно разные.

Что такое нормальные формы 1НФ, 2НФ, 3НФ?

«Если говорить простыми словами, **нормализация (1НФ, 2НФ, 3НФ)** — это наведение порядка в базе данных, чтобы избавиться от дублирования информации. Главная цель — сделать так, чтобы при любом изменении (например, если у товара сменилась цена) нам нужно было обновить это только в одной ячейке, а не искать по всей базе.

По сути, мы берем одну огромную таблицу, в которой всё свалено в кучу, и по правилам дробим её на несколько аккуратных связанных таблиц:

- **В Первой форме (1НФ)** мы следим, чтобы в каждой ячейке лежало только одно значение. Никаких перечислений через запятую. Одно поле — одно значение.
- **Во Второй форме (2НФ)** мы убираем дублирование и выносим независимые данные в справочники. То есть, не пишем состав пиццы в каждом заказе, а выносим это в отдельную таблицу "Меню".
- **В Третьей форме (3НФ)** мы избавляемся от данных, которые притянуты "через посредника". Например, не пишем в таблице заказов марку машины курьера, потому что машина относится к курьеру, а не к заказу. Выносим это в таблицу "Курьеры".

На реальных проектах базы обычно доводят именно до 3НФ. Для меня как для тестировщика это значит, что данные жестко разнесены по разным таблицам, и чтобы их проверить, мне нужно будет использовать SQL-запросы с объединением JOIN

Какие базовые SQL-запросы должен знать тестировщик?

📁 Шпаргалка: 5 главных команд SQL

1. SELECT («Дай мне»)

Самая частая команда. Ты просто просишь базу показать тебе данные.

- **Запрос:** `SELECT * FROM Orders;`
- **Перевод:** «Выдай мне всё, что есть в таблице заказов».
- **Зачем тестировщику:** Чтобы проверить, создал ли ты заказ или изменился ли статус заказа после оплаты.

2. INSERT («Добавь»)

Создает новую строку (новый заказ, нового юзера).

- **Запрос:** `INSERT INTO Orders (pizza_name) VALUES ('Маргарита');`
- **Перевод:** «Вставь в таблицу заказов заказ на пиццу Маргарита».
- **Зачем тестировщику:** Чтобы быстро создать тестового пользователя, не заполняя 100 полей на сайте.

3. UPDATE («Исправь»)

Меняет данные в уже существующих записях.

- **Запрос:** `UPDATE Orders SET status = 'Оплачен' WHERE id = 123;`

- **Перевод:** «Поменяй статус заказа №123 на 'Оплачен'».
- **Зачем тестировщику:** Чтобы искусственно сменить статус заказа, «заблокировать» пользователя или изменить баланс кошелька, чтобы проверить, как приложение на это отреагирует.

4. DELETE («Удали»)

Стирает строчку навсегда.

- **Запрос:** `DELETE FROM Orders WHERE id = 123;`
- **Перевод:** «Удали заказ №123».
- **Зачем тестировщику:** Чтобы почистить «мусор» (тестовые заказы) после того, как ты их проверил.

5. JOIN («Склей»)

Соединяет информацию из разных таблиц в одну.

- **Запрос:** `SELECT Orders.name, Couriers.name FROM Orders JOIN Couriers ON ...;`
- **Перевод:** «Покажи мне имя заказчика из таблицы заказов и имя курьера из таблицы курьеров, соединив их по ID».
- **Зачем тестировщику:** Чтобы посмотреть связи (например, какой курьер везет какой заказ), если данные разбросаны по разным таблицам.

□ Золотое правило для тебя (запомни на всю жизнь!)

Всегда добавляй слово **WHERE** (условие) к командам **UPDATE** и **DELETE**.

- **Если ты пишешь:** `DELETE FROM Orders;` — ты удаляешь все заказы пиццерии вообще. Босс тебя уволит через 5 секунд.
- **Если ты пишешь:** `DELETE FROM Orders WHERE id = 5;` — ты удаляешь только пятый заказ. Это безопасно.

Как проверять сохранение данных через интерфейс и БД?

«Я всегда проверяю данные методом "до и после". Сначала фиксирую исходное состояние записи в БД, затем выполняю действие в интерфейсе и сверяю результат в базе. Это позволяет исключить ситуацию, когда интерфейс отображает "успех", а данные на сервере не изменились. Если вижу расхождение — сразу завожу баг, так как это критическая проблема с целостностью данных».

Твой пошаговый план:

1. **Шаг 1: Смотрим «ДО» (База данных)** Перед тем как что-то менять, открой базу данных и посмотри, что там написано сейчас.
 - *Пример:* Ты хочешь поменять имя пользователя «Иван» на «Петр». Зайди в базу и убедись: в колонке `name` стоит «Иван».
 - *Зачем:* Чтобы убедиться, что ты вообще работаешь с тем самым пользователем.
2. **Шаг 2: Делаем «ДЕЙСТВИЕ» (Интерфейс)** Иди на сайт, открой профиль, поменяй имя на «Петр» и нажми кнопку «Сохранить».
 - *Важно:* Дождись сообщения «Успешно» или крутящегося колесика загрузки.
3. **Шаг 3: Смотрим «ПОСЛЕ» (База данных)** Снова открой базу данных и выполни тот же запрос.
 - *Проверка:* Теперь в колонке `name` должно быть «Петр». Если там всё еще «Иван» — поздравляю, ты нашел баг! Приложение обмануло пользователя.
4. **Шаг 4: Смотрим «ПОСЛЕ» (Интерфейс)** Вернись на сайт и обнови страницу (F5). Теперь сайт должен «сходить» в базу, забрать оттуда новое имя «Петр» и показать его тебе.

Почему это важно? (Ловушки)

Иногда случаются забавные (и обидные) вещи:

- **«Иллюзия успеха»:** Ты нажал кнопку, и сайт сам, прямо в браузере, подставил новое имя (без запроса к базе). Ты думаешь, что всё сохранилось, обновляешь страницу, а там — бац! — опять старое имя. Это значит, что приложение «забыло» отправить данные на сервер.
- **Кэш:** Ты всё поменял, в базе данные обновились, но на сайте всё равно старое имя. Это не баг, это просто браузер «запомнил» старую версию страницы. Попробуй обновить страницу или зайти в режиме «Инкогнито».
- **Задержка:** В больших проектах данные иногда «бегут» до базы пару секунд. Не паникуй сразу, если не видишь изменений мгновенно — дай системе пару секунд.

Когда тестировщику нужен доступ к БД?

«Мне доступ к базе нужен как к инструменту для глубокой проверки. Во-первых, чтобы убедиться, что данные, которые я ввожу через интерфейс, корректно сохраняются (а не просто отображаются как "успех"). Во-вторых, чтобы быстрее готовить тестовые данные — это экономит кучу времени. Ну и, конечно, для расследования багов: когда приложение выдает "общую ошибку", база помогает мне найти реальную причину проблемы».

1. Проверить, «долетели» ли данные (Validation)

Ты нажал кнопку «Сохранить», приложение написало: «Всё ок!». Но ты не веришь на слово. Ты идешь в базу данных и смотришь: *«А реально ли там появилась запись? Не обманывает ли меня интерфейс?»*.

2. Сделать «идеальные» условия для теста (Test Data Setup)

Представь, тебе надо протестировать, как сайт ведет себя, когда у пользователя **заблокирована карта, на счету минус, и он из другой страны**. Если ты будешь пытаться создать такого пользователя через сайт — ты потратишь час. А если ты зайдешь в базу данных, ты за 5 секунд поправишь три цифры в таблице, и у тебя будет «идеально сломанный» пользователь.

3. Расследовать баг (Debugging)

Пользователь жалуется: «Я нажимаю кнопку, а мне выдает ошибку: *Что-то пошло не так*». Это ни о чем не говорит. Ты лезешь в базу или логи и видишь реальную причину: *«Ошибка: поле Email пустое»* или *«Дубликат записи»*.

4. Проверить «тихие» процессы (Background tasks)

Некоторые действия происходят незаметно для пользователя. Например, ты нажал «Оформить подписку», и через 30 дней система должна сама списать деньги. Ты не можешь ждать месяц. Ты залезаешь в базу, находишь запись о подписке и вручную меняешь дату с «через 30 дней» на «завтра».

Что такое CLI?

«Я понимаю, что такое CLI, и не боюсь работать в терминале. Я знаю базовые команды (ориентирование в папках, создание файлов, просмотр логов), умею пользоваться терминалом для подключения к серверам или работы с Docker. Понимаю, что CLI — это инструмент для быстрой автоматизации задач, и

мне комфортнее работать с ним, чем через графический интерфейс, когда нужно сделать что-то быстро или массово».

Короче говоря: CLI — это просто способ «поговорить» с компьютером напрямую, минуя посредников-кнопки.

Что такое Git?

«Я понимаю, что Git — это система контроля версий. Я знаю, что это инструмент для работы с историей изменений кода, который позволяет не потерять работу и легко откатываться назад, если что-то пошло не так.

В работе я использую Git, чтобы получать актуальный код автотестов (команда `pull`), переключаться между ветками (команда `checkout`), если нужно протестировать новую фичу до того, как она попала в основную версию, и иногда просматриваю историю коммитов, чтобы понять, какие изменения в коде могли вызвать баг».

А в чем разница: Git и GitHub?

Это самая частая ошибка новичков.

- **Git** — это сама программа, которая «следит» за историей твоих файлов на твоём компьютере.
- **GitHub** — это «облако» (сайт в интернете), где эти проекты хранятся. Это как флешка в интернете. Ты делаешь «сейв» (коммит) у себя на компьютере (Git), а потом «отправляешь» его на GitHub, чтобы другие коллеги могли его увидеть.
- Если тебя спросят на собеседовании: «Сможешь клонировать репозиторий?», не пугайся. Отвечай так:
- «Да, конечно. Я знаю, что нужно зайти на GitHub, скопировать ссылку через кнопку Code и выполнить `git clone` в терминале. Также я понимаю, что для поддержания кода в актуальном состоянии мне нужно будет периодически делать `git pull`».

Какие базовые команды Git нужны QA?

Шпаргалка: Базовые команды Git для тестировщика

- `git clone <ссылка>` — «Скачивание» проекта. Используется один раз в самом начале, чтобы забрать проект с GitHub (или другого облака) к себе на компьютер.
- `git pull` — «Обновление». Самая частая команда. Подтягивает все последние изменения, которые сделали программисты, чтобы твои тесты всегда были актуальными.
- `git branch` — «Список веток». Показывает, в какой «параллельной вселенной» (ветке) ты сейчас находишься. Полезно, чтобы не тестировать не ту задачу.
- `git checkout <имя_ветки>` — «Переключение». Позволяет перепрыгнуть из одной ветки в другую (например, чтобы потестировать фичу, которую сейчас делает разработчик).
- `git status` — «Состояние». Показывает, что происходит с файлами: что ты поправил, что добавил и есть ли что-то «сломанное». Твой главный инструмент для контроля ситуации.
- `git log` — «История». Показывает список всех сохранений (коммитов) проекта. Помогает понять, кто и когда внес изменения или какая версия кода сейчас активна.
- `git checkout .` — «Кнопка спасения». Если ты случайно поправил файл в коде и боишься, что всё сломал — эта команда мгновенно отменит твои правки и вернет файл к состоянию «как было до этого».

Маленький совет: Если забудешь, что делает команда — не переживай, это нормально. В терминале ты всегда можешь написать `git --help` или просто погуглить «git команда [название]» — это делает любой профессиональный тестировщик, если вдруг что-то вылетело из головы.

- **Что такое ветки `main`, `develop`, `feature`?**

Main (или Master) — «Готовые квартиры» Это самый важный код. В этой ветке лежит то, что уже работает у реальных пользователей. Здесь всегда должен быть порядок. Сюда нельзя вносить правки просто так («на коленке»), иначе у всех пользователей упадет сайт. Если ты тестируешь что-то в `main` — значит, ты проверяешь то, что уже в «продакшене» (на реальном сайте).

- **Develop (или Dev)** — «Стройплощадка» Это ветка, где всё собирается вместе. Сюда стекаются все наработки программистов. Это «черновик» следующей версии программы. Она может быть чуть более нестабильной, чем `main`, но именно здесь тестировщики чаще всего гоняют свои тесты, чтобы убедиться, что все новые фичи дружат друг с другом.
- **Feature (или Task-ветки)** — «Ремонт в конкретной комнате» Это временные ветки. Если программисту нужно сделать конкретную задачу (например, «добавить кнопку в корзину»), он не лезет сразу в `develop`. Он создает себе «песочницу» — ветку `feature/button-in-cart`. Там он спокойно пишет код, ломает, чинит, и никто другой не страдает. Когда всё готово и работает, он «вливает» (`merge`) эту ветку в `develop`. После этого ветка `feature` обычно удаляется, она больше не нужна.

Почему тебе как тестировщику важно понимать эту «иерархию»?

1. **Откуда брать код для тестов?** Когда ты приступаешь к тестам, тебя спросят: «На какой ветке тестируем?».
 - Если ты тестируешь новую фичу — скорее всего, это будет **feature-ветка**.
 - Если ты делаешь комплексное регрессионное тестирование перед релизом — это будет **develop**.
 - Если ты проверяешь, что сломалось в «боевой» версии сайта — это **main**.
2. **Где найти баг?** Если ты нашел баг в ветке `feature`, ты идешь к конкретному программисту, который эту ветку делает. Если баг нашелся в `develop`, значит, где-то произошел конфликт между наработками разных программистов, и нужно разбираться всей командой.
3. **Безопасность:** Ты теперь знаешь: `main` — это «святое», его нельзя трогать. `feature` — это «черновик», там можно пробовать всё что угодно.

Что такое Docker?

Я понимаю, что Docker — это инструмент для упаковки приложений в контейнеры, чтобы они работали одинаково на любой машине. Я знаю, что это позволяет избежать проблем "у меня на компе не работает". В работе я умею запускать контейнеры (через `docker run`), останавливать их и знаю разницу между образом и контейнером. Если нужно развернуть окружение для тестов — я легко могу поднять нужный контейнер, например, с базой данных».

Итог: Docker — это то, что позволяет тестировщику перестать быть «настройщиком» (который часами возится с установкой софта) и стать «пользователем» готовых сред.

Итого: твой алгоритм действий на работе:

1. Спросить: «Где лежит проект и есть ли там `docker-compose.yml`?»
2. Открыть терминал, перейти в папку проекта.
3. Написать `docker compose up`.
4. Если побежали логи — всё супер.
5. Если ошибка — прочитать её (часто там написано, чего не хватает, например, «не найден файл») и отправить скриншот ошибки программисту.

Что такое JMeter?

Я понимаю, что JMeter предназначен для нагрузочного тестирования. Я знаю, как создать базовый сценарий: задать количество виртуальных пользователей (Thread Group), добавить запросы к серверу (Samplers) и посмотреть результаты в отчетах (Listeners). Я понимаю, что с помощью JMeter мы проверяем, не упадет ли сервис под нагрузкой и как быстро он отвечает при большом количестве запросов. Если потребуется, я смогу разобраться в настройках сценария или запустить уже готовый тест».

Зачем тебе как тестировщику знать JMeter?

Ты не будешь заниматься этим каждый день, но в компаниях часто просят «прогнать простую нагрузку». Тебе важно понимать:

1. **Выдержит ли сайт высокую нагрузку?**
2. **Насколько быстро отвечает сервер**, когда на нем 100 человек одновременно?
3. **Где «узкое горлышко»?** (Может, сайт тормозит не из-за кода, а из-за того, что база данных слишком медленная).

Как это работает (база):

В JMeter есть 3 главных понятия, которые тебе встретятся:

- **Thread Group (Группа потоков):** Это «толпа». Здесь ты задаешь, сколько «покупателей» (виртуальных юзеров) будет одновременно «ломиться» на сайт.
- **Samplers (Сэмплеры):** Это «действия». Ты говоришь программе: «Каждый пользователь должен зайти на главную страницу» или «Нажать кнопку Купить».
- **Listeners (Слушатели):** Это «графики и отчеты». Они показывают, сколько времени занял ответ сервера. Если время ответа растет — значит, система не справляется.

Как тестировать desktop-приложения?

«Я понимаю, что тестирование десктопа требует больше внимания к взаимодействию с ОС, чем веб. Я уделяю особое внимание процессу установки/удаления, чтобы программа не оставляла "мусор". Обязательно проверяю использование ресурсов через Диспетчер задач (чтобы не было утечек памяти) и корректность поведения интерфейса при изменении масштабирования системы. Также я знаю, что нужно проверять работу приложения в условиях нестабильной сети или её отсутствия»

1. Жизненный цикл «Установка — Удаление — Обновление»

Это первое, что должен проверить QA. Если программа не встанет или не удалится корректно — остальное не имеет смысла.

- **Установка:** Ставится ли программа на разные версии Windows (или macOS)? Есть ли права администратора? Что будет, если на диске кончилось место посередине установки?
- **Удаление:** Оставляет ли программа «мусор»? Хорошая программа должна удалять свои файлы, ключи реестра и временные папки.
- **Обновление:** Что будет, если накатить новую версию поверх старой? Не потеряются ли пользовательские настройки?

2. Взаимодействие с ОС

Desktop-приложение живет глубоко внутри системы.

- **Файловая система:** Куда программа сохраняет данные? (В Program Files, в AppData или где-то еще?). Что будет, если у пользователя нет прав на запись в эту папку?
- **Реестр (для Windows):** Программа не должна «мусорить» в системном реестре так, чтобы от этого тормозил весь компьютер.
- **Горячие клавиши:** Работают ли они корректно? Не конфликтуют ли они с другими программами (например, не перехватывает ли твое приложение Ctrl+C, мешая копировать текст в других окнах)?

3. Ресурсы и производительность

Если веб-сайт тормозит, пользователь просто закроет вкладку. Если desktop-приложение «ест» 100% процессора, оно вешает **весь компьютер**.

- **Диспетчер задач:** Всегда держи его открытым. Следи, сколько оперативной памяти (RAM) и процессора (CPU) отъедает программа в покое и под нагрузкой.
- **Утечки памяти:** Если программа открыта 24 часа и начинает «кушать» всё больше и больше памяти — это баг (memory leak).

4. UI/UX (Внешний вид)

Здесь всё не так просто, как в браузере.

- **Масштабирование:** Что будет, если у пользователя включен масштаб экрана 125% или 150%? Часто текст «едет», а кнопки исчезают.
- **Разные мониторы:** Проверь поведение программы при переносе с 4K-монитора на обычный HD-монитор.
- **Свертывание/Развертывание:** Не «ломается» ли интерфейс при изменении размера окна?

5. Сеть и автономность

- **Offline-режим:** Как ведет себя программа, если пропал интернет? Она должна вежливо сообщить об этом, а не просто закрыться с ошибкой («крэшнуться»).
- **Переподключение:** Восстанавливается ли работа после того, как интернет снова появился?

Твои лучшие друзья (инструменты) для десктопа:

- **Диспетчер задач (Task Manager):** База для проверки производительности.
- **Process Monitor (ProcMon):** Очень мощная штука от Microsoft. Показывает, к каким файлам и веткам реестра обращается твое приложение в реальном времени. Если программа что-то не сохраняет — ты увидишь там ошибку доступа.
- **Утилиты для снятия логов:** Если приложение «падает», оно часто создает файл dump (дамп). Нужно уметь его находить и передавать разработчикам, чтобы они поняли, где именно код «споткнулся».

Что такое автоматизация тестирования?

Автоматизация тестирования — это **написание программ (скриптов), которые проверяют другие программы вместо тебя**.

Представь, что тебе нужно каждый раз перед выпуском новой версии сайта проверять, работает ли кнопка «Купить».

- **Ручное тестирование:** Ты идешь на сайт, логинишься, кликаешь кнопку, смотришь результат. Это занимает 2 минуты.

- **Автоматизация:** Ты пишешь скрипт (код), который делает то же самое за 2 секунды. Ты нажимаешь одну кнопку, и «робот» за тебя проверяет эту кнопку, форму регистрации и логин.

Зачем это нужно? (Главные причины)

1. **Регрессионное тестирование (самое важное):** Когда программисты что-то чинят в одной части сайта, они могут случайно сломать другую. Руками всё проверять каждый день — свихнуться можно. Автотесты прогоняют весь функционал за 10 минут, пока ты пьешь кофе.
2. **Скорость:** «Робот» не спит, не устает и не ошибается, когда делает рутину.
3. **Покрывтие:** Можно запустить тысячи тестов на разных браузерах одновременно. Человеку это не под силу.

«Пирамида тестирования» (Это должен знать каждый QA)

Тимлиды обожают этот вопрос. Тестирование строится как пирамида:

- **Верхушка (UI-тесты / E2E):** Тестируют интерфейс «как пользователь». Это самые дорогие и медленные тесты (например, **Playwright, Cypress, Selenium**). Их не должно быть много.
- **Середина (API-тесты):** Проверяют обмен данными между сервером и клиентом. Быстрые и надежные. (Инструменты: **Postman, RestAssured**).
- **Основа (Unit-тесты):** Проверяют маленькие кусочки кода (функции). Их пишут в основном разработчики. Они самые быстрые и дешевые.

Золотое правило: Автоматизируй то, что не меняется (стабильные фичи), и то, что нужно проверять часто (рутина). Не автоматизируй то, что меняется каждый день — замучаешься переписывать код.

Что стоит автоматизировать?

✓ЧТО СТОИТ АВТОМАТИЗИРОВАТЬ (Зеленый свет)

1. **Регрессионное тестирование (Топ-1)** Это проверка того, что старые функции не сломались после правок в коде. Если разработчик добавил кнопку «Оплатить», а из-за этого перестала работать «Регистрация» — автотесты должны мгновенно это показать.
2. **Smoke-тесты (Дымовые тесты)** Самые важные проверки: «Сайт открывается?», «Логин работает?», «Корзина не пустая?». Если эти тесты упали — дальше тестировать нет смысла.
3. **Рутинные и скучные задачи** Например, создание 100 тестовых аккаунтов, заполнение длинной анкеты 50 раз с разными данными, загрузка файлов. Компьютер сделает это за секунды и без ошибок.
4. **Тестирование на разных конфигурациях** Проверить одну и ту же форму на 5 разных браузерах (Chrome, Safari, Firefox и т.д.) и 3 размерах экрана. Человеку это делать мучительно долго, для кода — пара кликов.
5. **Critical Path (Критический путь)** Путь пользователя, который приносит деньги компании. Например, сценарий: *Открыть сайт -> Найти товар -> Положить в корзину -> Оплатить*. Если этот путь упадет — компания теряет деньги. Это должно быть автоматизировано всегда.

✗ЧТО НЕ СТОИТ АВТОМАТИЗИРОВАТЬ (Красный свет)

1. **Нестабильные функции (UI в активной разработке)** Если дизайн страницы меняется каждый день (кнопки переезжают, цвета меняются), ты будешь тратить 90% времени на «починку» своих тестов. Автоматизируй только то, что «заморожено» и работает стабильно.
2. **Тестирование «один раз»** Если тебе нужно проверить какую-то редкую акцию, которая пройдет завтра и больше никогда не повторится — сделай это руками. Написание автотеста займет больше времени, чем ручной тест.

3. **UX и визуальное восприятие** Код может проверить, есть ли кнопка на странице, но он не может сказать: «*Эта кнопка выглядит уродливо и находится в неудобном месте*». Ощущения, удобство, дизайн — это работа для человеческих глаз.
4. **Исследовательское тестирование (Exploratory testing)** Это когда ты «ходишь» по сайту, пробуешь странные комбинации, пытаешься его сломать креативными способами. Компьютер делает только то, что ему прописали. Найти неочевидный баг может только человек с его «интуицией».

Что такое CI/CD?

CI/CD — это **конвейер сборки и доставки кода**.

Если **Git** — это просто хранилище кода, а **Docker** — способ упаковать код, то **CI/CD** — это автоматическая «фабрика», которая берет код у программиста, тестирует его, «упаковывает» и сама выкладывает на сайт к пользователям.

Без CI/CD программист должен вручную всё проверить, собрать файлы, загрузить их на сервер... это долго и ведет к ошибкам. CI/CD делает это за него за считанные минуты.

Разбираем аббревиатуру:

1. CI (Continuous Integration — Непрерывная интеграция)

Это этап, где всё проверяется. Как только программист нажал кнопку «Сохранить» (сделал push кода в Git):

- **Автосборка:** Сервер (например, Jenkins, GitLab CI, GitHub Actions) сам собирает проект.
- **Автотесты:** Система автоматически запускает твои автотесты.
- **Результат:** Если какой-то тест «упал» (нашел баг), вся команда моментально получает уведомление: «Ребята, кто-то сломал логин!». Это не дает плохому коду попасть в общую кучу.

2. CD (Continuous Delivery / Deployment — Непрерывная доставка/развертывание)

Это этап, где код превращается в работающий продукт.

- **Delivery:** Код готов, все тесты пройдены, и он ждет нажатия кнопки «Выкатить на сайт» (или выкатывается на тестовый стенд автоматически).
- **Deployment:** Код летит на реальный сайт к пользователям **без участия человека**. В крупных компаниях (как Facebook или Netflix) изменения выкатываются сотни раз в день.

Почему это важно для тестировщика?

1. **Ты всегда тестируешь «свежак»:** Как только программист что-то поправил, через 5-10 минут это уже стоит на тестовом сервере. Не нужно ждать, пока кто-то вручную выложит новую версию.
2. **Ты видишь результаты автотестов:** Твоя задача часто — смотреть, не «упал» ли какой-то тест в CI/CD-конвейере. Это твой главный «монитор здоровья» продукта.
3. **Ты становишься частью процесса:** Если автотесты падают в CI/CD, ты должен уметь зайти в логи этого «конвейера» (например, в **Jenkins** или **GitLab**) и посмотреть: «Это код плохой или автотест глючит?».

Как это выглядит для тебя (пример из жизни):

1. Программист вносит правки.
2. CI/CD система видит это и запускает твой набор автотестов.

3. Если тест падает, тебе (или в чат в Telegram/Slack) приходит сообщение: *«Build Failed: кнопка купить не работает»*.
4. Ты заходишь в систему CI/CD, смотришь логи, подтверждаешь, что это реальный баг, и заводишь тикет в Jira.

Что такое Selenium?

Selenium — это самый известный в мире «робот» для браузера. Это база, с которой начиналась вся автоматизация веб-тестирования.

Если в двух словах: **Selenium** — это библиотека, которая умеет открывать браузер, нажимать кнопки, вводить текст в поля и проверять, что написано на экране, вместо тебя.

Как это работает (на пальцах)?

Представь, что ты пишешь код, который дает браузеру команды:

1. «Открой сайт `google.com`».
2. «Найди строку поиска».
3. «Введи туда слово "тестирование"».
4. «Нажми кнопку Enter».
5. «Проверь, что на экране появилось слово "Wikipedia"».

Без Selenium тебе пришлось бы делать это руками. С Selenium твой скрипт делает это за секунду.

Почему Selenium — это «легенда»?

- **Всеядность:** Он работает с любыми браузерами (Chrome, Firefox, Safari, Edge) и на любых операционных системах.
- **Мультиязычность:** Ты можешь писать тесты на Python, Java, C#, Ruby, JavaScript — на чем угодно, Selenium «поймет» тебя везде.
- **Гигантское сообщество:** За 20 лет его существования для него написали столько инструкций и готовых решений, что если у тебя возникла ошибка — ответ на неё 100% есть в интернете.

Почему про Selenium говорят «старый, но надежный»?

Сейчас появились новые инструменты — **Playwright** и **Cypress**. Они быстрее, современнее и умеют делать много вещей «из коробки» (например, ждать, пока страница загрузится).

- **Playwright** сейчас считается «круче» и современнее.
- **Selenium** считается «классикой».

Что такое Playwright?

Если **Selenium** — это легендарный «дедушка» автоматизации, то **Playwright** — это современный, мощный и очень быстрый «спорткар».

Это инструмент для автоматизации веб-тестирования, который сейчас активно вытесняет Selenium в новых проектах. Его разработала компания **Microsoft**, и он создавался с учетом всех «болей», с которыми тестировщики сталкивались годами.

Почему Playwright — это «любовь» большинства современных QA?

1. **Он сам «ждет» элементы:** В Selenium была главная боль: тест падал, потому что страница не успела загрузиться (нужно было писать сложные «ожидания»). Playwright «умный»: он сам понимает, что кнопка появилась, и нажимает на неё только тогда, когда она стала доступна. Тесты падают гораздо реже.
2. **Скорость:** Он работает быстрее Selenium, так как общается с браузером напрямую, без «посредников» (драйверов).
3. **Тестирование «в одной вкладке» (Параллельность):** Playwright умеет запускать тесты параллельно настолько легко, что твой компьютер не «умирает». Он создает «изолированные контексты» (как будто разные пользователи зашли с чистого листа), что делает тесты стабильными.
4. **Сразу всё «из коробки»:**
 - **Trace Viewer:** Если тест упал, Playwright делает запись экрана и скриншоты *каждого шага*. Ты можешь «промотать» тест назад, как видео, и увидеть, где именно была ошибка.
 - **Генератор кода:** У него есть встроенный рекордер. Ты просто кликаешь мышкой по сайту, а Playwright сам пишет код теста за тебя.

Что такое Cypress?

Cypress — это еще один «мощный инструмент» для автоматизации тестирования веба, который стоит в одном ряду с Playwright.

Если Playwright — это «современный спорткар», то **Cypress** — это «уютный, удобный и очень дружелюбный городской электромобиль». Он был создан специально для фронтенд-разработчиков и тестировщиков, чтобы делать тесты прямо внутри браузера.

Чем он крут и почему его любят?

1. **«Магия» внутри браузера:** В отличие от Selenium или Playwright, которые управляют браузером «снаружи», Cypress запускается **прямо внутри браузера**. Это дает ему «суперсилу»: он видит всё, что происходит внутри веб-страницы в реальном времени.
2. **Время путешествия (Time Travel):** Это киллер-фича Cypress. Ты можешь нажать на любой шаг своего теста, и Cypress «отмотает» состояние сайта назад к тому моменту, когда выполнялся этот шаг. Ты видишь, как сайт выглядел, какие данные были в базе — это божественно удобно при отладке.
3. **Невероятно удобная отладка:** Интерфейс Cypress выглядит как панель управления. Ты видишь свой тест слева, а сайт — справа. Если что-то упало, ты прямо в браузере можешь открыть консоль разработчика и посмотреть, что пошло не так.
4. **Никакой головной боли с ожиданиями:** Как и Playwright, он «умный» и сам ждет, пока элемент станет доступен.

В чем подвох? (Сравнение с Playwright)

Cypress был первым, кто сделал автоматизацию такой простой, но у него есть ограничения, о которых стоит знать:

- **Ограничение браузеров:** Долгое время Cypress умел работать только с Chrome. Сейчас он поддерживает другие, но Playwright в этом плане всё равно более «мультиплатформенный».
- **Ограничение вкладок:** Cypress исторически плохо работал с несколькими открытыми вкладками или окнами (хотя это активно дорабатывается).
- **Скорость:** Playwright считается чуть быстрее за счет своей архитектуры.

Appium — это тот же **Selenium**, но для **мобильных приложений**.

Если Selenium/Playwright — это «роботы» для сайтов в браузере, то Appium — это «робот» для твоих приложений на смартфоне (iOS или Android).

Как это работает?

Appium не «взламывает» телефон. Он работает как посредник:

1. Твой код посылает команду: «Нажми на кнопку в приложении».
2. Appium переводит эту команду на язык системы (Android или iOS).
3. Система выполняет действие на реальном телефоне или на эмуляторе (виртуальном телефоне в компьютере).

Почему Appium — это стандарт для мобилок?

- **Один код для всего:** Ты пишешь один скрипт, и он работает и на Android, и на iOS. (Хотя есть нюансы, но общая логика одна).
- **Не нужно менять код приложения:** Тебе не нужно вшивать в приложение какие-то специальные «шпионские» модули, чтобы Appium им управлял. Он работает с «черным ящиком» — тем же файлом приложения (.apk или .ipa), которое скачивает обычный пользователь.
- **Поддерживает всё:** Можно тестировать:
 - **Native-приложения** (написанные на Java/Kotlin для Android или Swift для iOS).
 - **Hybrid-приложения** (это когда внутри приложения «живет» веб-сайт).
 - **Мобильный веб** (тестирование сайтов прямо в браузере Safari или Chrome на телефоне).

Что такое Page Object Model?

Page Object Model (POM) — это не программа и не библиотека. Это **правило хорошего тона** (шаблон проектирования), которое помогает держать код твоих автотестов в чистоте и порядке.

Если совсем просто: **POM** — это когда для каждой страницы сайта ты создаешь отдельный файл со списком кнопок и полей.

Проблема: Как пишут тесты новички (Без POM)

Представь, что у тебя есть 10 разных тестов. И в каждом тесте робот должен сначала зайти на страницу логина, ввести пароль и нажать кнопку «Войти». В коде это выглядит так: *«Найди поле с ID 'login', введи текст, найди кнопку с классом 'btn-submit', кликни»*.

И тут разработчики меняют дизайн: кнопка «Войти» теперь называется не `btn-submit`, а `login-button`.

- **Итог:** Все твои 10 тестов падают. Тебе приходится заходить в каждый из 10 файлов и вручную менять старое название кнопки на новое. А если тестов 500? Ты просто уволишься.

Решение: Как это работает с POM

Ты создаешь один отдельный файл (класс) и называешь его, например, `LoginPage`. Inside этого файла ты прописываешь:

- «Поле логина — это элемент X»
- «Кнопка Войти — это элемент Y»
- «Функция: ввести логин, пароль и нажать кнопку»

А в самих тестах ты больше не пишешь технические детали (ID кнопок, селекторы). Ты просто пишешь: `LoginPage.login(user, password)`.

Что происходит, если разработчики меняют кнопку? Ты заходишь в один файл `LoginPage`, меняешь ID кнопки в одном месте, и — магия! — все твои 500 тестов снова работают.

3 главных плюса POM:

1. **Легко поддерживать:** Код элементов страницы лежит в одном месте. Изменился сайт — изменил один файл, тесты не трогаешь.
2. **Тесты легко читать:** Твой тест превращается в понятные шаги: `MainPage.open()`, `MainPage.goToCart()`, `CartPage.checkout()`. Это может прочитать даже человек, не знающий программирования.
3. **Код не дублируется:** Ты не пишешь поиск одной и той же кнопки «Выход» в двадцати разных тестах.

Какие бывают паттерны автоматизации?

Паттерны (шаблоны) автоматизации — это проверенные временем архитектурные подходы, которые помогают писать тесты так, чтобы они не превращались в кашу, легко читались и не ломались от каждого изменения в коде разработчиков.

Page Object Model (POM), о котором мы говорили, — самый популярный паттерн, но далеко не единственный. Давай разберем другие важные паттерны, которые часто используют в реальных проектах.

1. Page Factory (Фабрика страниц)

Это близкий родственник POM, его встроенная модификация (особенно популярная в Java + Selenium).

- **Суть:** Вместо того чтобы вручную инициализировать элементы страницы, ты используешь специальные аннотации (например, `@FindBy` в Java). Элементы ищутся на странице не в момент создания объекта, а **только в момент обращения к ним** (ленивая инициализация).
- **Зачем:** Делает код еще более лаконичным, но в современных инструментах (вроде Playwright) используется реже, так как они умеют делать это из коробки.

2. Screenplay Pattern (Паттерн «Сценарий»)

Это «POM на стероидах» для очень больших проектов, где даже Page Object начинает раздуваться. Он ориентирован не на *страницы*, а на *актеров* и их *активности*.

- **Суть:** Вместо описания страниц ты описываешь:
 - **Актёр (Actor):** Кто делает действие (например, «Покупатель» или «Админ»).
 - **Способность (Ability):** Что он может (например, «Имеет доступ к браузеру»).
 - **Задача (Task):** Крупное действие («Оформить заказ»).
 - **Действие (Action):** Мелкий шаг («Нажать на кнопку»).
- **Зачем:** Тесты становятся похожи на сценарий фильма на чистом английском языке. Этот паттерн невероятно гибкий, но сложнее в реализации для новичков.

3. Data-Driven Testing (DDT — Тестирование на основе данных)

Представь, что тебе нужно протестировать форму калькулятора кредитов. Тебе нужно проверить 50 разных комбинаций цифр. Писать 50 отдельных тестов — безумие.

- **Суть:** Ты пишешь **один** тест-скрипт, а все тестовые данные выносишь в отдельный файл (Excel, CSV, JSON или базу данных). Тест запускается по кругу, каждый раз подтягивая новую строку с данными из таблицы.

- **Зачем:** Разделяет логику теста и сами данные. Позволяет легко расширять тесты: чтобы добавить новую проверку, не нужно писать код, достаточно дописать строчку в Excel.

4. Keyword-Driven Testing (Тестирование на основе ключевых слов)

Этот паттерн часто используют, чтобы ручные тестировщики могли писать автотесты, не зная языков программирования.

- **Суть:** Программисты пишут код для базовых действий и называют их ключевыми словами: `OpenBrowser`, `ClickButton`, `EnterText`. А тестировщик в специальной таблице (или в таком фреймворке, как **Robot Framework**) собирает тест из этих слов, как конструктор.
- **Зачем:** Позволяет команде без навыков кодинга участвовать в автоматизации.

5. Fluent Interface / Chain of Responsibility (Текущий интерфейс)

Этот паттерн делает код тестов безумно красивым и читаемым за счет «цепочек» методов.

- **Суть:** Каждый метод страницы возвращает саму эту страницу (или следующую). Благодаря этому в тесте можно вызывать команды через точку друг за другом.
- **Пример в коде:**

JavaScript

```
// Вместо трех строк пишем одну красивую цепочку
loginPage.open()
    .enterUsername('admin')
    .enterPassword('12345')
    .clickSubmit();
```

- **Зачем:** Повышает читаемость кода, тест выглядит как единый непрерывный поток.

Как правильно работать с ожиданиями в автотестах?

Работа с ожиданиями (Waits) — это **самая главная тема**, от которой зависит, будут ли твои тесты стабильными или превратятся в «кошмар автоматизатора».

Основная проблема веб-приложений: элементы на странице появляются не мгновенно. Картинка может грузиться 2 секунды, кнопка — активироваться после ответа сервера, а спиннер — крутиться неопределенное время. Если робот попытается кликнуть по кнопке, которой еще нет в коде, тест упадет с ошибкой `NoSuchElementException`.

Давай разберем, как с этим бороться правильно, а как — категорически нельзя.

✗Главный грех: Жесткие ожидания (`thread.sleep` или `time.sleep`)

Это первое, что делают новички. Они пишут в коде: *«Подожди ровно 5 секунд, а потом кликни»*.

- **Почему это плохо:** Если страница загрузилась за 1 секунду, тест всё равно будет тупо сидеть и ждать еще 4 секунды. А если у тебя 100 тестов? Ты будешь ждать результаты часами. Если же в этот день интернет лагал, и страница грузилась 6 секунд — тест всё равно упадет.
- **Правило:** Забудь про `sleep`. В реальных проектах за жесткие ожидания бьют по рукам на ревью кода.

□ Как надо: Динамические ожидания

Динамические ожидания «умные»: они ждут **до** указанного времени, но как только элемент появился — тест сразу идет дальше. Если ты задал лимит в 10 секунд, а кнопка появилась за 0.5 секунды, тест выполнится мгновенно.

В автоматизации (особенно в Selenium) динамические ожидания делятся на два типа:

1. Неявные ожидания (Implicit Waits)

Это глобальная настройка для всего браузера. Ты один раз в начале проекта говоришь: *«Робот, если ты не видишь элемент, подожди его 10 секунд, прежде чем паниковать»*.

- **Плюс:** Пишется одна строчка кода на весь проект.
- **Минус:** Робот будет ждать вообще *каждый* элемент, которого нет (даже если его там и не должно быть по сценарию). Это замедляет сложные проверки.

2. Явные ожидания (Explicit Waits) — Самый правильный подход

Ты настраиваешь ожидание для **каждого конкретного случая** и ждешь не просто «появления в коде», а конкретного состояния (Expected Conditions).

- *«Жди, пока кнопка станет кликабельной»*
- *«Жди, пока текст в поле изменится на "Успешно"»*
- *«Жди, пока исчезнет иконка загрузки (лоадер)»*

🔧 Современный подход: Как это устроено в Playwright и Cypress?

Разработчики современных инструментов (Playwright, Cypress) тоже страдали от ожиданий в Selenium. Поэтому они придумали **Auto-waiting (Авто-ожидания)**.

Когда ты пишешь в Playwright:

JavaScript

```
await page.click('button#submit');
```

Playwright под капотом **сам автоматически** проверяет целый чек-лист:

- Есть ли элемент в DOM-дереве?
- Виден ли он на экране (не скрыт ли)?
- Не перекрыт ли он другой анимацией?
- Стал ли он кликабельным?

То есть в современных инструментах 90% проблем с ожиданиями решены «из коробки». Вручную прописывать явные ожидания приходится только для очень сложных сценариев (например, дождаться окончания загрузки файла или специфического ответа от сервера).

Что должен знать Junior QA?

быстро учиться, снижать рутину команды и качественно находить баги, используя базу тест-дизайна. Я хорошо освоил теорию, умею работать с DevTools и Postman для проверки API, могу написать SQL-запрос к базе данных и заглянуть в логи на сервере через CLI. Я понимаю, как устроен процесс разработки, и готов развиваться дальше, в том числе в сторону автоматизации, если это потребуется проекту».

1. Теория тестирования (Фундамент)

Это то, на чем тебя будут «гонять» по определениям. Ты должен не просто зубрить, а понимать суть:

- **Жизненный цикл разработки (SDLC) и тестирования (STLC):** Кто за кем идет, когда начинается тестирование (подсказка: на этапе анализа требований!).
- **Виды тестирования:** Чем функциональное отличается от нефункционального, что такое Smoke (дымовой), Регресс (проверка старого) и Санити (санитарная проверка).
- **Техники тест-дизайна (Топ-вопрос):** Ты обязан уметь применять **Эквивалентное разделение** и **Анализ граничных значений**. На собеседовании обязательно дадут задачку типа: «*Есть поле для ввода возраста от 18 до 60. Какие значения ты введешь для проверки?*».
- **Тест-документация:** Как писать понятные чек-листы, тест-кейсы (без «воды») и баг-репорты. Ты должен четко знать структуру бага: *Summary, Steps to Reproduce, Expected Result, Actual Result*.

2. Технический минимум (Твои инструменты)

Ручной тестировщик, который умеет только «кликать мышкой», рынку не нужен. Ты должен уметь заглядывать «под капот» системы:

- **DevTools (Панель разработчика в браузере):** Вкладка *Elements* (посмотреть верстку), *Console* (увидеть ошибки JS) и особенно *Network* (смотреть, какие запросы уходят на сервер и какие приходят ответы).
- **Тестирование API (Postman):** Что такое клиент-серверная архитектура, разница между методами GET, POST, PUT, DELETE. Умение отправить простой запрос в Postman и проверить статус-код (200 OK, 404 Not Found, 500 Server Error).
- **Базы данных (SQL):** Ты должен уверенно писать простые запросы: SELECT, WHERE, ORDER BY, INSERT, UPDATE и понимать, как работают базовые объединения таблиц (JOIN).
- **Командная строка (CLI):** Базовые команды Linux/Terminal: как зайти на сервер, посмотреть логи (`tail -f`), переместиться по папкам (`cd, ls`).

3. Понимание процессов (Кругозор)

Ты должен понимать язык, на котором говорит IT-команда:

- **Гибкие методологии (Agile/Scrum):** Что такое спринты, ежедневные созвоны (Daily), планирование и ретроспектива.
- **Jira / Confluence / Test IT:** Понимание, как устроены таск-трекеры и системы ведения тест-кейсов.
- **Git:** Зачем он нужен разработчикам, что такое ветки (branches), коммиты и пулл-реквесты (хотя бы на концептуальном уровне).

4. Понимание «взрослых» концепций (Твой бонус)

Тебе **не нужно** уметь кодить или настраивать сервера с нуля, но если ты сможешь объяснить простыми словами, что это такое — ты моментально обойдешь 80% конкурентов:

- **Docker** (запуск приложения в изолированных коробках-контейнерах).
- **CI/CD** (автоматический конвейер, который тестирует и выкатывает код на сайт).
- **Автоматизация** (понимание «Пирамиды тестирования» и зачем нужны автотесты).

□ Главный секрет: Soft Skills (Мягкие навыки)

Для джуна они важнее хард-скиллов на 50%. Тимлид думает: «*Техническим штукам я его научу за месяц, а вот смогу ли я с ним нормально общаться каждый день?*».

- **Внимательность к деталям:** Не пропускай опечатки в требованиях.
- **Умение задавать вопросы:** Не сиди часами над проблемой. Попробуй разобраться сам 15–20 минут, если не вышло — иди с четко сформулированным вопросом к коллегам.
- **Здоровый скептицизм:** Хороший QA не верит на слово фразе разработчика «Я всё починил, проверять не надо».

Чем сильный Junior QA отличается от слабого?

Вот 5 ключевых отличий, по которым сразу видно «сильного» новичка:

1. Подход к баг-репортам и локализации

- **Слабый Junior:** Нашел баг (например, не нажимается кнопка «Купить») -> Сразу бежит писать баг-репорт: *«Кнопка не работает, почините»*.
- **Сильный Junior:** Проведет мини-расследование. Он откроет **DevTools**, зайдет во вкладку **Network** и посмотрит, что происходит в момент клика.

В баг-репорте сильного джуна будет написано: *«Кнопка "Купить" не реагирует на клик. В консоли ошибка 500 от эндпоинта /api/v1/cart/add. Логи сервера прикрепил»*. Разработчик за такой баг-репорт готов расцеловать, потому что проблема локализована.

2. Умение задавать вопросы (Правило 20 минут)

- **Слабый Junior:** Сталкивается с проблемой (например, не запускается Docker-контейнер). Варианта два: либо он мгновенно бежит отвлекать старшего коллегу по любому пустяку, либо молча сидит три дня, боясь признаться, что застрял.
- **Сильный Junior:** Использует «правило 20 минут». Сначала он сам гуглит ошибку, смотрит логи, ищет решение в корпоративной базе знаний (Confluence). Если через 20 минут решение не найдено, он приходит к ментору с готовой структурой:

«Привет! У меня не поднимается Docker-контейнер, выдает ошибку X. Я уже проверил порты, перезапустил демона и попробовал сделать то-то, но не помогло. Подскажи, куда еще можно посмотреть?»

3. Технический кругозор («Под капот» vs «Поверхность»)

- **Слабый Junior:** Тестирует только то, что видит глаз (интерфейс). Если поле принимает цифры от 1 до 10, он проверит 1, 10, 11 и успокоится. Он боится баз данных, терминала и Postman как огня.
- **Сильный Junior:** Понимает, как устроена система целиком. Он знает, что данные из формы летят через **API** в **Базу данных**. Поэтому после отправки формы он зайдет в базу через SQL-запрос и перепроверит: а действительно ли данные записались корректно? Не обрезалась ли строка?

4. Работа с требованиями и критическое мышление

- **Слабый Junior:** Воспринимает ТЗ (техническое задание) как абсолютную истину. Написано «кнопка должна быть синей» — он проверит, что она синяя. Если в ТЗ забыли описать, что будет, если пользователь введет отрицательное число, слабый джун это просто пропустит.
- **Сильный Junior:** Обладает здоровым скептицизмом. Он ищет «серые зоны» в требованиях до того, как программисты написали код. Он спросит на планировании: *«Ребята, а в ТЗ не указано, как должна вести себя система, если у пользователя в этот момент пропадет интернет. Мы выдаем ошибку или сохраняем данные локально?»*.

5. Отношение к рутине

- **Слабый Junior:** Смиренно (или с ворчанием) изо дня в день делает одну и ту же скучную ручную работу, превращаясь в кликающего биоробота.
- **Сильный Junior:** Стремится к оптимизации. Если ему нужно каждый день заводить 20 тестовых пользователей, он не поленится, потратит вечер, разберется с Postman (напишет простенький скрипт коллекции) или напишет базовый автотест, чтобы генерировать их одной кнопкой.

Тестирование требований — это проверка текстовой документации (технического задания, макетов, спецификаций) **до того, как программисты написали хотя бы одну строчку кода.**

Это самый ранний этап тестирования, который также называют **статическим тестированием.**

Главный секрет: Зачем тестировать то, чего еще нет?

Есть золотое правило IT-индустрии: **чем раньше найден баг, тем дешевле его исправить.**

Представь ситуацию: аналитик написал в требованиях: *«Пользователь может перевести любую сумму денег со счета».*

- **Сценарий А (Без тестирования требований):** Программист пишет код. Тестировщик получает готовую программу, вводит сумму -100 рублей (минус сто) или 100 000 000 000 рублей. Система ломается, деньги исчезают. Приходится переписывать код, переделывать базу данных, двигать сроки релиза. Компания теряет кучу денег.
- **Сценарий Б (С тестированием требований):** Тестировщик читает эту строчку в ТЗ на этапе планирования и говорит аналитику: *«Слушай, а "любую" — это какую? Нужен ли лимит? Можно ли вводить отрицательные числа? Какая максимальная сумма перевода за раз?».* Аналитик бьет себя по лбу, дописывает три строчки в документ. Баг исправлен за 2 минуты с помощью клавиатуры.

Тестируя требования, ты ловишь баги «в зародыше», экономя недели работы программистов.

1. Завершенность (Completeness)

Требование должно описывать поведение системы во **всех возможных ситуациях**, а не только идеальный путь пользователя (Happy Path).

- **Плохой пример:** *«Пользователь вводит в поле сумму, нажимает кнопку "Купить", и с его карты списываются деньги».*
- **Где баг тестирования требований:** Аналитик описал только то, как всё работает в идеале. А что, если на карте нет денег? Что, если банк отклонил операцию? Что, если пользователь нажал кнопку пять раз подряд?
- **Правильный вариант:** Должны быть четко описаны альтернативные сценарии (Edge Cases) и сообщения об ошибках для каждого сбоя.

2. Однозначность (Unambiguity)

Текст должен быть написан так, чтобы у всей команды — аналитика, разработчика и тестировщика — в голове сложилась **одна и та же картинка.** Никаких двусмысленностей.

- **Плохой пример:** *«Интерфейс личного кабинета должен загружаться быстро».*
- **Где баг:** Понятие «быстро» у всех разное. Для разработчика 5 секунд — это быстро, а для пользователя — вечность. Или: *«Система должна поддерживать популярные браузеры».* Какие именно? Каких версий?
- **Правильный вариант:** *«Время отрисовки первого экрана личного кабинета (First Contentful Paint) не должно превышать 1.5 секунд при скорости соединения от 10 Мбит/с».*

3. Непротиворечивость (Consistency)

Внутри одного документа (или смежных документов проекта) требования **не должны конфликтовать друг с другом**.

- **Плохой пример:** В Главе 1 написано: *«Пароль пользователя должен состоять только из цифр для удобства ввода с мобильных устройств»*. А в Главе 4 читаем: *«Для повышения безопасности системы пароль должен содержать как минимум одну заглавную букву и спецсимвол»*.
- **Где баг:** Разработчик напишет код по Главе 4, а тестировщик заведет баг по Главе 1. Начнется спор.
- **Правильный вариант:** Требования должны быть согласованы между собой до начала кодинга.

4. Проверяемость / Тестируемость (Testability)

Ты должен физически иметь возможность **проверить, выполнено требование или нет**. Если критерий нельзя измерить цифрами или конкретным действием, он не тестируем.

- **Плохой пример:** *«Приложение должно быть максимально удобным и интуитивно понятным для людей старшего поколения»*.
- **Где баг:** Как ты это проверишь? Приведешь в офис бабушку? Удобство — понятие субъективное, его нельзя засунуть в тест-кейс со статусом Passed/Failed.
- **Правильный вариант:** *«Пользователь должен иметь возможность оформить заказ не более чем за 4 клика с главной страницы. Размер шрифтов в интерфейсе должен регулироваться в настройках от 12px до 24px»*.

5. Ясность (Clarity)

Требование должно быть написано **простым, четким языком**, без грамматических ребусов и предложений на полстраницы, где к концу текста забываешь, что было в начале.

- **Плохой пример:** *«В случае возникновения ситуации, при которой аутентификационный токен пользователя признается валидатором как скомпрометированный или имеющий истекший срок легитимности, сессионное взаимодействие должно быть немедленно terminated с последующей инициацией редиректа субъекта на корневую страницу аутентификации»*.
- **Где баг:** Написано слишком громоздко. Есть риск, что джун-разработчик запутается в терминах и сделает не то.
- **Правильный вариант:** *«Если токен авторизации просрочен или невалиден, система должна разлогинить пользователя и перенаправить его на страницу входа»*.

6. Актуальность и Прослеживаемость (Maintainability / Traceability)

Документация должна соответствовать **текущему состоянию продукта** и быть удобно структурирована.

- **Плохой пример:** Ты открываешь ТЗ, чтобы проверить форму регистрации, а там написано, что поле «Отчество» является обязательным. Но ты точно знаешь, что месяц назад на созвоне директор сказал убрать это поле, и разработчики его уже убрали. Документ просто забыли обновить.
- **Где баг:** Неактуальное ТЗ путает новых сотрудников и порождает ложные баг-репорты.
- **Прослеживаемость (Traceability):** Каждое требование должно иметь свой номер (например, REQ-001). Тогда в Jira ты сможешь привязать задачу разработчика к REQ-001, а в тестовой системе связать свой тест-кейс с этим же номером. Если требование REQ-001 изменится, ты сразу увидишь, какие тесты нужно переписать.

Белый ящик (White Box Testing) — это метод тестирования, при котором тестировщик **полностью видит и знает внутреннее устройство программы, её исходный код и архитектуру**.

Если при тестировании «черного ящика» (Black Box) ты ведешь себя как обычный пользователь — просто кликаешь на кнопки и смотришь на результат, то в «белом ящике» ты действуешь как хирург или автомеханик: открываешь капот и смотришь, как крутятся шестеренки и как написан код.

Где применяется «белый ящик»?

Тестирование белого ящика применяется на самых ранних этапах разработки и в основном находится на территории **разработчиков**, а не классических QA-инженеров.

- **Модульное тестирование (Unit Testing):** Это самый частый кейс. Разработчик написал небольшую функцию (например, расчет скидки в корзине) и тут же пишет специальный код-тест, который проверяет эту функцию изнутри.
- **Интеграционное тестирование (Integration Testing):** Проверка того, как внутренние модули программы или разные сервисы общаются друг с другом на уровне кода и интерфейсов передачи данных.
- **Тестирование безопасности (Security Testing):** Поиск уязвимостей, дыр в безопасности и скрытых путей обхода защиты прямо в исходном коде приложения (статический и динамический анализ кода).

Как он используется? (Основные техники)

При работе с «белым ящиком» задача — написать тесты так, чтобы они проверили как можно больше строчек реального кода. Для этого используют метрику **Покрытие кода (Code Coverage)** и следующие техники:

1. Покрытие операторов / строк кода (Statement Coverage)

Тесты составляются так, чтобы каждая строчка кода выполнялась хотя бы один раз.

Пример: Если в коде есть 10 строк, а твои тесты активировали только 8 строк, то покрытие кода составляет 80%. Задача — дописать тест, который заставит работать оставшиеся 2 строки.

2. Покрытие ветвей / условий (Branch / Decision Coverage)

В коде постоянно встречаются развилки: операторы `if / else` (если... то... иначе...). Техника требует, чтобы тест зашел в **каждое** ответвление этой развилки.

Давай посмотрим на простой пример кода (на языке JavaScript):

JavaScript

```
function checkAccess(age) {  
  if (age >= 18) {  
    return "Доступ разрешен"; // Ветвь №1  
  } else {  
    return "Доступ запрещен"; // Ветвь №2  
  }  
}
```

- Чтобы покрыть этот код по технике «белого ящика», тебе нужно написать минимум **два** теста:
 1. Один тест передаст `age = 20`, чтобы сработала Ветвь №1.
 2. Второй тест передаст `age = 15`, чтобы сработала Ветвь №2.

3. Покрытие путей (Path Coverage)

Самый сложный и глубокий уровень. Если в программе много развилок, они образуют комбинации путей. Тестирование путей пытается проверить все возможные маршруты, по которым может пройти программа от начала до конца.

Плюсы и минусы метода

Плюсы:

- **Раннее обнаружение багов:** Ошибки находятся до того, как появится графический интерфейс.
- **Оптимизация кода:** Помогает находить «мертвый код» — строчки, которые вообще никогда не выполняются и просто занимают место.
- **Точность:** Разработчик сразу видит, в какой именно строчке кода произошла ошибка.

Минусы:

- **Сложность и дороговизна:** Нужно уметь программировать на высоком уровне и досконально знать архитектуру проекта.
- **Не видит картину в целом:** Код может быть написан идеально и все Unit-тесты пройдут, но если аналитик неверно понял клиента и программа делает *вообще не то, что просил заказчик*, «белый ящик» этот баг не поймает. Тут нужен «черный ящик».

Черный ящик (Black Box Testing) — это метод тестирования, при котором тестирующий **вообще не знает и не видит, как устроена программа внутри**. У него нет доступа к исходному коду, архитектуре или базе данных.

Ты смотришь на программу точно так же, как обычный пользователь: видишь только внешний интерфейс (окна, кнопки, поля ввода).

Главный принцип «черного ящика»

Принцип работы строится на схеме: **Вход → Результат**.

[Входные данные] →  ЧЕРНЫЙ ЯЩИК (Программа) → [Ожидаемый результат]

Ты подаешь на вход системы какие-то данные и проверяешь, соответствует ли то, что программа выдала на экран, техническому заданию (ТЗ).

Пример: Ты тестируешь форму авторизации. Вводишь правильный логин и пароль (Вход) → Нажимаешь «Войти» → Тебя пустило в личный кабинет (Результат). Как именно программа проверяла этот пароль в коде, какие серверные функции вызывала — тебе абсолютно неважно.

Где применяется «черный ящик»?

Этот метод применяется **везде, где тестируется готовый продукт или его крупные части**. Это основная зона ответственности классических QA-инженеров (особенно ручных тестировщиков и автоматизаторов уровня UI).

- **Системное тестирование (System Testing):** Проверка всего приложения целиком (например, ты открываешь мобильное приложение банка на телефоне и пытаешься перевести деньги).
- **Приемочное тестирование (Acceptance Testing):** Финальная проверка продукта перед сдачей заказчику или релизом в сторы, чтобы убедиться, что он решает бизнес-задачи.
- **Функциональное тестирование:** Проверка того, работают ли функции сайта (поиск, корзина, фильтры, регистрация) так, как написано в требованиях.

Как он используется? (Техники тест-дизайна)

Поскольку ты не видишь код, ты не можешь проверить каждую его строчку. Чтобы не вводить миллиард разных комбинаций наобум, в «черном ящике» используют специальные техники (тест-дизайн) для выбора самых эффективных проверок:

1. **Эквивалентное разделение (Equivalence Partitioning):** Ты делишь все возможные входные данные на группы, где программа должна вести себя одинаково, и тестируешь только по одному значению из каждой группы.
 - *Пример:* Поле принимает скидочный код от 1 до 100%. Группы: Невалидные (меньше 1), Валидные (1–100), Невалидные (больше 100). Ты проверяешь всего три числа: 0, 50 и 105.
2. **Анализ граничных значений (Boundary Value Analysis):** Баги чаще всего обитают на границах разрешенных диапазонов. Поэтому ты проверяешь стыки групп.
 - *Пример:* Для того же диапазона 1–100% границами будут: 1 и 100 (валидные), а также 0 и 101 (невалидные).
3. **Таблица принятия решений (Decision Table):** Используется, когда поведение системы зависит от комбинации нескольких условий. Ты рисуешь таблицу, где перебираешь варианты (например: Пользователь залогинен? Да/Нет. Товар в наличии? Да/Нет. Кнопка «Купить» активна? Да/Нет).

Плюсы и минусы метода

Плюсы:

- **Близость к пользователю:** Тестировщик находит именно те баги, с которыми столкнется реальный человек.
- **Не нужно уметь кодить:** Для старта в Black Box достаточно понимать логику работы приложений и владеть теорией тестирования.
- **Независимость:** Тестировщик не привязан к коду. Разработчики могут полностью переписать внутренности программы с Java на Python, но для «черного ящика» тесты останутся прежними, если дизайн и логика кнопок не изменились.

Минусы:

- **Сложно найти скрытые дефекты:** Если в коде есть лишняя ветка (например, скрытая лазейка для разработчиков), через интерфейс «черного ящика» ее практически невозможно обнаружить.
- **Избыточность или пропуски:** Без взгляда на код сложно понять, все ли сценарии ты покрыл тестами.

Серый ящик (Grey Box Testing) — это идеальный компромисс и «золотая середина» между черным и белым ящиками.

При этом методе ты знаешь, как устроена система внутри (ее архитектуру, базу данных, API), но тестируешь ее через внешние интерфейсы, лишь частично заглядывая в код или логи. Ты уже не просто кликаешь по кнопкам, как обычный пользователь (черный ящик), но и не пишешь Unit-тесты сам (белый ящик).

Простая аналогия: ты не умеешь собирать двигатель автомобиля с нуля (белый ящик), но отлично знаешь, куда залить масло, как проверить предохранители и как прочитать код ошибки на бортовом компьютере, если загорелся *Check Engine*.

Где применяется «серый ящик»?

Это основной метод работы для сильного Junior QA и Middle QA. На реальной работе чистого «черного ящика» почти не осталось — от инженеров всегда требуют заглядывать под капот.

- **Тестирование API (через Postman/Charles/Fiddler):** Ты видишь структуру запросов (JSON/XML) и понимаешь, как сервер обрабатывает данные, но отправляешь их «снаружи», как клиент.
- **Тестирование баз данных (SQL):** Ты заполнил форму на сайте (черный ящик), а затем открыл базу данных и проверил, правильно ли там создались строка и заполнились ли нужные таблицы (серый ящик).
- **Интеграционное тестирование:** Проверка того, как сайт общается со сторонними сервисами (например, отправляет ли интернет-магазин запрос в платежную систему Сбербанка или Яндекс.Пэй и что получает в ответ).
- **Анализ логов:** Если на сайте выскочила ошибка 500, ты идешь на сервер через терминал (CLI) и читаешь логи, чтобы понять, в каком модуле произошел сбой.

Как он используется на практике?

При тестировании серого ящика ты используешь знания о внутреннем устройстве системы, чтобы спроектировать более хитрые и точные тесты для интерфейса.

Пример 1: Регистрация пользователя

- *Как бы проверял «черный ящик»:* Заполнил форму → Нажал «Зарегистрироваться» → Увидел на экране надпись «Успешно».
- *Как проверяет «серый ящик»:* Ты делаешь то же самое, но параллельно:
 1. Открываешь вкладку **Network** в DevTools и смотришь, ушел ли запрос `POST /api/v1/register` со всеми нужными полями.
 2. Пишешь **SQL-запрос** в базу данных: `SELECT * FROM users WHERE email = 'test@test.ru'` и проверяешь, что пароль в базе зашифрован (зашифрован), а не лежит в открытом виде.
 3. Смотришь в логи почтового сервера — ушло ли пользователю письмо для подтверждения.

Пример 2: Безопасность и доступы

Ты знаешь, что в системе есть роли: «Пользователь» и «Администратор». Ты заходишь под обычным пользователем (у которого нет кнопки «Удалить товар»), но вручную вставляешь в адресную строку браузера секретную ссылку админки `сайт.py/admin/delete`. Ты проверяешь, защищен ли этот внутренний путь от обычных юзеров.

Плюсы и минусы метода

Плюсы:

- **Высокая точность локализации багов:** Ты не просто говоришь «не работает», а точно указываешь разработчику: «База данных не принимает длинные имена» или «API возвращает неверный формат даты».
- **Умный тест-дизайн:** Зная, из каких модулей состоит программа, ты не тратишь время на лишние проверки, а бьешь точно в места стыков этих модулей.
- **Не нужно глубоко знать код:** Достаточно понимать схемы данных, архитектуру и уметь читать логи.

Минусы:

- **Покрывание кода всё еще неполное:** Ты не можешь проверить абсолютно все скрытые ветки алгоритмов внутри кода, как в белом ящике.
- **Требуется больше навыков:** От тестировщика уже ждут знаний SQL, REST API, DevTools и понимания клиент-серверной архитектуры.

ВИДЫ ТЕСТИРОВАНИЯ

Классификация видов тестирования огромна, но в реальной работе на собеседованиях все они делятся на **четыре главные группы**. Представь это как карту, по которой тестировщик выбирает, под каким углом сегодня проверить приложение.

1. По объекту тестирования (Что тестируем?)

Этот вид определяет, на какую часть системы направлены тесты.

- **Функциональное тестирование (Functional):** Проверка того, что программа делает то, что должна по ТЗ (работа кнопок, корзины, логики расчетов).
- **Нефункциональное тестирование (Non-functional):** Проверка характеристик системы, которые не относятся к конкретным фичам, но критичны для пользователя. Сюда входят:
 - *Тестирование производительности (Performance/Load):* Как сайт ведет себя под нагрузкой в 10 000 пользователей.
 - *Тестирование безопасности (Security):* Защищено ли приложение от хакеров и утечки данных.
 - *Тестирование удобства использования (Usability):* Насколько интерфейс понятен и удобен человеку.
- **Тестирование структуры (Связанное с кодом):** Это то самое тестирование «белого ящика», о котором мы говорили.

2. По запуску кода (Статическое vs Динамическое)

- **Статическое тестирование (Static):** Программа **не запускается**. Мы тестируем код глазами (ревью кода) или проверяем текстовые документы.
 - *Где и как:* Применяется на самых ранних этапах. Самый яркий пример — **тестирование требований** аналитиков, о котором мы подробно говорили ранее.
- **Динамическое тестирование (Dynamic):** Программа **запускается**. Ты вводишь данные, кликаешь по интерфейсу или отправляешь запросы к API и смотришь, как ведет себя работающая система.

3. По степени автоматизации (Кто кликает?)

- **Ручное тестирование (Manual):** Тесты выполняет человек. Ты своими руками по чек-листу вводишь данные и проверяешь результат. Идеально для проверки UX/UI и исследования новых фич.
- **Автоматизированное тестирование (Automation):** Вместо тебя тесты запускает специальный код (скрипт) с помощью инструментов вроде *Playwright*, *Cypress* или *Appium*. Используется для рутинных, часто повторяющихся проверок.

4. По степени знания системы (Уровень «Ящиков»)

Это наши старые знакомые, определяющие, заглядываем ли мы внутрь кода при тесте:

- **Черный ящик** (кода не видим, тестируем через интерфейс).
- **Серый ящик** (видим API, логи и базу данных, тестируем снаружи).
- **Белый ящик** (полностью тестируем код изнутри, пишем Unit-тесты).

5. По времени/цели проведения (Когда тестируем?)

Эта группа отвечает на вопрос: «В какой момент разработки мы запускаем тесты?».

Smoke-тестирование (Дымовой тест)

- **Где применяется:** Сразу после того, как программисты собрали новую версию приложения (билд) и выкатили её на тестовый сервер.
- **Как используется:** Проверяются только самые критические функции, без которых приложение вообще теряет смысл. Например, для интернет-магазина это: *открывается ли сайт, работает ли поиск, можно ли положить товар в корзину*. Если Smoke упал — билд возвращают разработчикам на доработку, дальше тестировать нет смысла.

Регрессионное тестирование (Regression)

- **Где применяется:** Когда программисты добавили новую фичу или исправили старый баг.
- **Как используется:** Ты проверяешь **уже работавшие ранее части сайта**, чтобы убедиться, что новые изменения ничего не сломали. Это самый частый кандидат на автоматизацию, так как регресс приходится гонять постоянно перед каждым релизом.

Монолит и микросервисы — это два принципиально разных подхода к архитектуре (устройству) программного обеспечения. Простыми словами: это то, как бэкенд (серверная часть) приложения устроен внутри и как его модули общаются между собой.

Давай разберем каждый подход, где они применяются и как тестировщику работать с каждым из них.

1. Монолитная архитектура (Монолит)

Монолит — это когда всё приложение (логика, работа с пользователями, оплата, база данных) написано как **один огромный, неделимый кусок кода**, который запускается на сервере как единое целое.

- *Аналогия:* Большой многофункциональный швейцарский нож. Если у него сломается штопор, тебе придется нести в ремонт весь нож целиком.

Где применяется:

- В стартапах и на ранних этапах проектов (когда нужно быстро написать продукт и проверить идею).
- В небольших и средних приложениях, где логика не слишком запутанная.

Как используется и тестируется:

- **Плюс для QA:** Тестировать монолит относительно просто. У тебя есть один сервер, одна база данных. Ты нажал кнопку на сайте → запрос ушел на сервер → данные записались в БД. Всё происходит в одном месте, связи прозрачны.
- **Минус для разработки:** Если в коде модуля «Оплата» допущена критическая ошибка, из-за нее может упасть вообще весь сайт, включая каталог и авторизацию. Чтобы обновить маленькую кнопку, приходится заново перезапускать и тестировать всё огромное приложение целиком.

2. Микросервисная архитектура (Микросервисы)

Микросервисы — это когда одно большое приложение разбивают на **десятки маленьких, независимых программ (сервисов)**. Каждый микросервис отвечает строго за одну задачу: один — за авторизацию, второй — за каталог товаров, третий — за отправку СМС, четвертый — за оплату. У каждого микросервиса может быть даже своя собственная база данных.

Между собой они общаются по сети (чаще всего через **REST API** или брокеры сообщений типа RabbitMQ/Kafka).

- *Аналогия:* Конструктор LEGO. Если сломалась деталь «Оплата», ты просто вытаскиваешь этот блок, чинишь его или заменяешь на новый, а все остальные блоки (каталог, корзина) продолжают работать как ни в чем не бывало.

Где применяется:

- В крупных, высоконагруженных проектах (Netflix, Spotify, Amazon, Яндекс, Сбер).
- В больших командах, где над каждым микросервисом может работать отдельная независимая группа разработчиков.

Как используется и тестируется:

Для QA-инженера микросервисы — это одновременно и круто, и сложно. Здесь как раз применяется **тестирование серого ящика**.

- **Интеграционное тестирование (Integration Testing):** Твоя главная задача — проверить, как микросервисы общаются между собой. Например, когда пользователь покупает товар, микросервис «Корзина» должен корректно передать ID товара в микросервис «Оплата», а тот, в свою очередь, — в микросервис «Склад», чтобы списать остатки.
- **Тестирование API:** Ты часто тестируешь микросервисы до того, как готов красивый сайт (фронтенд). Ты открываешь **Postman** и шлешь запросы напрямую в конкретный микросервис.
- **Изолированность багов:** Если падает микросервис «Рекомендации», пользователь всё равно сможет купить товар. Тебе, как QA, нужно проверить это пограничное состояние: корректно ли сайт скрывает сломавшийся блок, не ломая всю страницу.

Окружение в тестировании (Test Environment) — это, попросту говоря, отдельная копия твоего сайта или приложения, развернутая на специальных серверах.

Разработчики никогда не выкатывают код сразу на живой сайт, где сидят реальные клиенты. Сначала код проходит через «слоеный пирог» из различных окружений, чтобы баги вылавливались на безопасной территории.

Вот 4 классических вида окружений, через которые проходит любая фича на пути к релизу.

1. Local (Локальное окружение / «Локалка»)

Это личный компьютер конкретного разработчика. Он пишет код и запускает сайт прямо у себя на ноутбуке (например, по адресу `localhost:3000`).

- **Где применяется:** Для написания кода и первичной проверки.
- **Как используется QA:** Обычные тестировщики сюда практически не заходят. Разработчик сам крутит свой код, пишет модульные (Unit) тесты и проверяет, что всё хотя бы собирается.

2. DEV (Окружение разработки / Стенд разработки)

Когда разработчик закончил кодить, он отправляет свои изменения на общий сервер разработки — **DEV-стенд**.

- **Где применяется:** Для интеграции кода от разных программистов. Здесь всё постоянно меняется, код обновляется по несколько раз в день.
- **Как используется QA:** Стенд очень нестабильный. Ты можешь зайти туда, а там всё «лежит», потому что Вася только что залил сырой кусок кода. QA используют DEV для ранних, быстрых проверок, но полноценно тестировать здесь регресс нельзя — условия меняются слишком быстро.

3. QA / STAGE / TEST (Тестовое окружение / Стейджинг)

Это главное рабочее место QA-инженера. Это изолированный сервер, который полностью администрируется командой тестирования.

- **Где применяется:** Для проведения основного функционального, регрессионного и интеграционного тестирования.
- **Как используется QA:** Код сюда заливается стабильными порциями (например, раз в день или раз в спринт). Никакой разработчик не может прийти и втихаря что-то поменять без твоего ведома. Сюда заливают чистые тестовые данные, подключают заглушки (стабы) сторонних сервисов и гоняют автотесты. Если тесты прошли успешно здесь — фича готова к релизу.

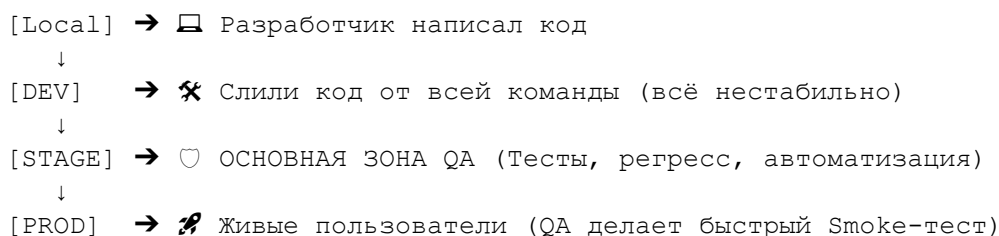
4. PROD (Продакшен / «Прод» / Живая среда)

Это тот самый боевой сайт или приложение, которым пользуются реальные люди (например, amazon.com или живое приложение твоего банка).

- **Где применяется:** Для реального бизнеса.
- **Как используется QA:** Тестировать на PROD — это огромный риск. Одно неосторожное действие, и ты сотрешь реальную базу данных или спишешь деньги у настоящего клиента. На проде QA проводят только **Smoke-тестирование** сразу после релиза (чтобы убедиться, что код долетел нормально и критические функции работают) и проверяют специфические баги, которые не воспроизводятся на тестовых стендах.

Важное правило: На ПРОДе тестируют только «подсматриванием» (Read-only) или используя специальные тестовые учетные записи, которые не влияют на финансовую и бизнес-отчетность.

Сводная цепочка движения фичи (Пайплайн):



Идемпотентность — это умное математическое слово, которое в IT означает очень простое и крутое свойство: **сколько бы раз ты ни совершал одно и то же действие с одинаковыми данными, результат всегда будет точно таким же, как от одного-единственного раза.**

Простая аналогия из жизни:

- **Идемпотентное действие:** Кнопка вызова лифта. Сколько бы раз ты на нее ни нажал (один раз или тридцать в панике), лифт просто приедет. Результат не изменится от количества нажатий.
- **НЕидемпотентное действие:** Кнопка отправки СМС или лифт, который за каждое нажатие списывал бы деньги. Нажал 5 раз — улетело 5 СМС, списалось 5 сумм. Результат зависит от количества повторений.

Где применяется идемпотентность?

Для QA-инженера это свойство критически важно в двух главных зонах: **в платежных системах и в дизайне REST API.**

1. Платежные системы и формы (Защита от двойных списаний)

Представь, что ты покупаешь билеты в кино. Ты нажал кнопку «Оплатить», и в этот момент у тебя моргнул интернет. Сайт завис. Ты в панике нажимаешь кнопку еще 3 раза.

- Если бэкенд сайта **НЕидемпотентный**, банк спишет с тебя деньги 4 раза и забронирует 4 места.
- Если бэкенд **идемпотентный**, система поймет, что эти 4 клика — одна и та же покупка (например, по уникальному ключу операции), проведет оплату только один раз, а на остальные клики просто ответит: «Уже оплачено, вот твой билет».

2. В REST API (Методы запросов)

Когда микросервисы общаются между собой или фронтенд общается с сервером, они используют HTTP-методы. По стандарту разработки часть из них **обязаны быть идиempотентными**:

- GET (Получить данные) — **Идиempотентный**. Ты можешь открыть страницу профиля 100 раз подряд. Данные на сервере от этого не изменятся, ты просто читаешь их.
- PUT (Обновить данные целиком) — **Идиempотентный**. Если ты обновляешь имя пользователя на «Иван» (`PUT /user/1 {name: "Иван"}`), то сколько бы раз этот запрос ни долетел до сервера, имя останется «Иван».
- DELETE (Удалить данные) — **Идиempотентный**. Первый запрос удалит объект (сервер ответит 200 OK или 204 No Content). Следующие 10 запросов уже ничего не удалят, так как объекта нет (сервер вернет 404 Not Found), но глобальное состояние системы не изменилось — объект стерт, цель достигнута.
- POST (Создать новые данные) — **НЕИДЕМПОТЕНТНЫЙ**. Каждый раз, когда ты шлешь `POST /user/create`, на сервере создается *новый* пользователь. Отправишь 5 раз — в базе появится 5 одинаковых Иванов с разными ID.

Как это используется и тестируется QA-инженером?

Сильный QA всегда проверяет систему на устойчивость к повторным действиям. Вот как это протестировать:

1. **Тестирование API в Postman (Повторные запросы)**: Ты берешь запрос создания заказа или оплаты и отправляешь его несколько раз подряд (Double Submit). Ты должен убедиться, что система выдает ошибку (например, 409 Conflict) или корректно игнорирует дубли, не создавая вторую запись в базе данных.
2. **Тестирование UI (Защита «от дурака»)**: Нажми на кнопку отправки формы регистрации быстро-быстро несколько раз (кликер-тест). Хороший фронтенд должен блокировать (делать серой/disabled) кнопку сразу после первого клика, пока крутится лоадер. Но если блокировку обошли, бэкенд все равно обязан подстраховать своей идиempотентностью.
3. **Использование уникальных токенов (Idempotency Key)**: В крутых системах при оформлении заказа фронтенд генерирует случайную строку — Idempotency Key (ключ идиempотентности) — и прикрепляет её к запросу. Сервер запоминает этот ключ. Если в течение пары минут прилетает еще один запрос с таким же ключом, сервер понимает: «Ага, это дубль из-за лага сети, выполнять заново не буду, просто отдам прошлый ответ». QA проверяет, как сервер реагирует на запросы с одинаковыми и разными ключами.

Кэширование — это процесс сохранения копии данных во временное и очень быстрое хранилище (кэш), чтобы при повторном запросе выдать их пользователю мгновенно, не тратя время на долгие вычисления или тяжелые запросы к базе данных.

Простая аналогия: ты сидишь за рабочим столом и учишь термины для собеседования.

- **Без кэширования:** Каждый раз, когда тебя спрашивают термин, ты встаешь, идешь в другую комнату, ищешь книгу на полке, листаешь ее, находишь страницу, читаешь и возвращаешься. Это долго.
- **С кэшированием:** Ты один раз сходил за книгой, выписал самые частые термины на листочек и положил перед собой на стол. Теперь ответ ты даешь за секунду. Этот листочек — твой кэш.

Где применяется кэширование?

Кэширование используется на **всех уровнях** современных IT-систем, от твоего браузера до огромных серверов:

1. В браузере пользователя (Client Cache / Browser Cache)

Когда ты первый раз заходишь на сайт, твой браузер скачивает «тяжелые» файлы: логотип, шрифты, стили оформления (CSS) и картинки. При переходе на другую страницу браузер берет эти файлы из памяти твоего компьютера (из кэша), а не качает заново из интернета. Сайт открывается моментально.

2. На сервере (Server-Side Cache)

Представь интернет-магазин, у которого на главной странице висит блок «Топ-10 товаров сегодня». Чтобы его собрать, серверу нужно сделать сложный SQL-запрос, перебрать миллион строк в базе данных и посчитать статистику.

- *Решение:* Сервер делает этот расчет один раз в час и сохраняет результат в супербыструю базу данных в оперативной памяти (например, **Redis** или **Memcached**). Следующие 100 000 пользователей получают этот список мгновенно прямо из памяти Redis, а основная база данных будет «отдыхать».

3. Сети доставки контента (CDN — Content Delivery Network)

Если сервера твоего сайта физически стоят в Париже, а пользователь открывает его из Токио, картинки будут лететь через весь океан долго. CDN — это сеть промежуточных серверов по всему миру. Они кэшируют (копируют) картинки твоего сайта на сервер в Токио, и японский пользователь скачивает их со скоростью света из своего же города.

Как это используется и тестируется QA-инженером?

Кэш — прекрасная штука для скорости, но это **главный источник запутанных багов**. Самая большая боль в IT: «как сделать так, чтобы кэш вовремя обновлялся (инвалидация кэша), если реальные данные изменились?».

Вот типичные сценарии, как тестировщик работает с кэшированием:

1. Тестирование сброса кэша на фронтенде (Hard Reload)

Разработчики обновили дизайн сайта, поменяли стили кнопок с синих на зеленые. Ты заходишь на сайт — а кнопки всё еще синие. Твой браузер «залип» на старом кэше.

- *Как использовать:* QA проверяет сайт, очистив кэш (в Chrome это делается через `Ctrl + F5` или `Cmd + Shift + R`). Также сильный QA проверяет, настроили ли разработчики автоматический сброс кэша для обычных пользователей (обычно к именам файлов стилей добавляют хэш, например `style.v2.css`).

2. Тестирование инвалидации кэша на бэкенде

Ты тестируешь админку сайта. Заходишь в настройки товара и меняешь цену с 500 рублей на 400 рублей. Сохраняешь. Открываешь сайт как покупатель — цена всё еще 500 рублей. Почему? Потому что бэкенд отдал тебе старую закэшированную страницу товара.

- *Как использовать:* Твой тест-кейс должен включать проверку: «Изменить данные в админке → Проверить отображение на клиенте → Данные должны обновиться сразу (кэш должен сброситься принудительно при сохранении)».

3. Тестирование API (Заголовки кэширования)

В REST API кэшированием управляют специальные HTTP-заголовки (Headers), которые сервер присылает браузеру (например, `Cache-Control: max-age=3600` говорит: «Этот ответ можно кэшировать на 1 час»). Ты открываешь Postman или DevTools и проверяешь, корректно ли проставлены эти заголовки для динамических и статических данных.